CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Hashing

Goals:

We want to define a **keyspace**, a (mathematical) description of the keys for a set of data.

…use a function to map the **keyspace** into a small set of integers.

Hashing

Goals:

We want to define a **keyspace**, a (mathematical) description of the keys for a set of data.

…use a function to map the **keyspace** into a small set of integers.

Hashing

Goals:

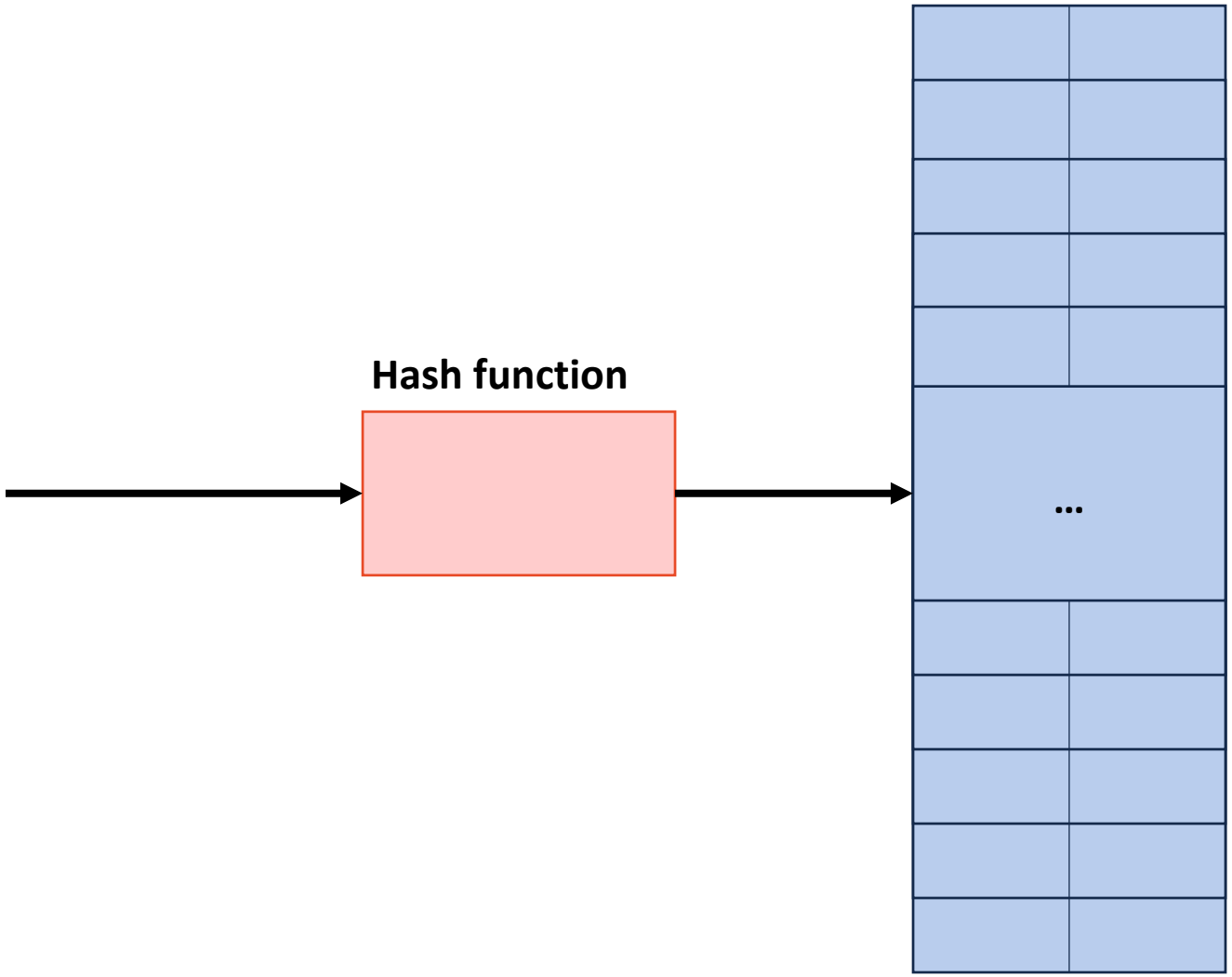
We want to define a **keyspace**, a (mathematical) description of the keys for a set of data.

K

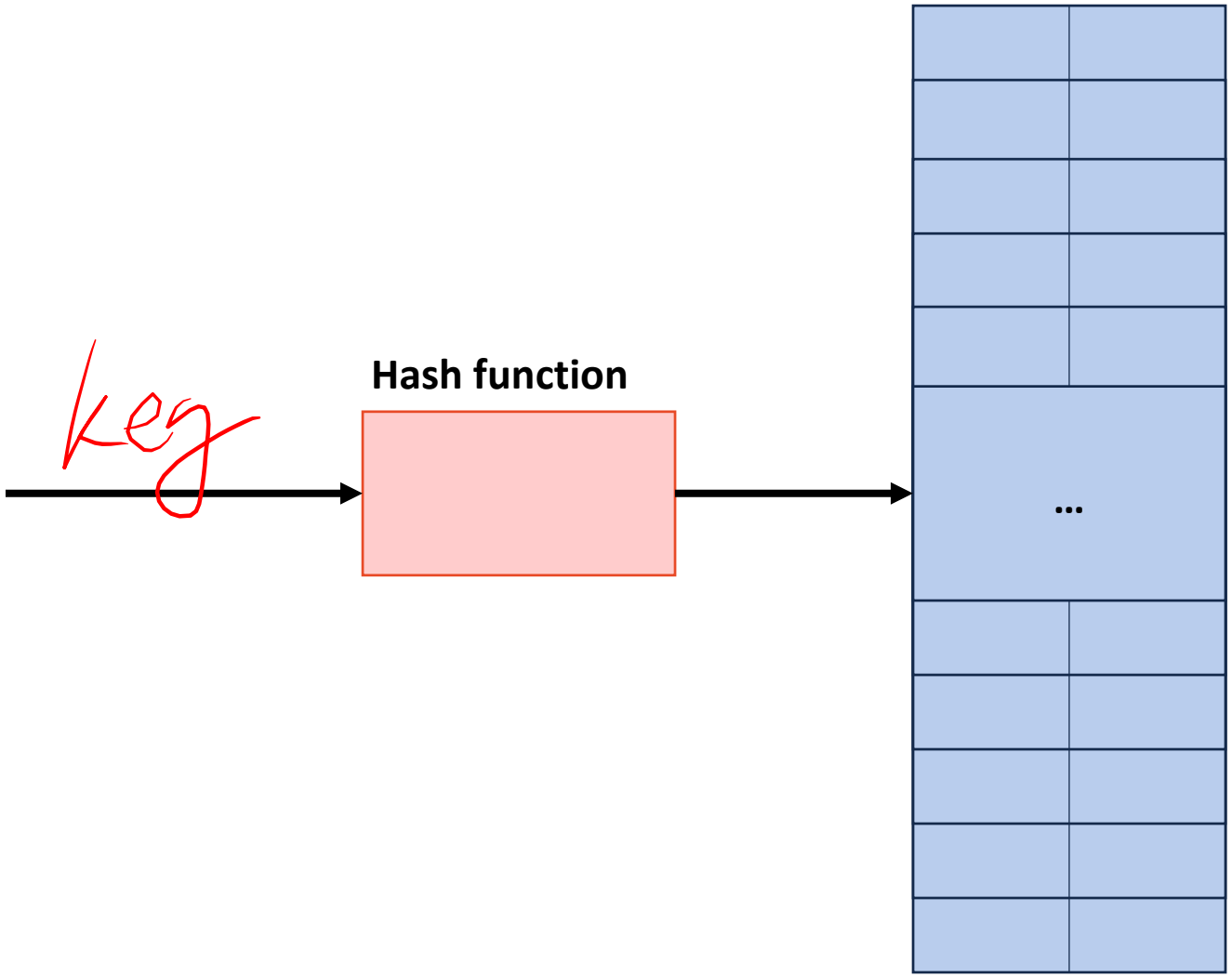
…use a function to map the **keyspace** into a small set of integers.

h(key) : key → N

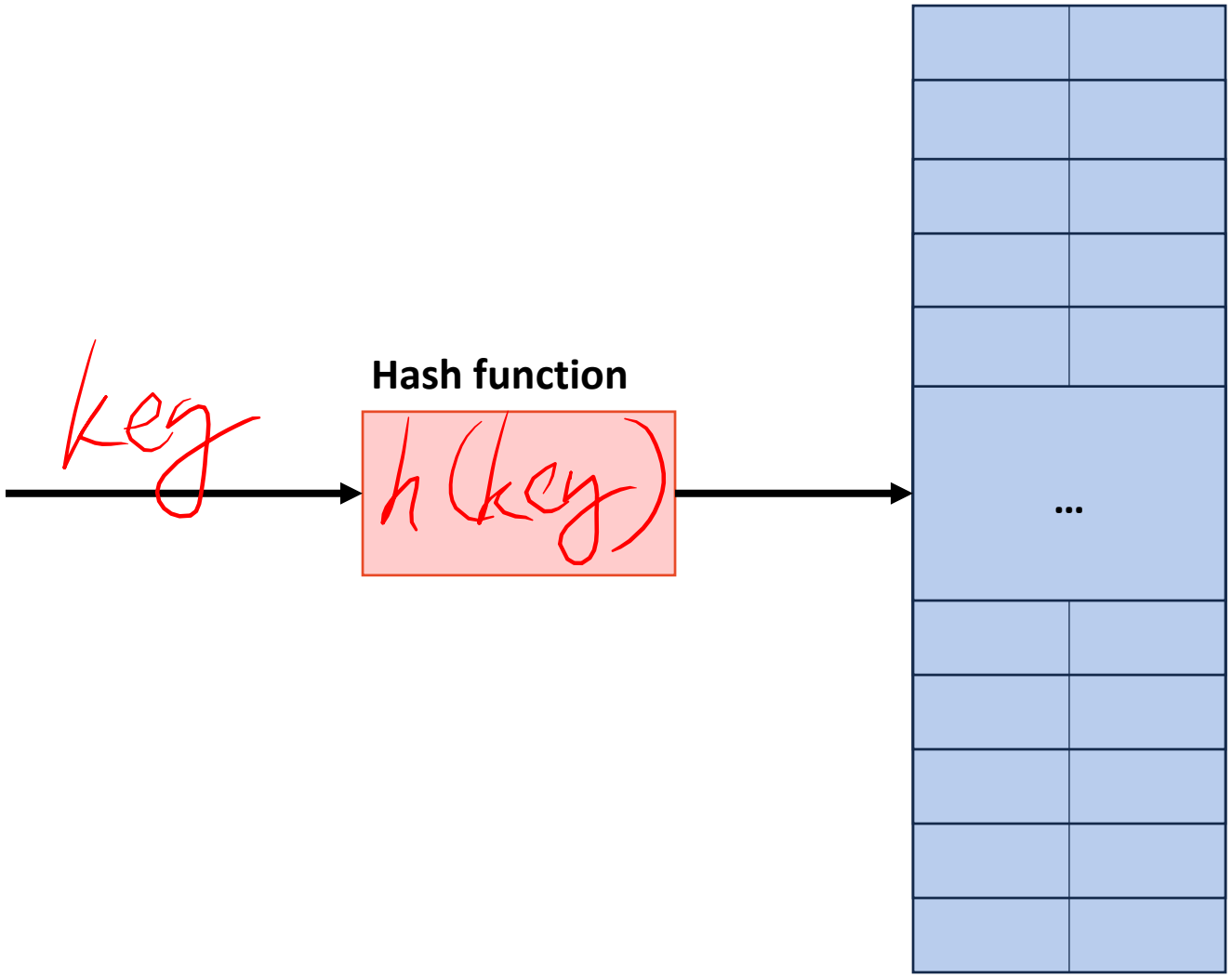
Hashing



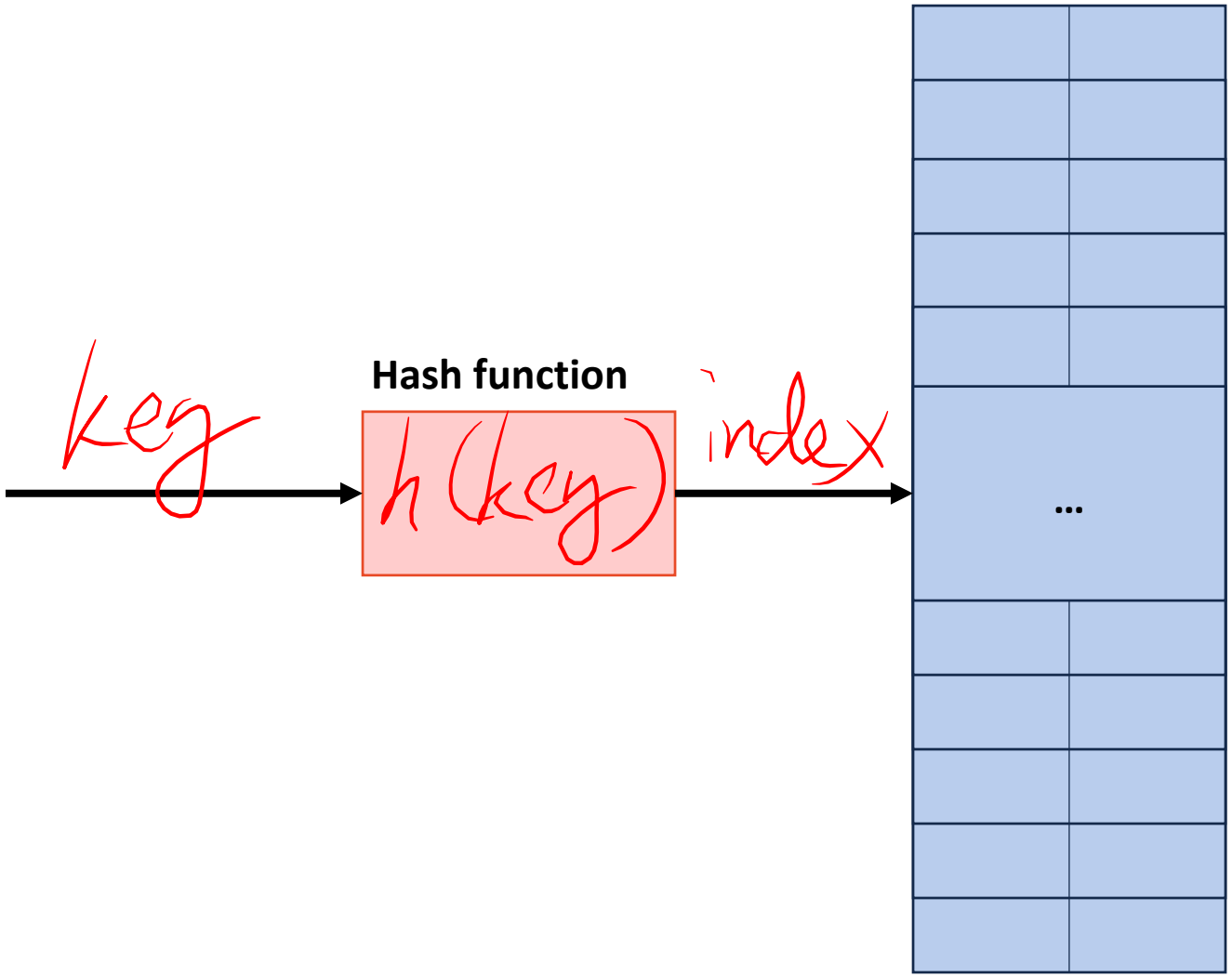
Hashing



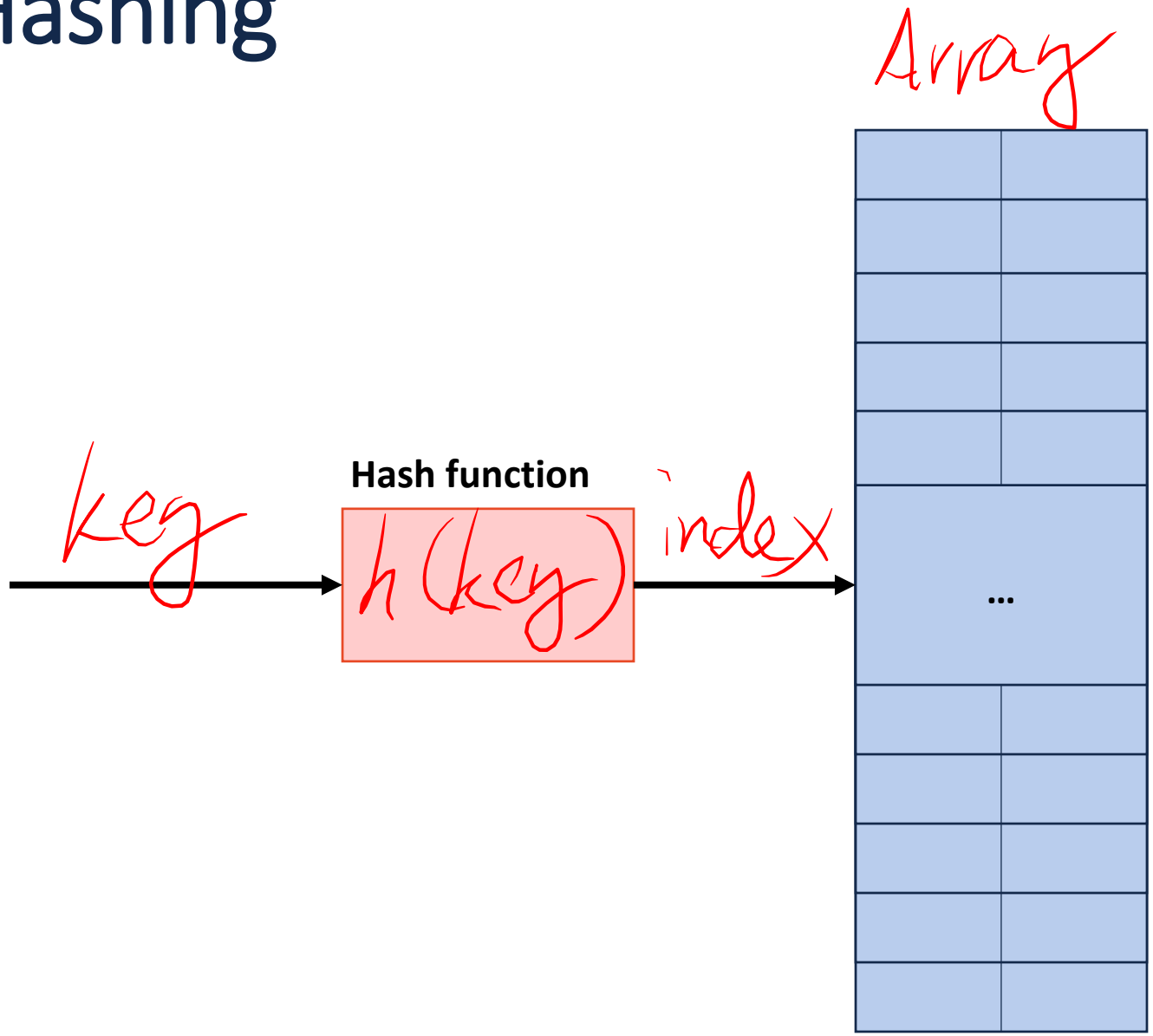
Hashing



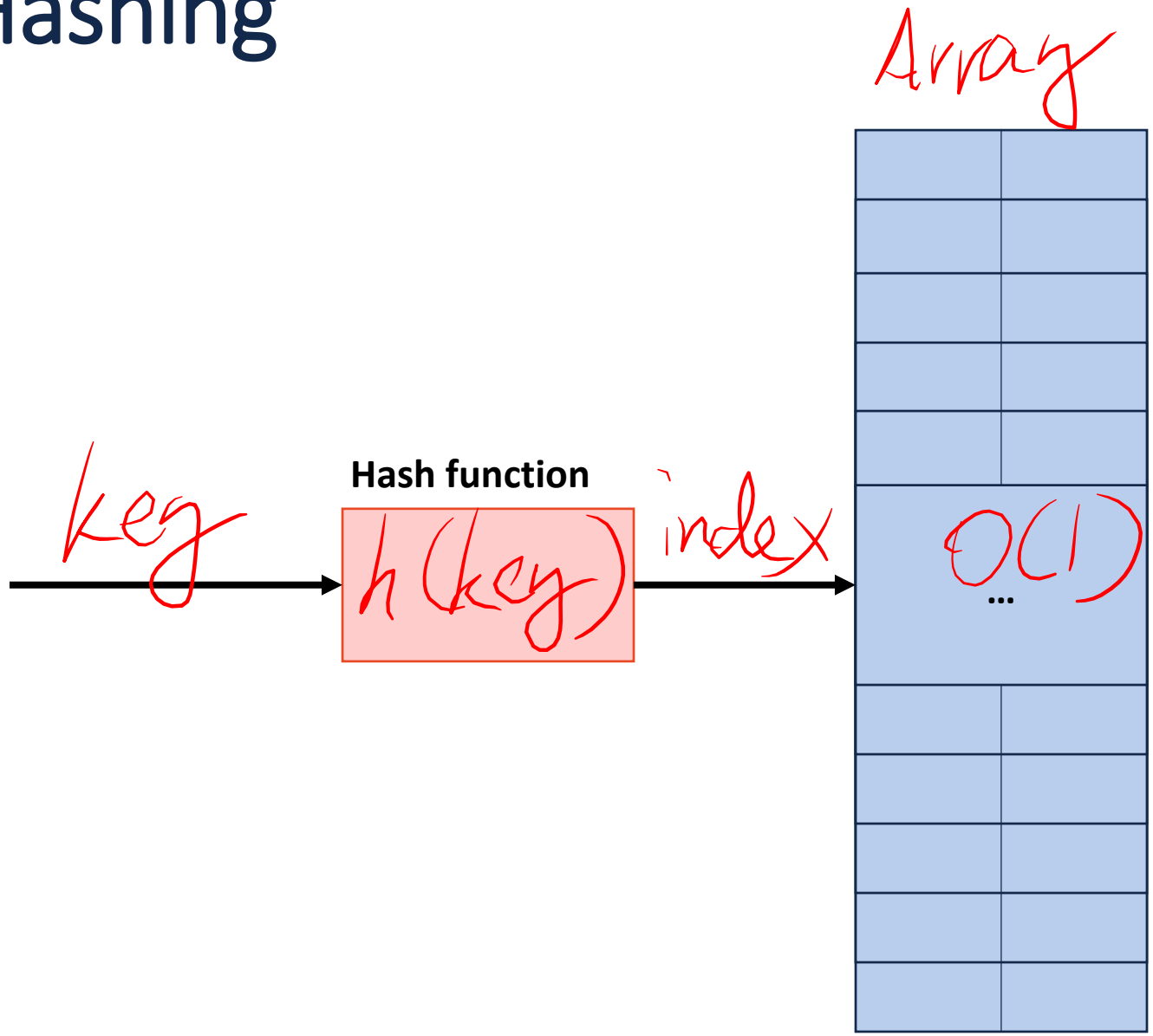
Hashing



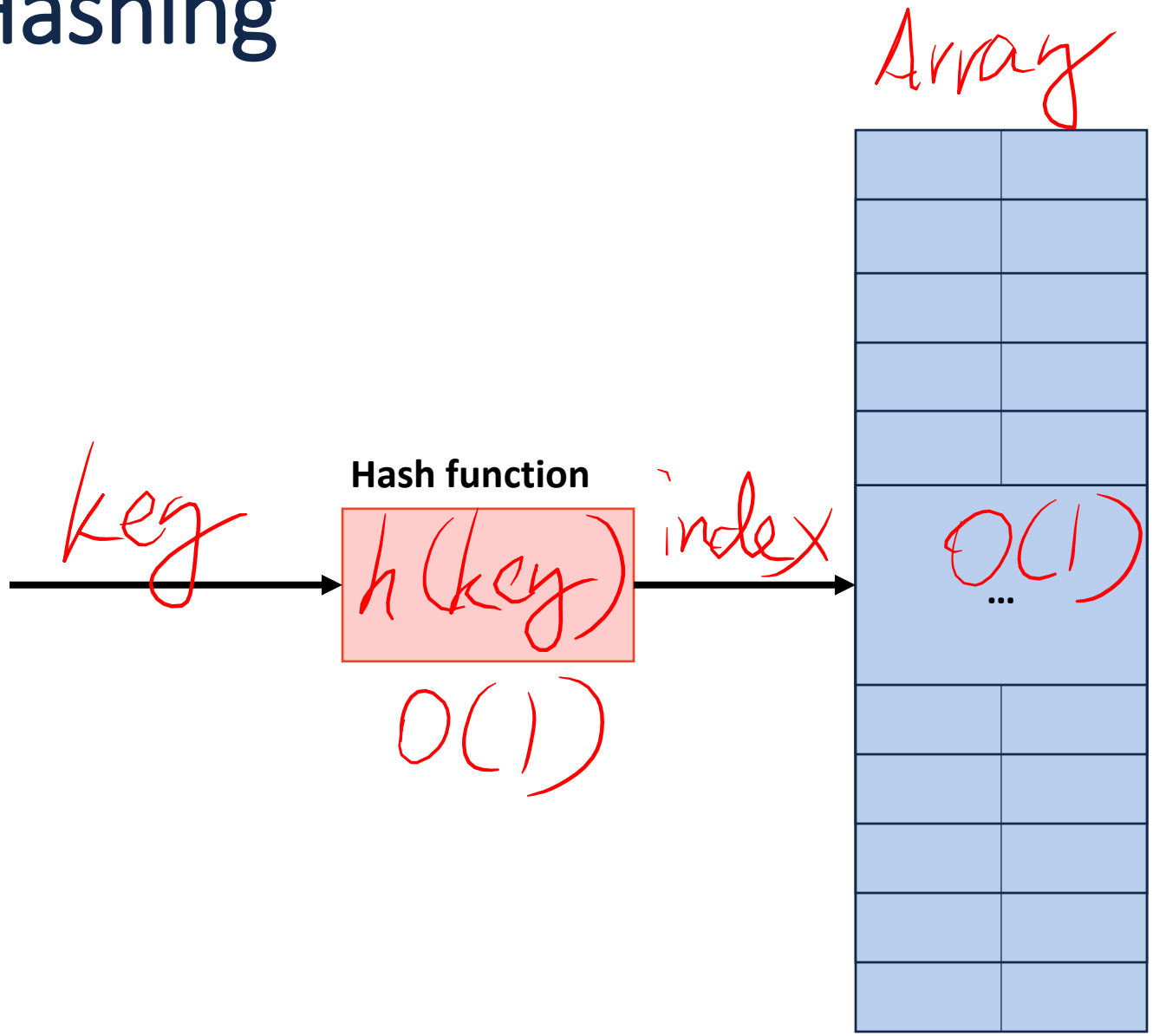
Hashing



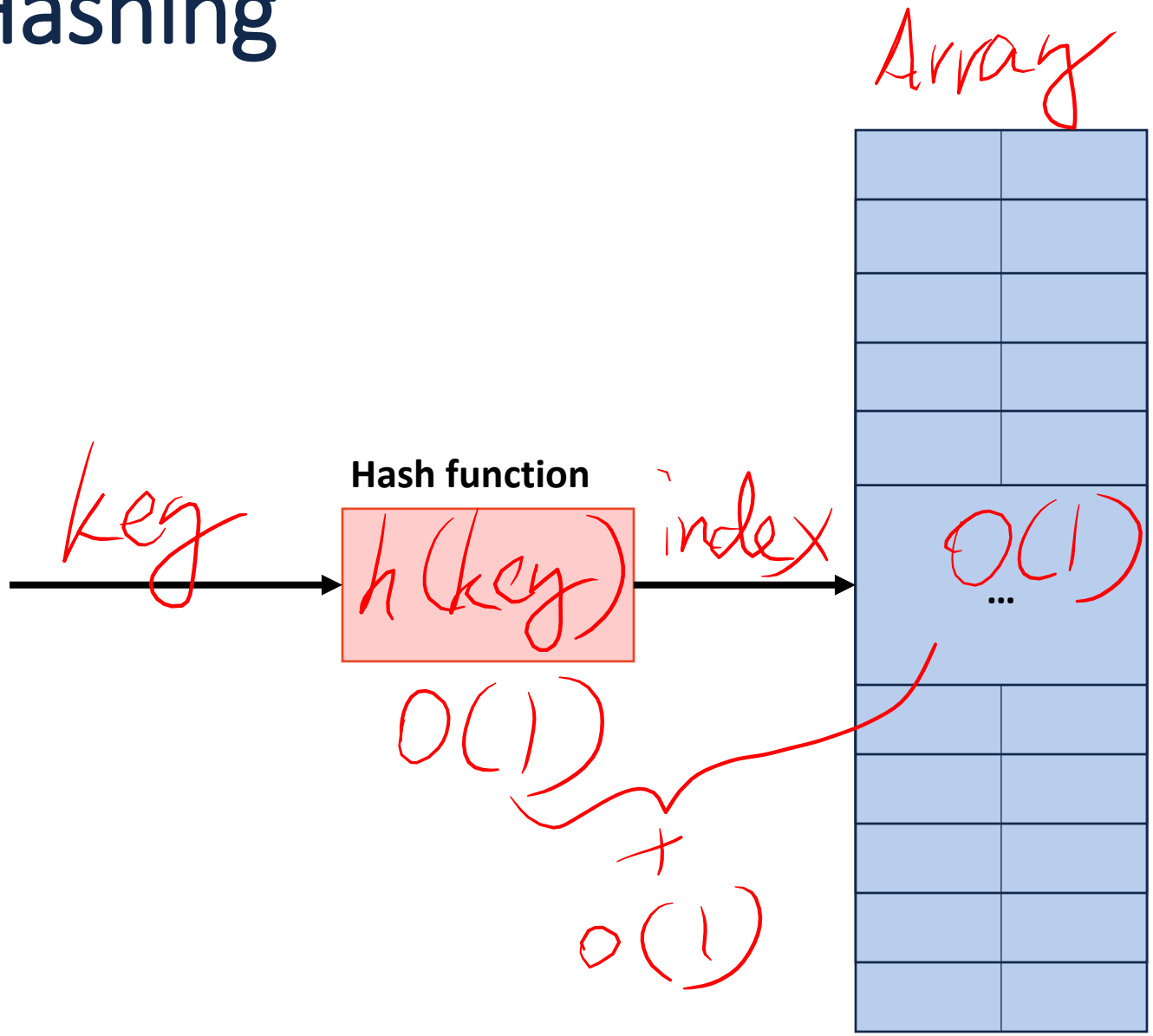
Hashing



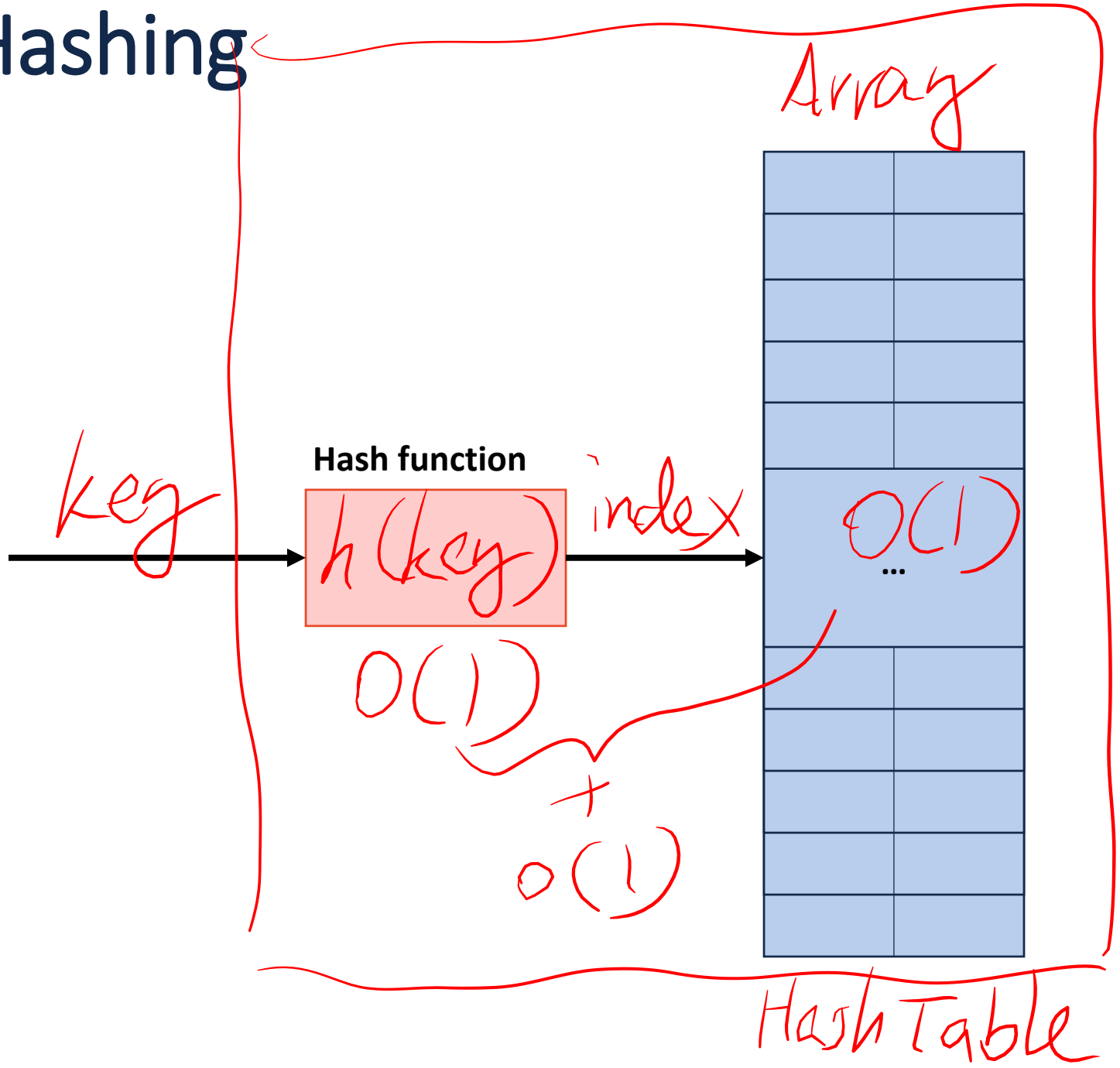
Hashing



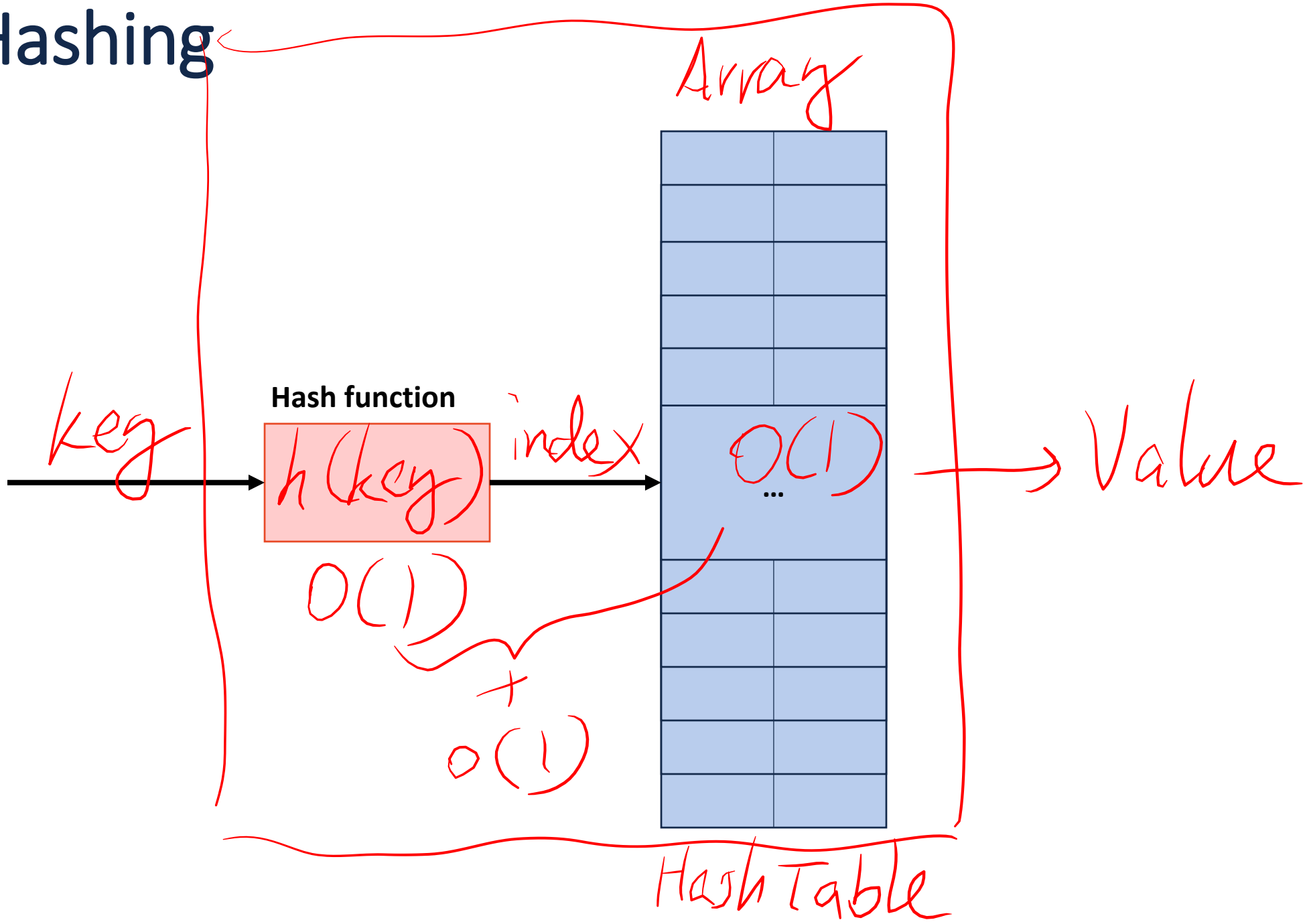
Hashing



Hashing



Hashing



A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1.

2.

3.

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1. A hash function
2.
3.

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1. A hash function
2. An array
3.

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1. A hash function
2. An array
3. ?? Mystery

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1. A hash function ☆
2. An array
3. ?? Mystery

A Hash Table based Dictionary

Client Code:

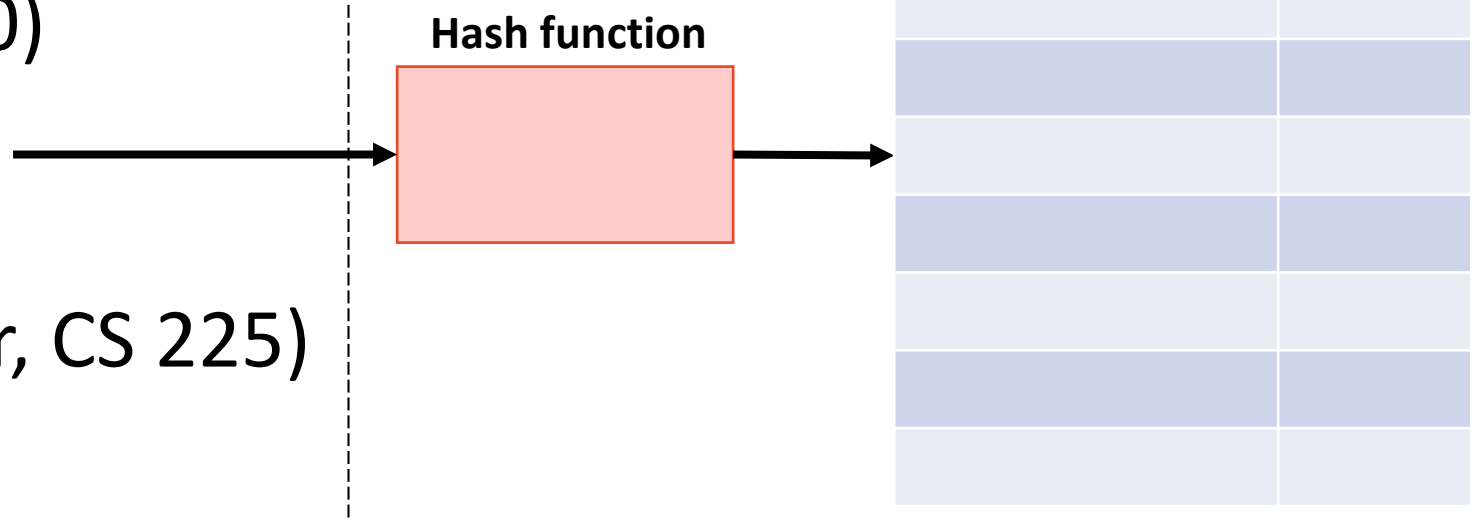
```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1. A hash function ☆
2. An array
3. ?? Mystery }

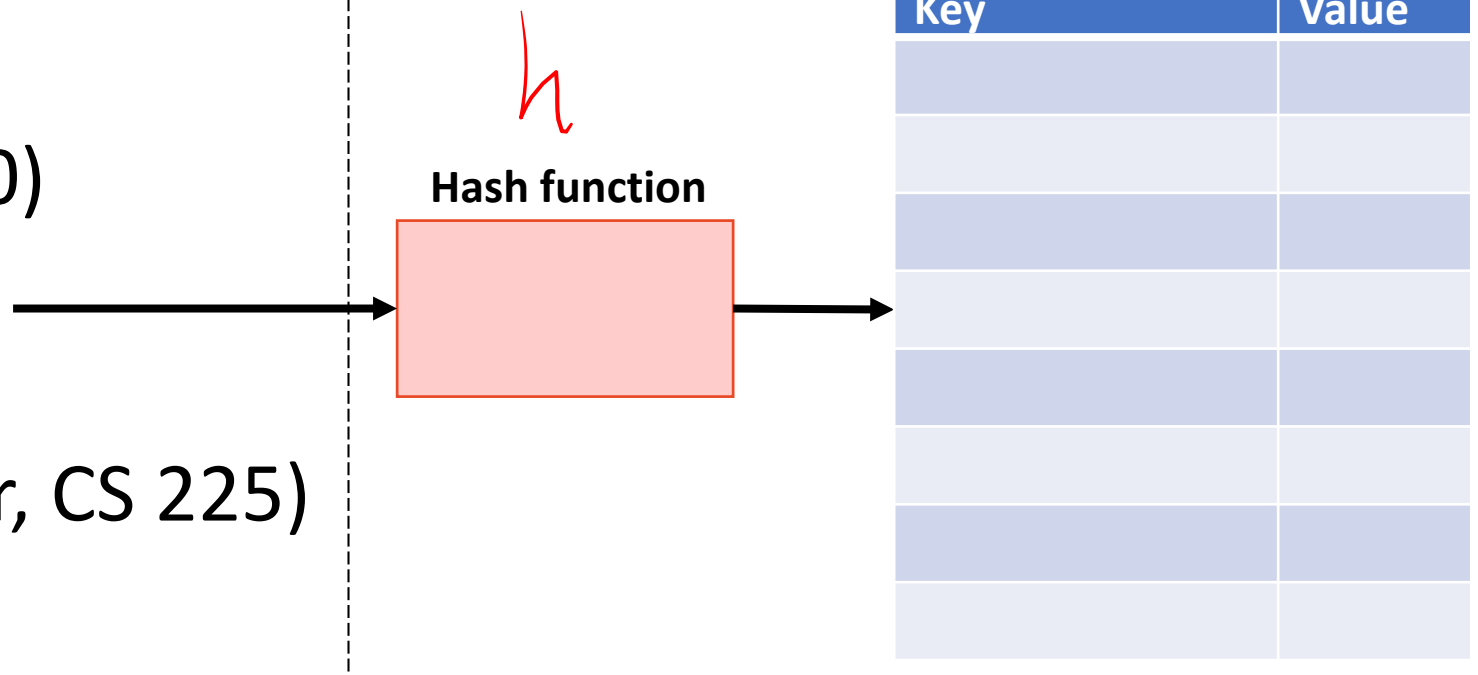
A Perfect Hash Function

(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



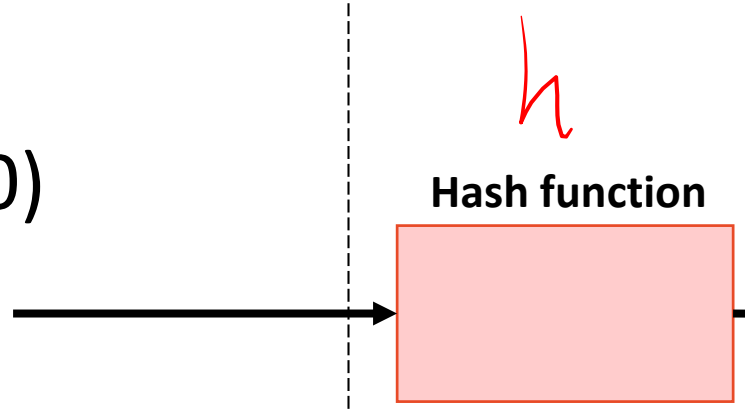
A Perfect Hash Function

(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



A Perfect Hash Function

(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)

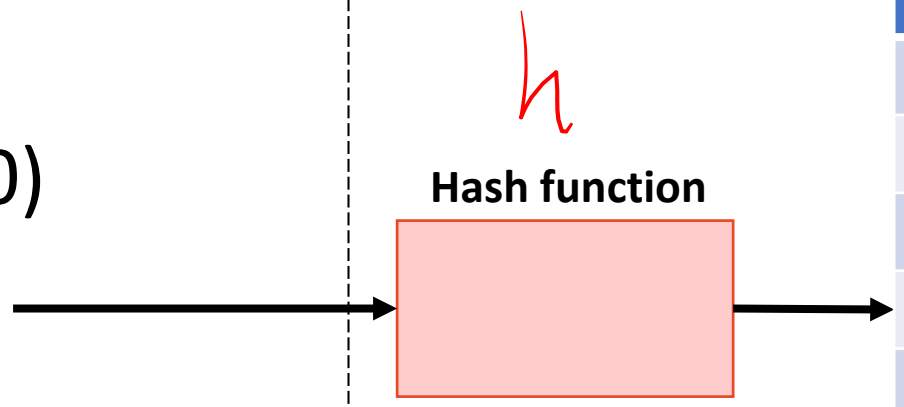


Key	Value
0	Angrave
1	Beckman
2	
3	
4	
5	
6	
7	

index

A Perfect Hash Function

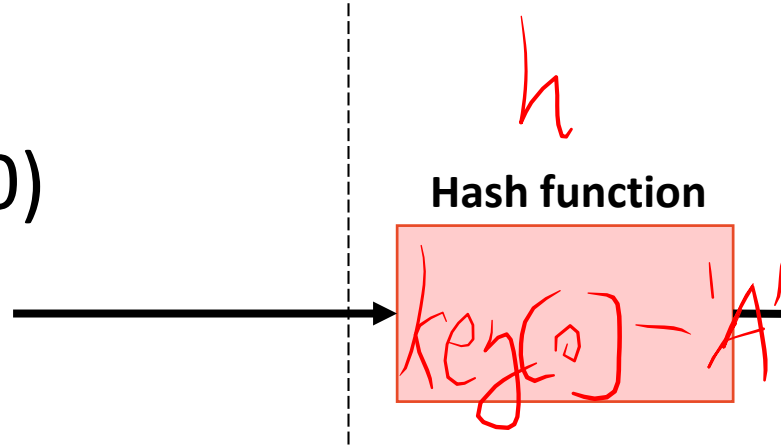
(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



Key	Value
0	Angrave
1	Beckman
2	
3	
4	
5	
6	
7	

A Perfect Hash Function

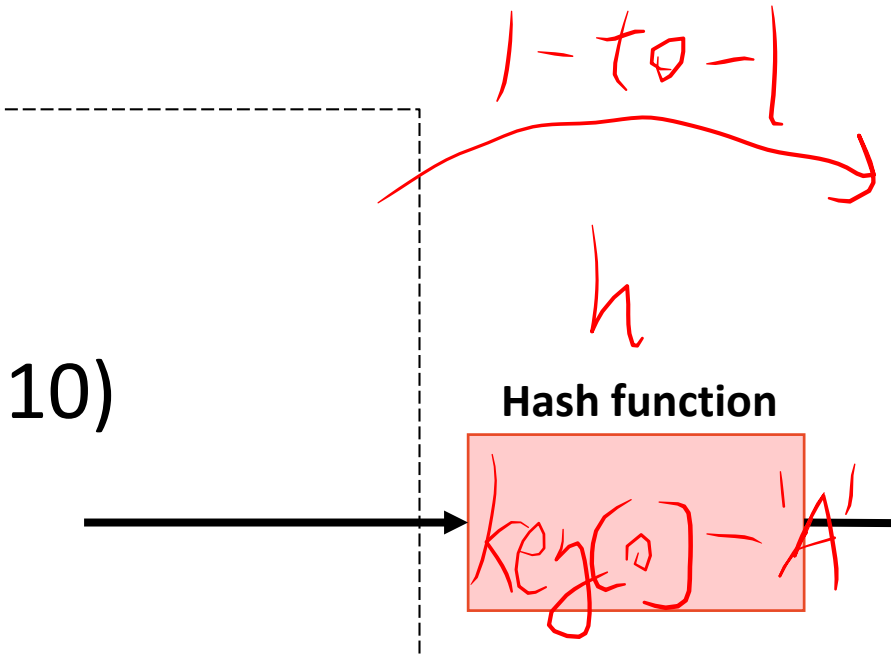
(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



Key	Value
0	Angrave
1	Beckman
2	
3	
4	
5	
6	
7	

A Perfect Hash Function

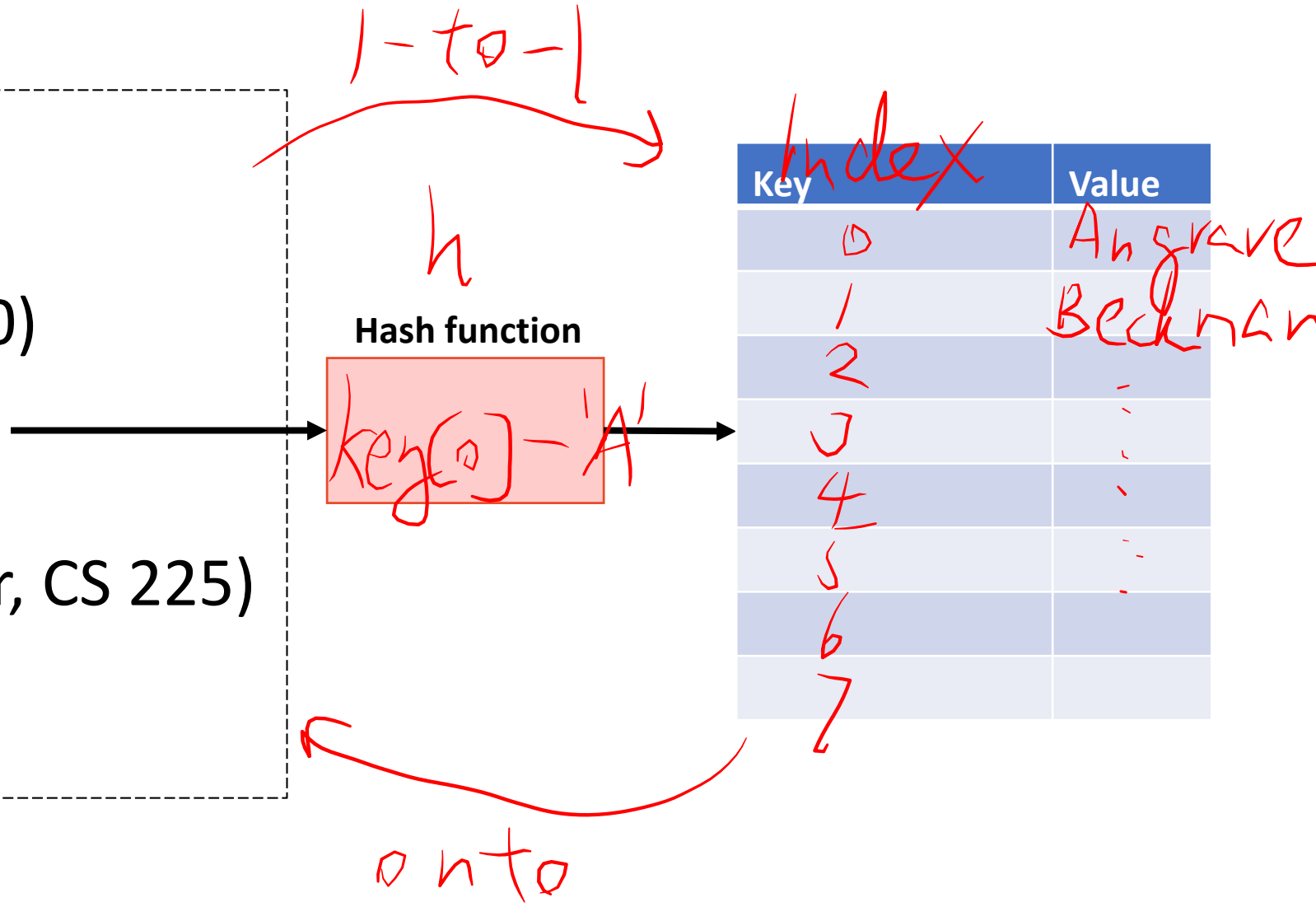
(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



Key	Value
0	Angrave
1	Beckman
2	
3	
4	
5	
6	
7	

A Perfect Hash Function

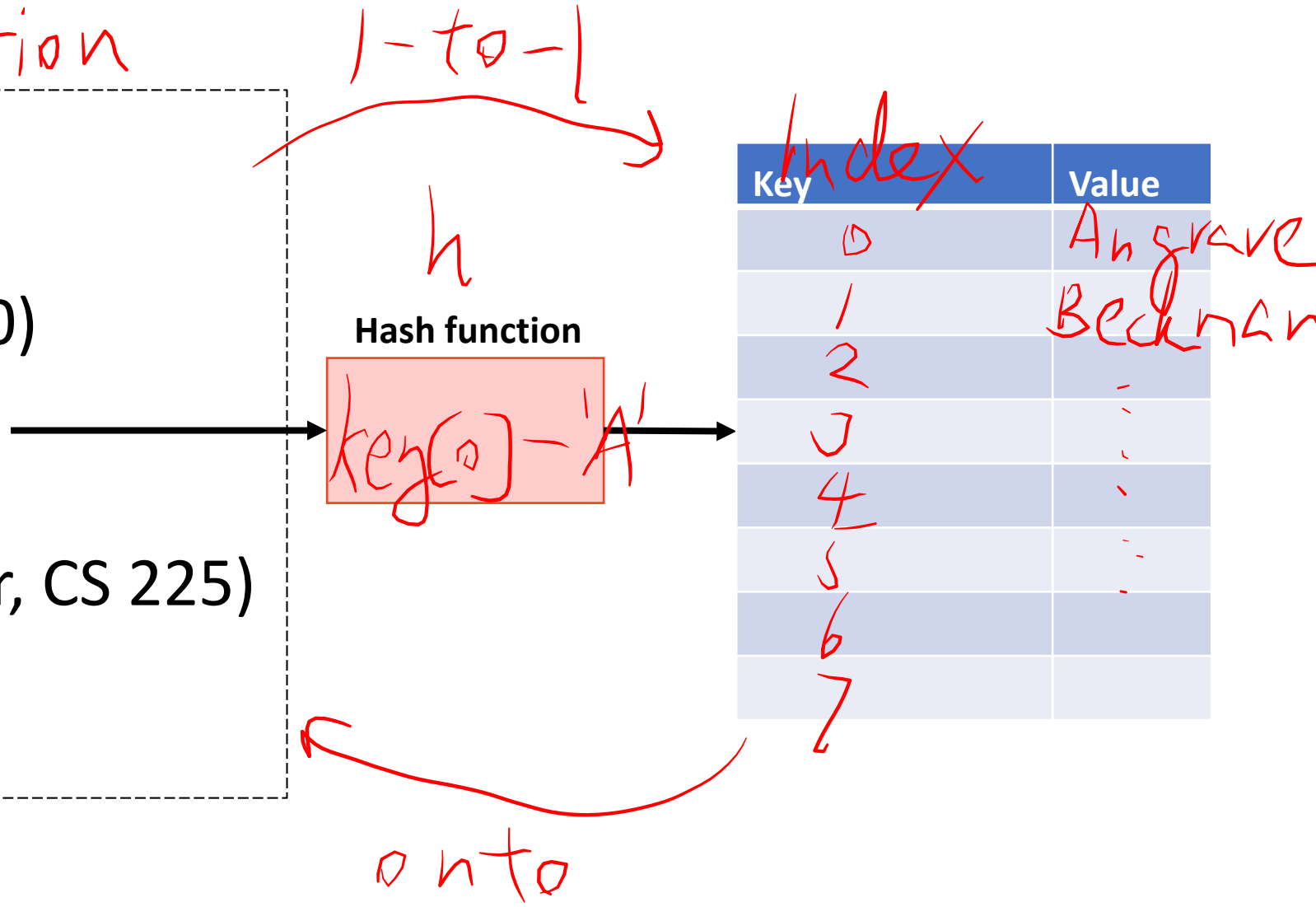
(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



A Perfect Hash Function

bijective function

(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



A Perfect Hash Function

bijective function

(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)

1-to-1

h

Hash function

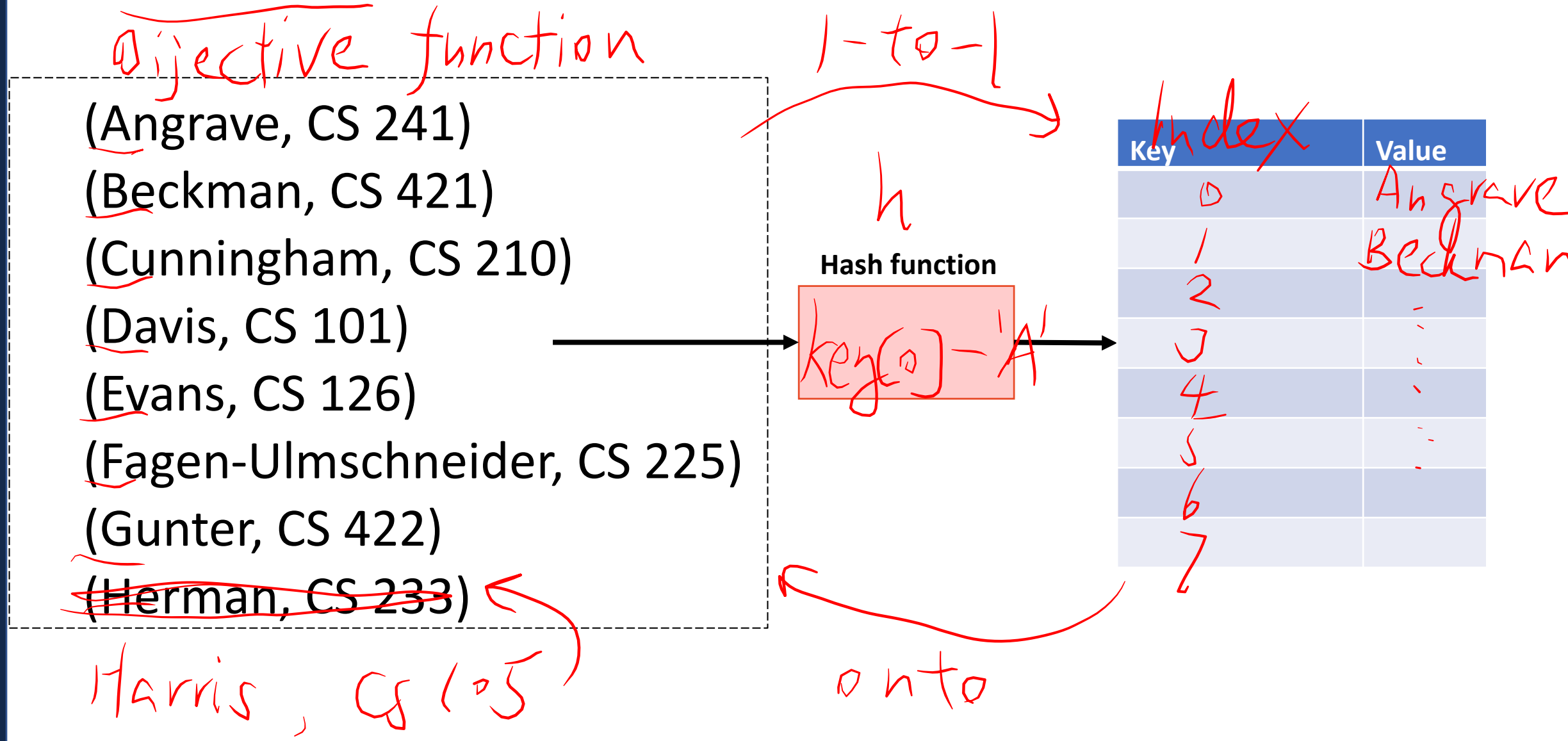
key[0] - 'A'

onto

Key (index)	Value
0	Angrave
1	Beckman
2	
3	
4	
5	
6	
7	

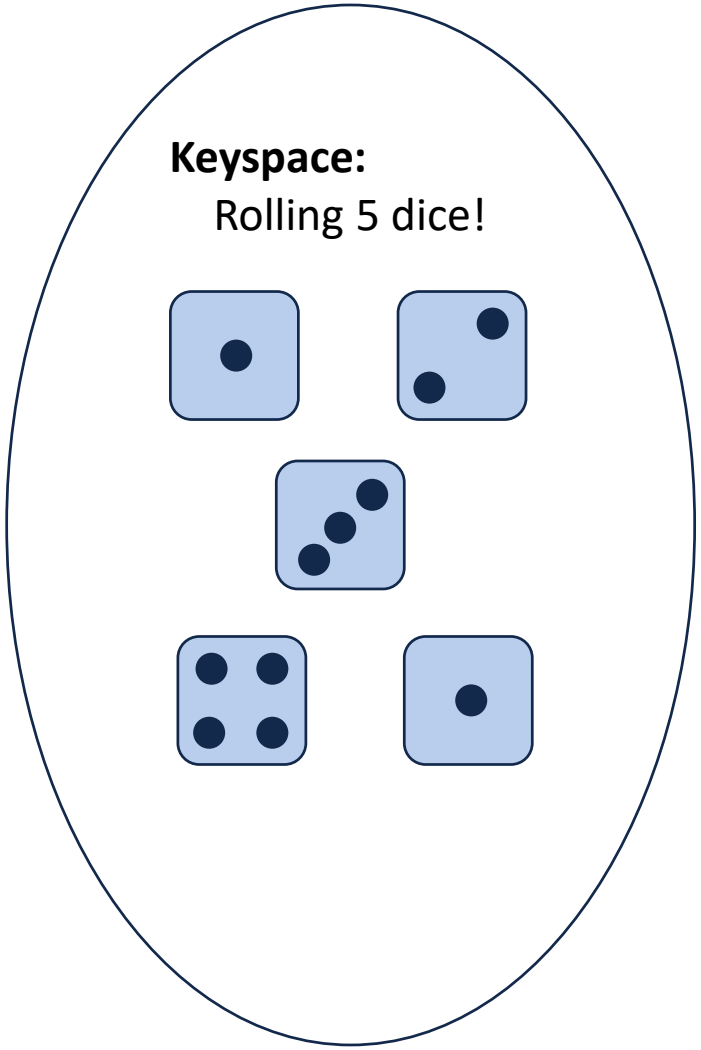
Harris, CS 105

A Perfect Hash Function



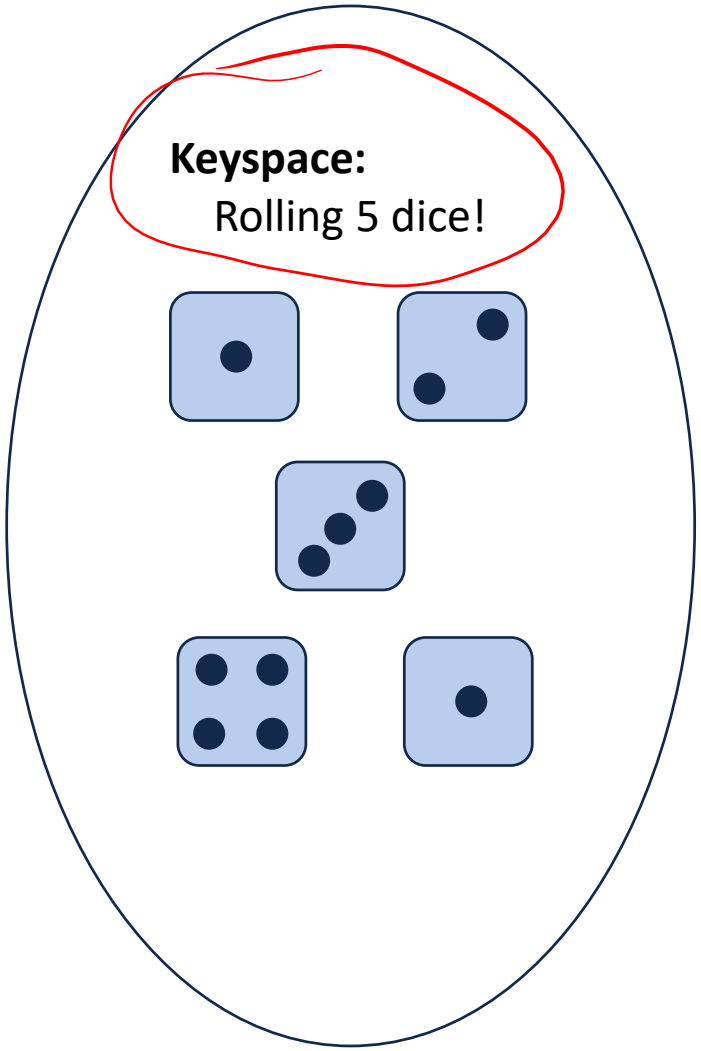
A Perfect Hash Function

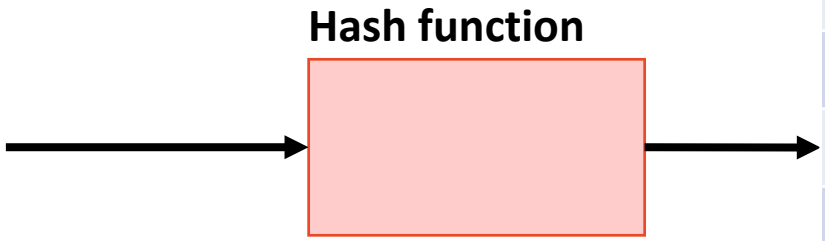
https://www.random.org/dice/?num=5



Key	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

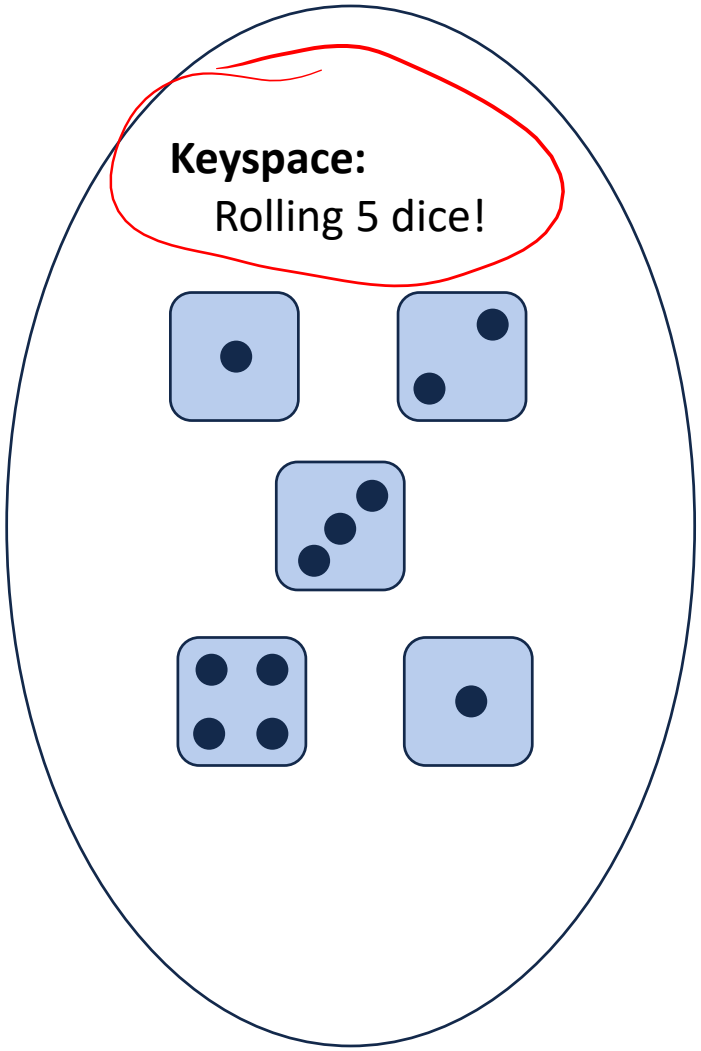
A Perfect Hash Function

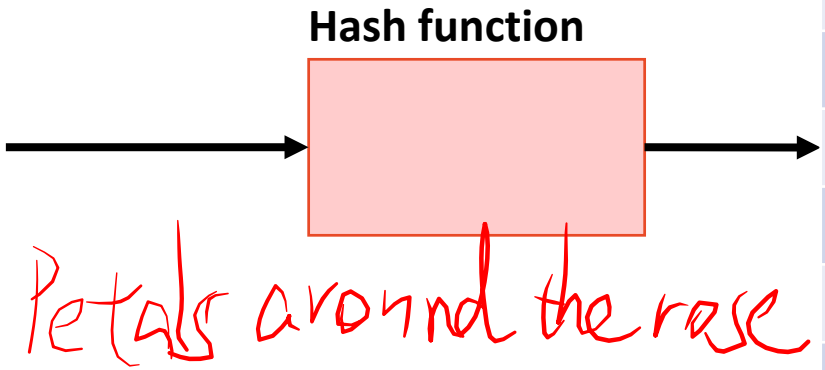




Key	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

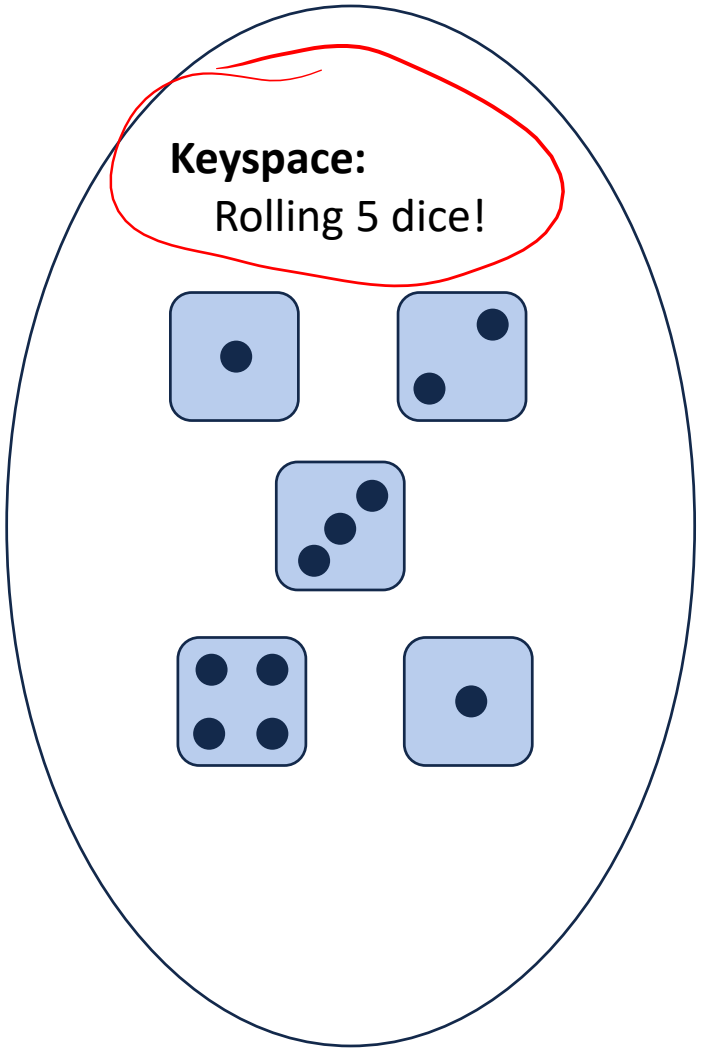
A Perfect Hash Function

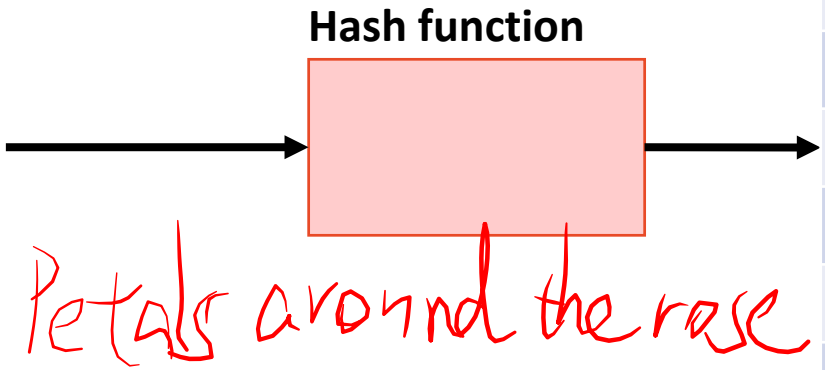




Key	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

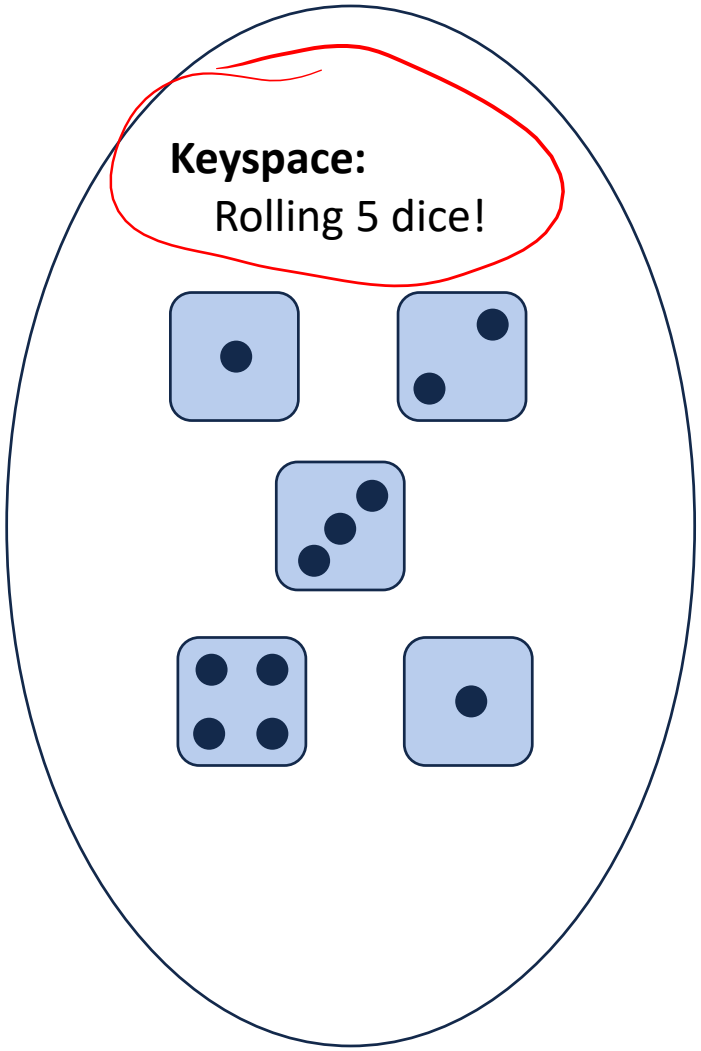
A Perfect Hash Function

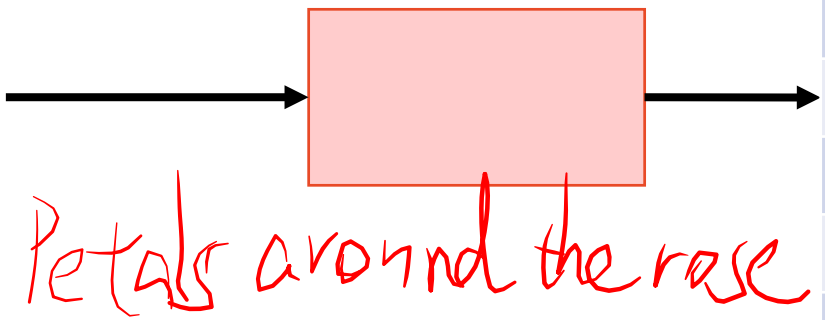




Key	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

A Perfect Hash Function

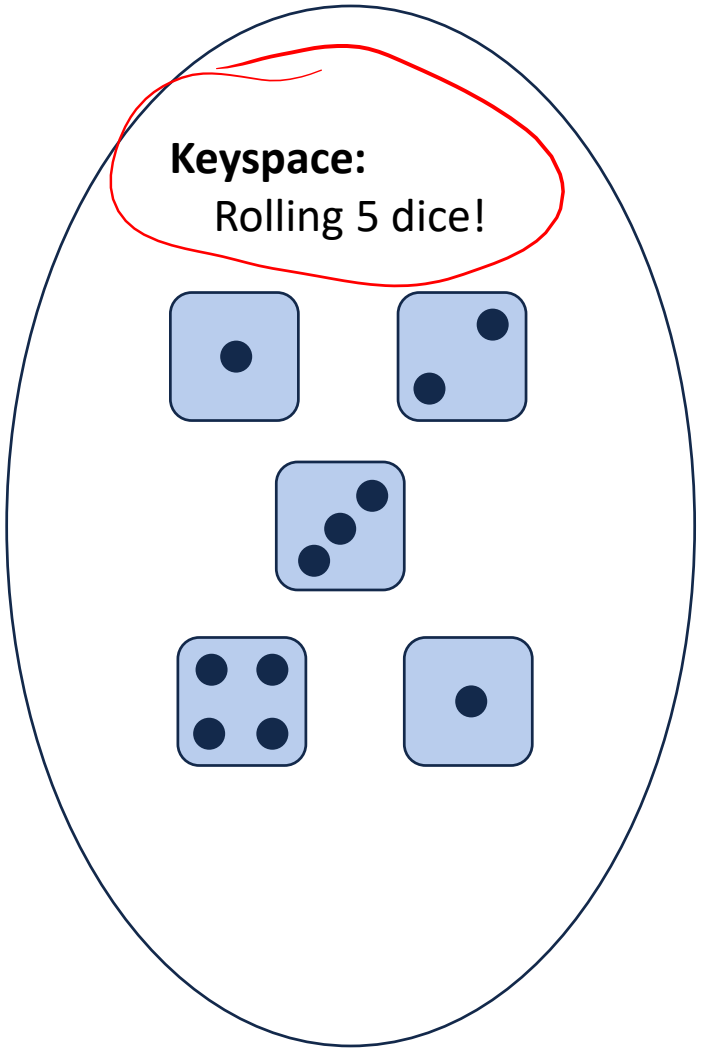


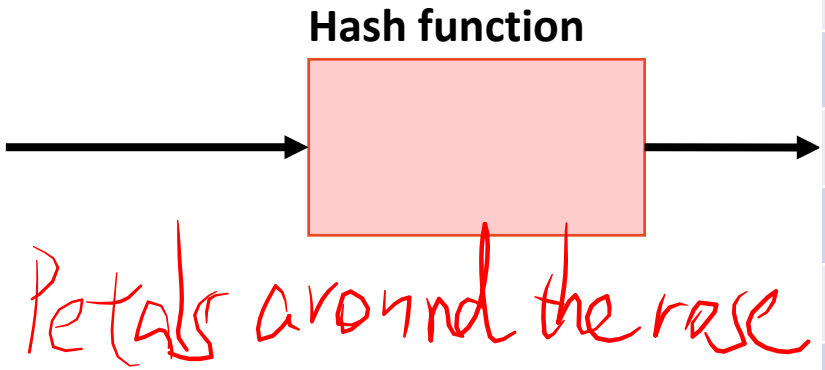


Petals around the rose

Key	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

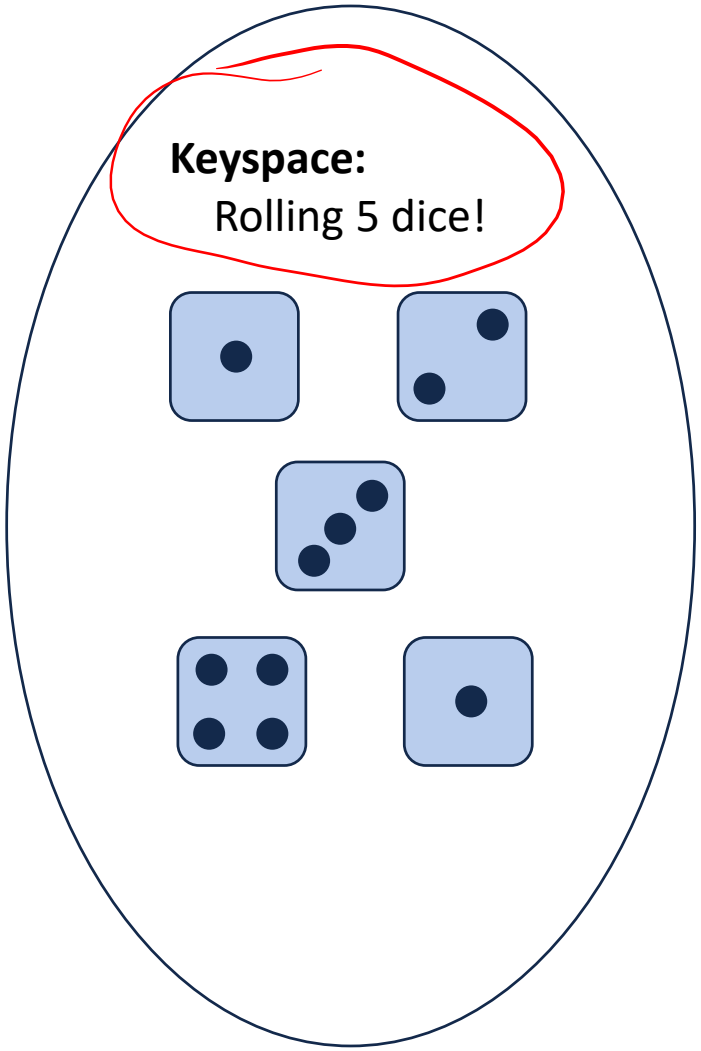
A Perfect Hash Function

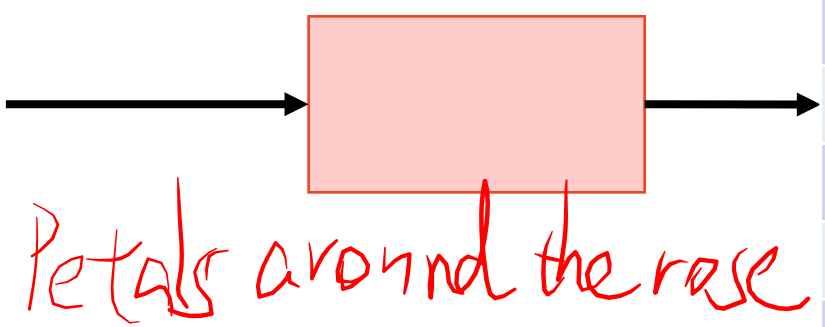




Key	Value
0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

A Perfect Hash Function

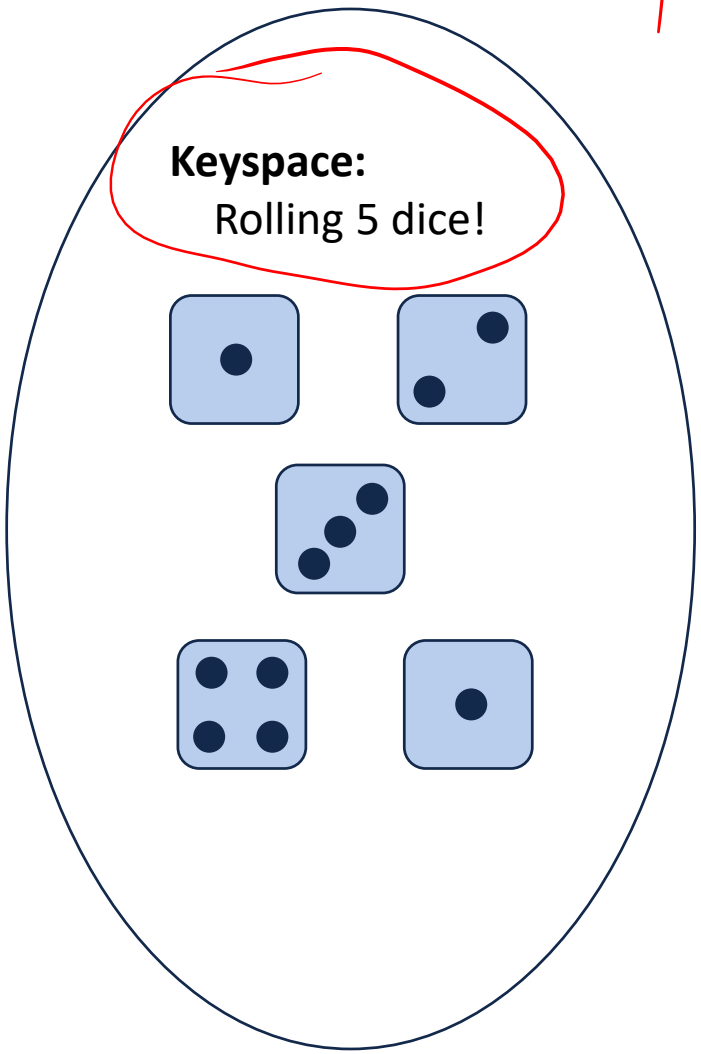




Key	Value
0	✓
1	✗
2	✓
3	✗
4	
5	✗
6	
7	✗
8	
9	✗
10	
11	✗
12	
13	✗
14	
15	✗

A Perfect Hash Function

1-to-1



Hash function

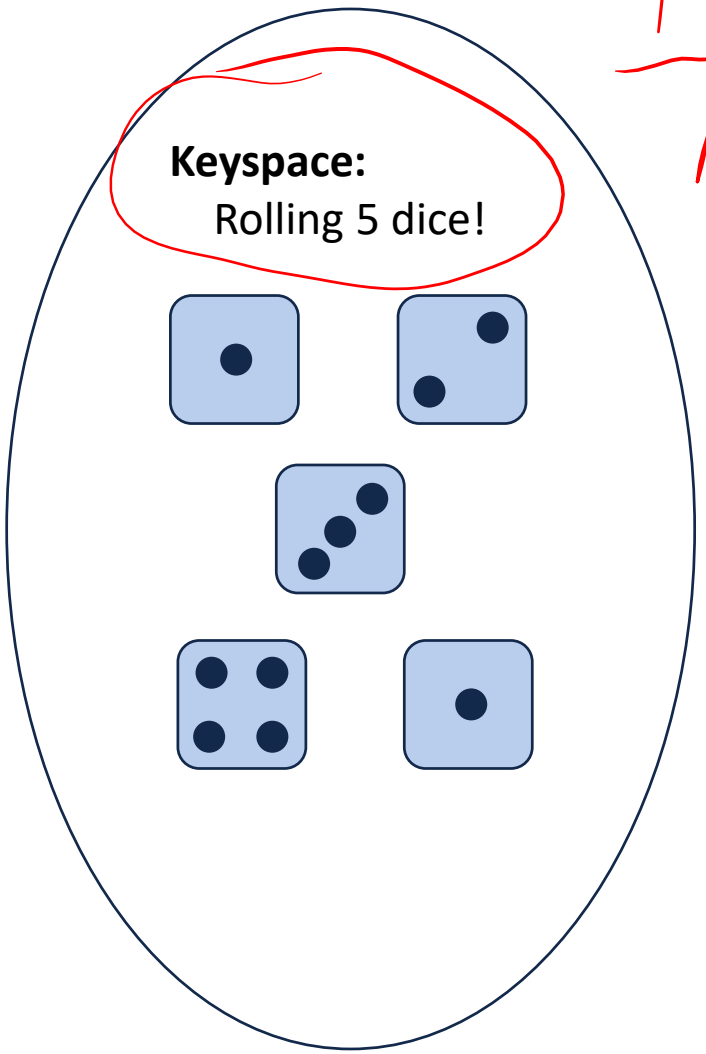
Petals around the rose

Key	Value
0	✓
1	✗
2	✓
3	✗
4	
5	✗
6	
7	✗
8	
9	✗
10	
11	✗
12	
13	✗
14	
15	✗

A Perfect Hash Function

1-to-1

No



Hash function

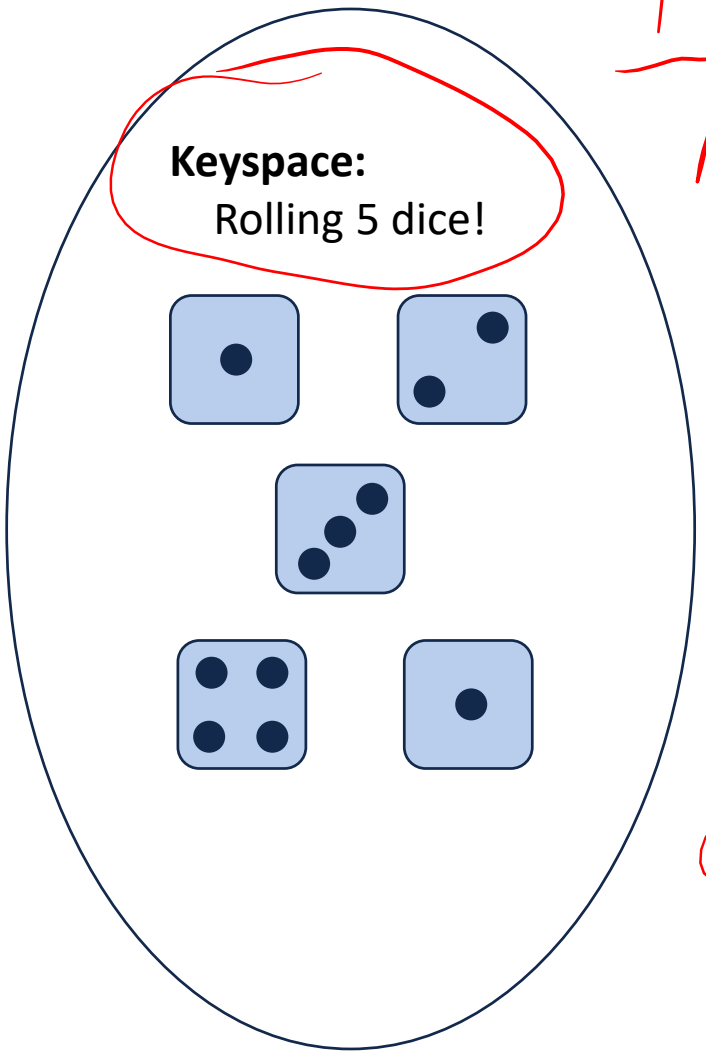
Petals around the rose

Key	Value
0	✓
1	X
2	✓
3	X
4	
5	X
6	
7	X
8	
9	X
10	
11	X
12	
13	X
14	
15	X

A Perfect Hash Function

1-to-1

No



Hash function

Petals around the rose

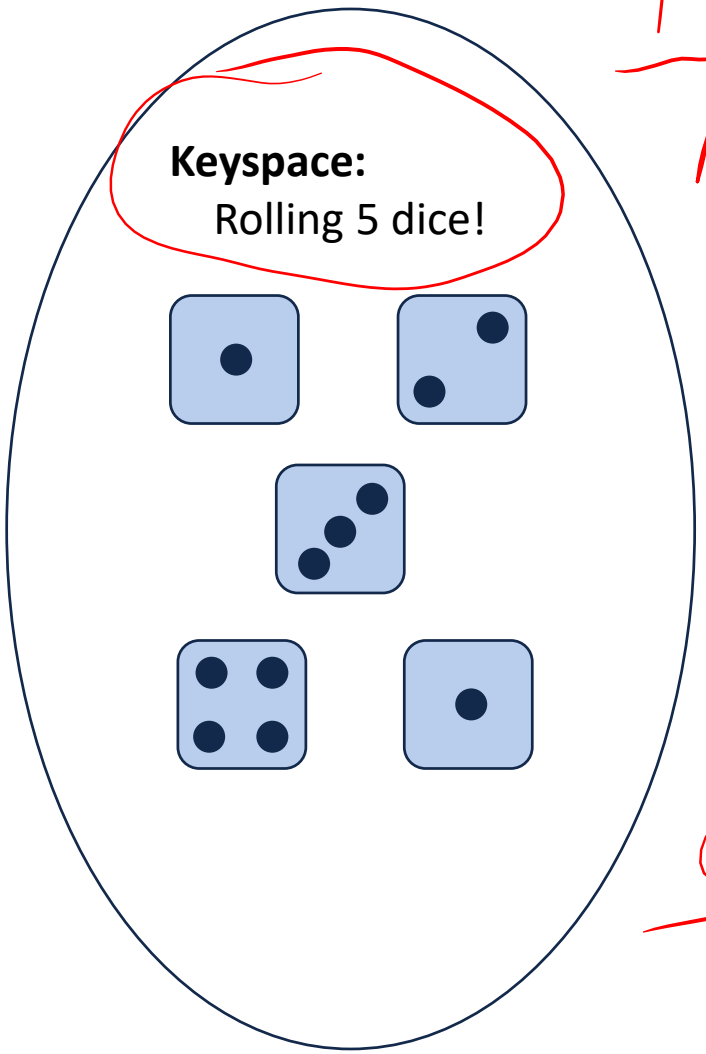
Onto

Key	Value
0	✓
1	X
2	✓
3	X
4	
5	X
6	
7	X
8	
9	X
10	
11	X
12	
13	X
14	
15	X

A Perfect Hash Function

1-to-1

No



Hash function

Petals around the rose

Onto

No : $h^{-1}(L)$ has no value

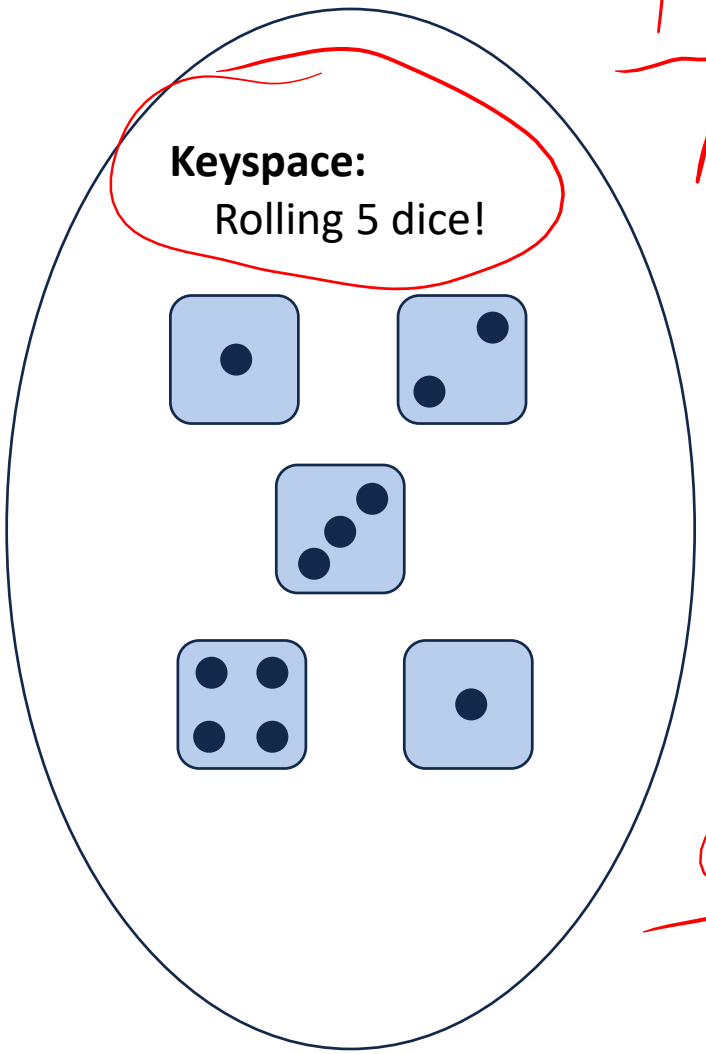
Key	Value
0	✓
1	X
2	✓
3	X
4	
5	X
6	
7	X
8	
9	X
10	
11	X
12	
13	X
14	
15	X

A Perfect Hash Function

P(20) =

1-to-1

No



Hash function

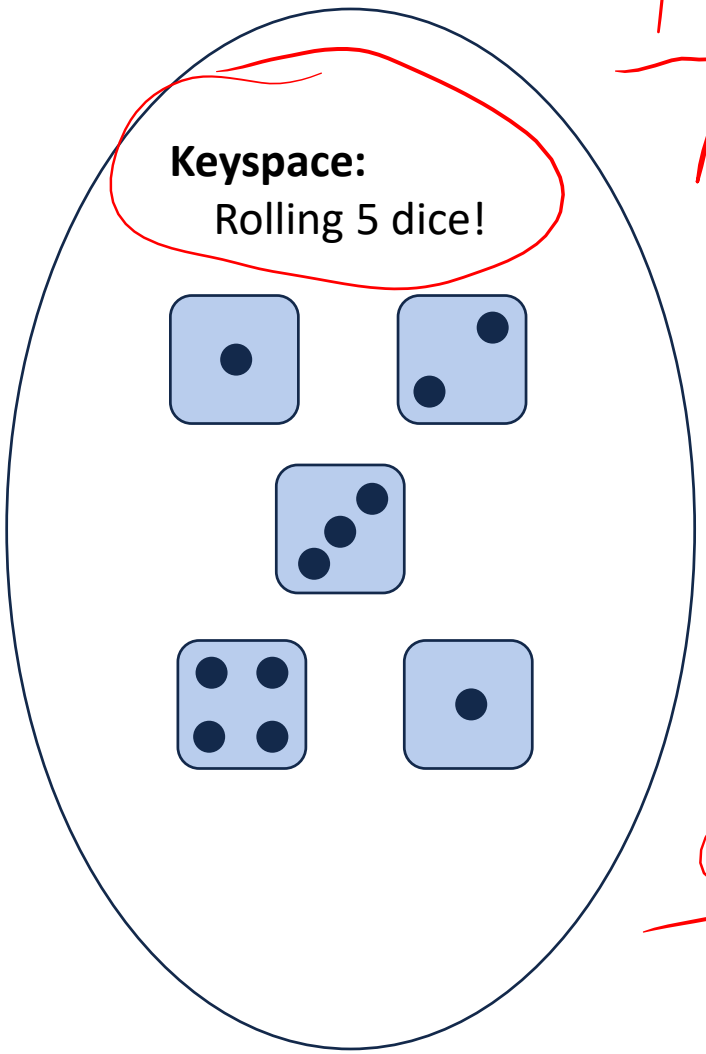
Petals around the rose

Onto

No: $h^{-1}(L)$ has no value

Key	Value
0	✓
1	X
2	✓
3	X
4	
5	X
6	
7	X
8	
9	X
10	
11	X
12	
13	X
14	
15	X

A Perfect Hash Function



1-to-1

No

Hash function

Petals around the rose

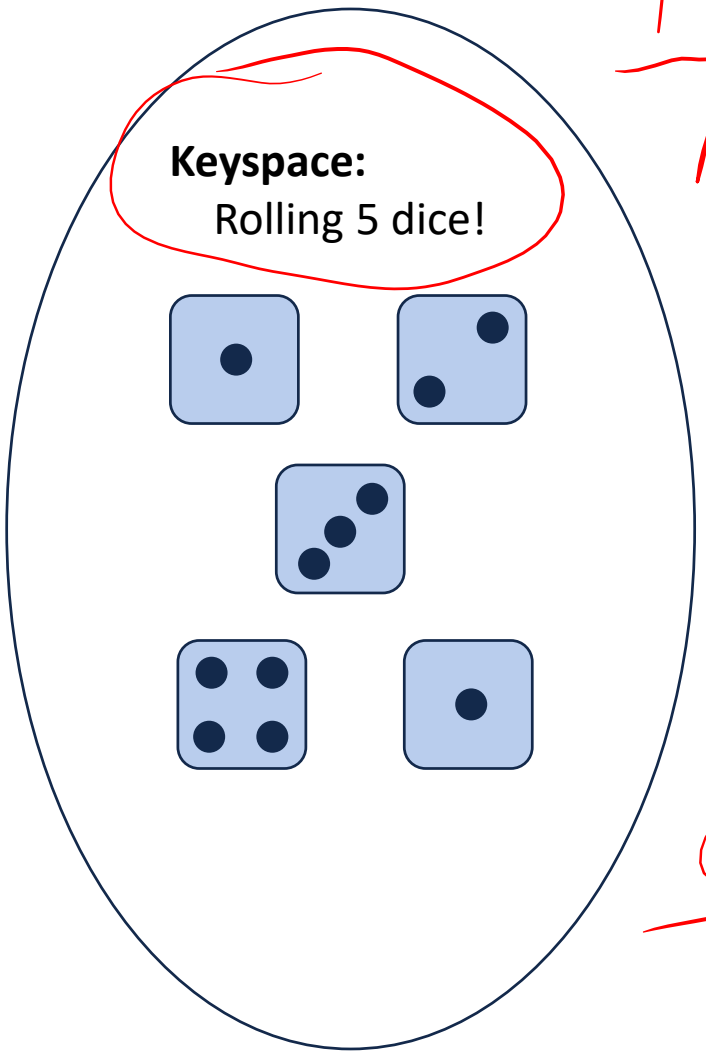
Onto

No: $h^{-1}(L)$ has no value

$P(20) = \frac{1}{6^5}$

Key	Value
0	✓
1	✗
2	✓
3	✗
4	
5	✗
6	
7	✗
8	
9	✗
10	
11	✗
12	
13	✗
14	
15	✗

A Perfect Hash Function



1-to-1

No

$P(20) = 1/6^5$

$P(0) > 1/6^5$

Hash function

Petals around the rose

Key	Value
0	✓
1	X
2	✓
3	X
4	
5	X
6	
7	X
8	
9	X
10	
11	X
12	
13	X
14	
15	X

Onto

No : $h^{-1}(L)$ has no value

Hash Function

Our **hash function** consists of two parts:

- A **hash**:

- A **compression**:

Choosing a good hash function is tricky…

- Don’t create your own (yet*)
- Very smart people have created very bad hash functions

Hash Function

Our **hash function** consists of two parts:

- A **hash**: h_H key → i (int)

- A **compression**:

Choosing a good hash function is tricky…

- Don’t create your own (yet*)
- Very smart people have created very bad hash functions

Hash Function

Our **hash function** consists of two parts:

- A **hash**: h_H key $\rightarrow$ i (int)

- A **compression**:

$$h_C: i \rightarrow [0, N-1]$$

Choosing a good hash function is tricky…

- Don’t create your own (yet*)
- Very smart people have created very bad hash functions

Hash Function

Our **hash function** consists of two parts:

- A **hash**: h_H key $\rightarrow$ i (int)

- A **compression**: h_C: i $\rightarrow$ [0, N-1] $\%N$

Choosing a good hash function is tricky…

- Don’t create your own (yet*)
- Very smart people have created very bad hash functions

Hash Function

Characteristics of a good hash function:

1. Computation Time:

2. Deterministic:

3. Satisfy the SUHA:

Hash Function

Characteristics of a good hash function:

1. Computation Time:

Runs in constant time, O(1)

2. Deterministic:

3. Satisfy the SUHA:

Hash Function

Characteristics of a good hash function:

1. Computation Time:

Runs in constant time, O(1)

2. Deterministic:

No random element

3. Satisfy the SUHA:

Hash Function

Characteristics of a good hash function:

1. Computation Time:

Runs in constant time, O(1)

2. Deterministic:

No random element

if $k_1 == k_2 \rightarrow h(k_1) == h(k_2)$

3. Satisfy the SUHA:

Hash Function

Characteristics of a good hash function:

1. Computation Time:

Runs in constant time, O(1)

2. Deterministic:

No random element

if $k_1 == k_2 \rightarrow h(k_1) == h(k_2)$

3. Satisfy the SUHA:

Simple Uniform hashing assumption

Hash Function

Characteristics of a good hash function:

1. Computation Time:

   Runs in constant time, O(1)

2. Deterministic:

   No random element

   if $k_1 == k_2 \rightarrow h(k_1) == h(k_2)$

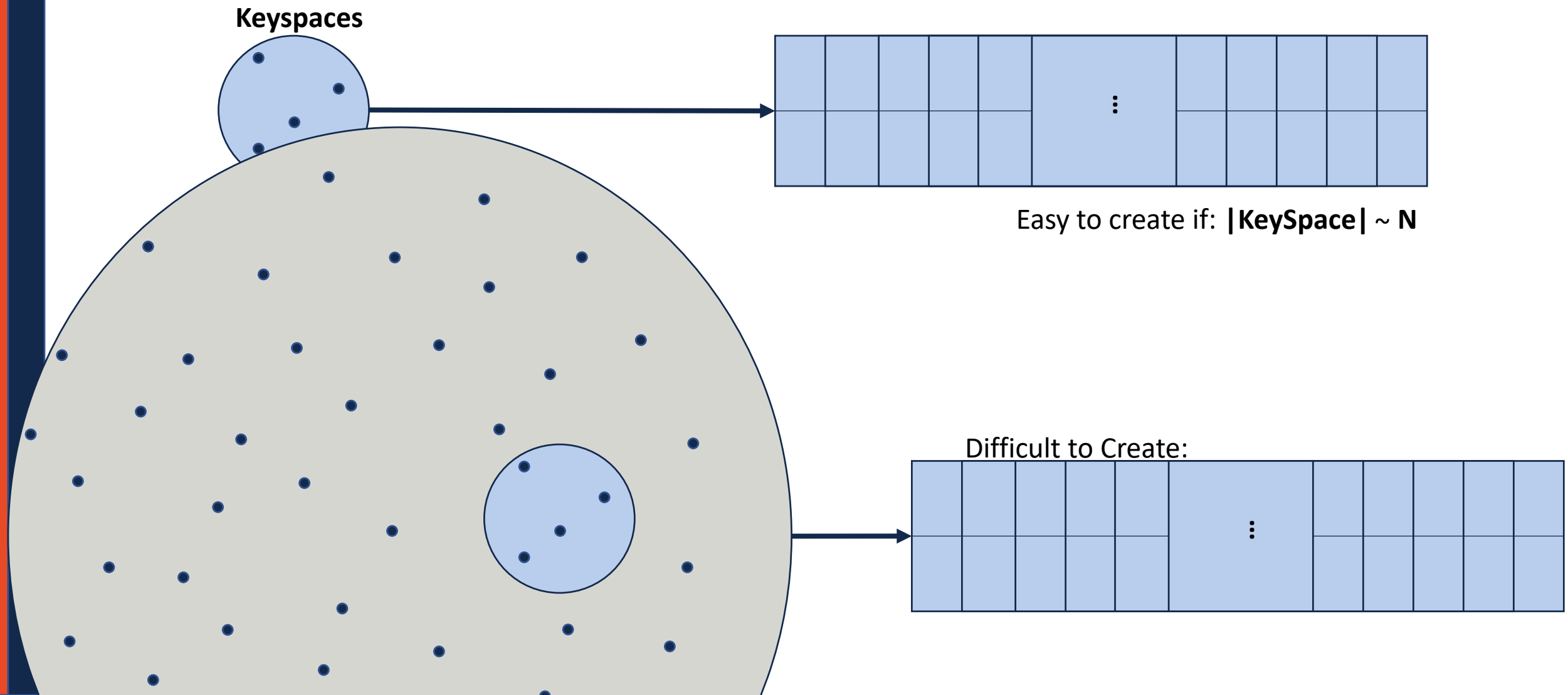
3. Satisfy the SUHA:

   Simple Uniform hashing assumption

   for keys k, j where k != j

   $P(h(k) == h(j)) = 1/N$

General Purpose Hash Function



Hash Function

Given: Easy to create a hash function of strings of length 8

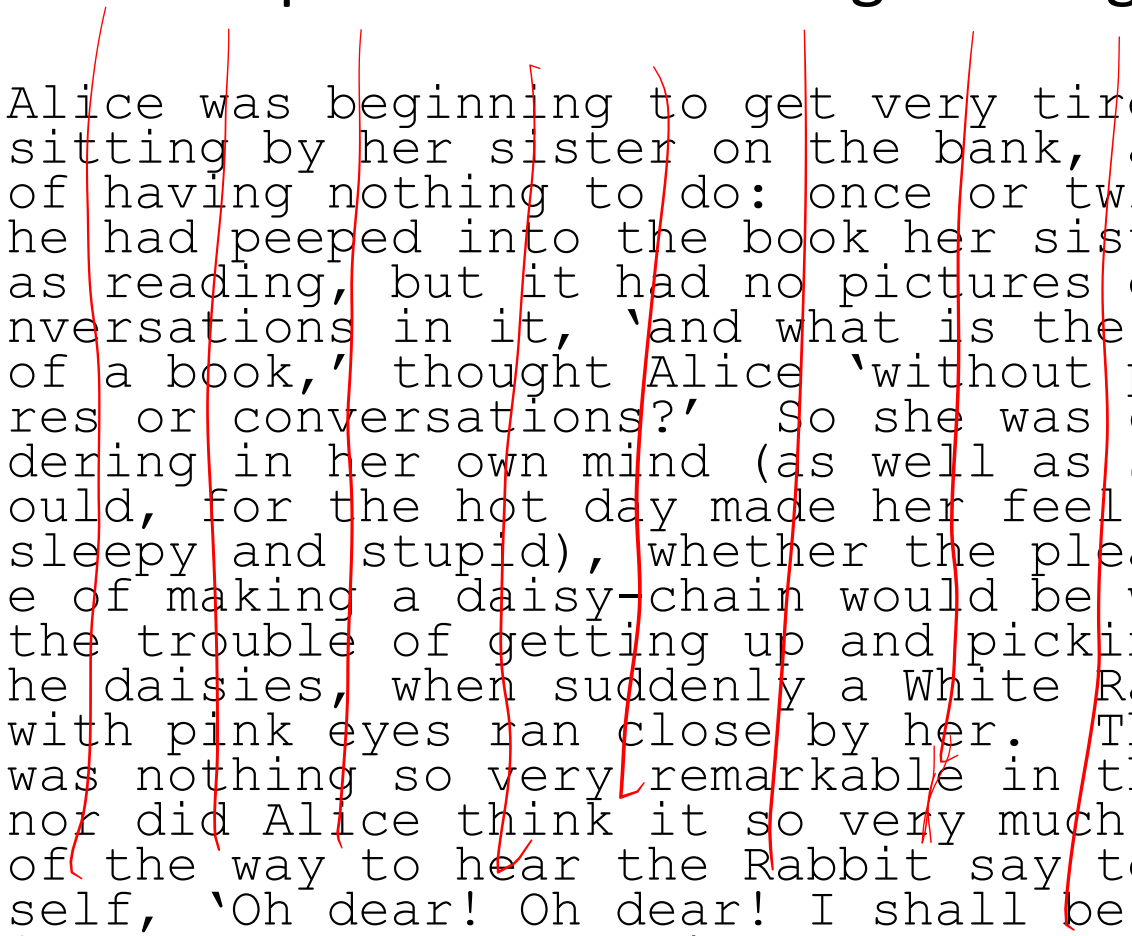
Hash Function

Idea: Map 40 character things to length 8:

```
Alice was beginning to get very tired of
sitting by her sister on the bank, and
of having nothing to do: once or twice s
he had peeped into the book her sister w
as reading, but it had no pictures or co
nversations in it, ‘and what is the use
of a book,’ thought Alice ‘without pictu
res or conversations?’  So she was consi
dering in her own mind (as well as she c
ould, for the hot day made her feel very
sleepy and stupid), whether the pleasur
e of making a daisy-chain would be worth
the trouble of getting up and picking t
he daisies, when suddenly a White Rabbit
with pink eyes ran close by her.  There
was nothing so very remarkable in that;
nor did Alice think it so very much out
of the way to hear the Rabbit say to it
self, ‘Oh dear! Oh dear! I shall be late
!’ (when she thought it over afterwards,
it occurred to her that she ought to ha
```

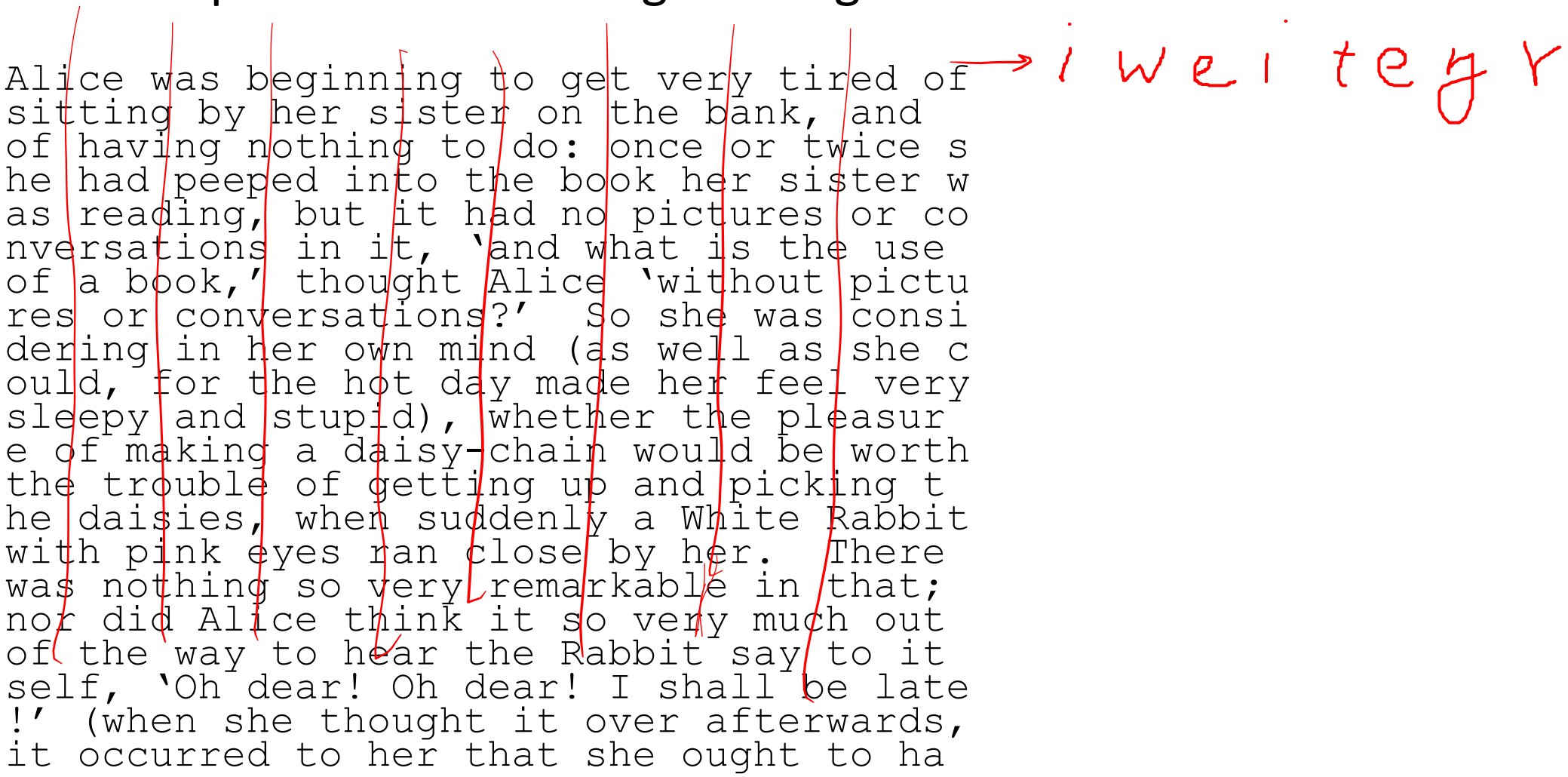
Hash Function

Idea: Map 40 character things to length 8:



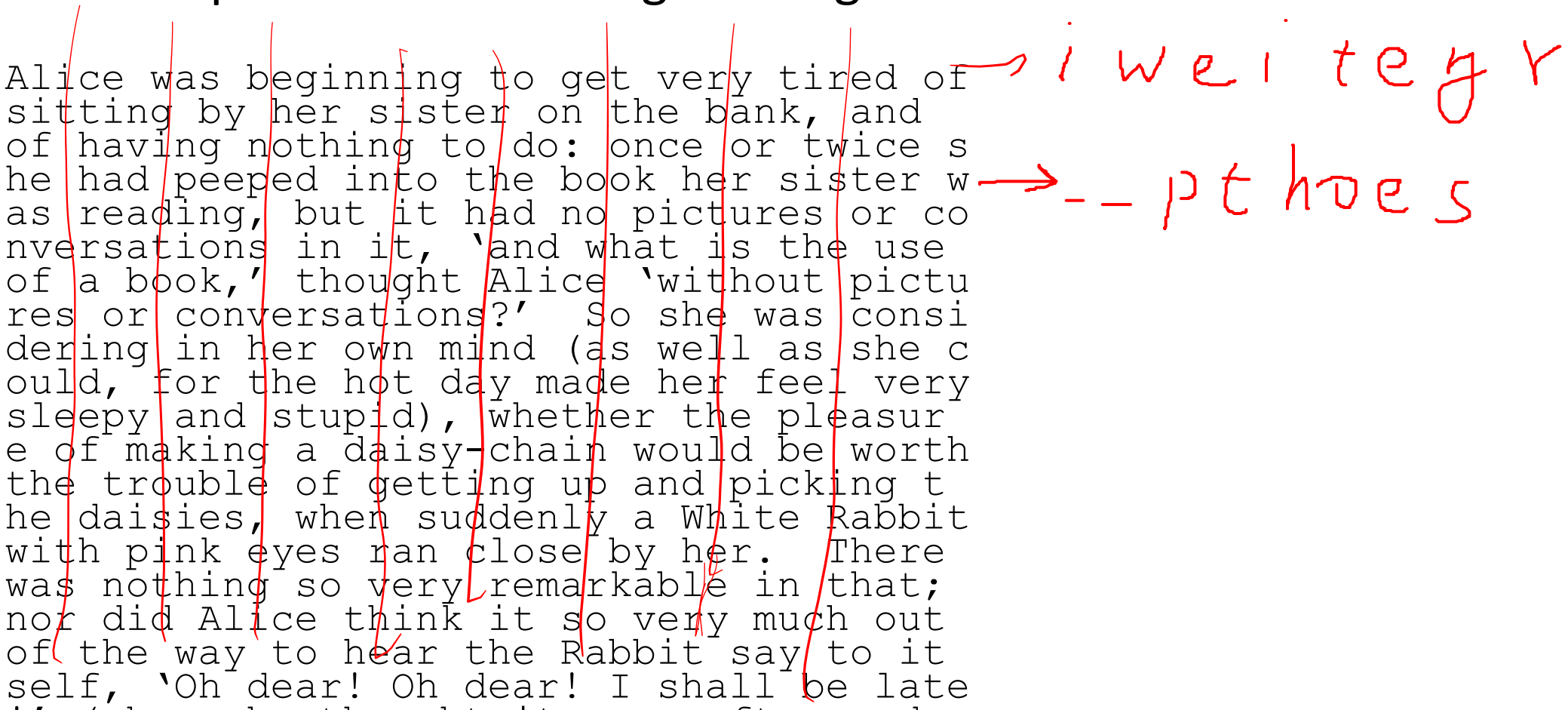
Hash Function

Idea: Map 40 character things to length 8:



Hash Function

Idea: Map 40 character things to length 8:



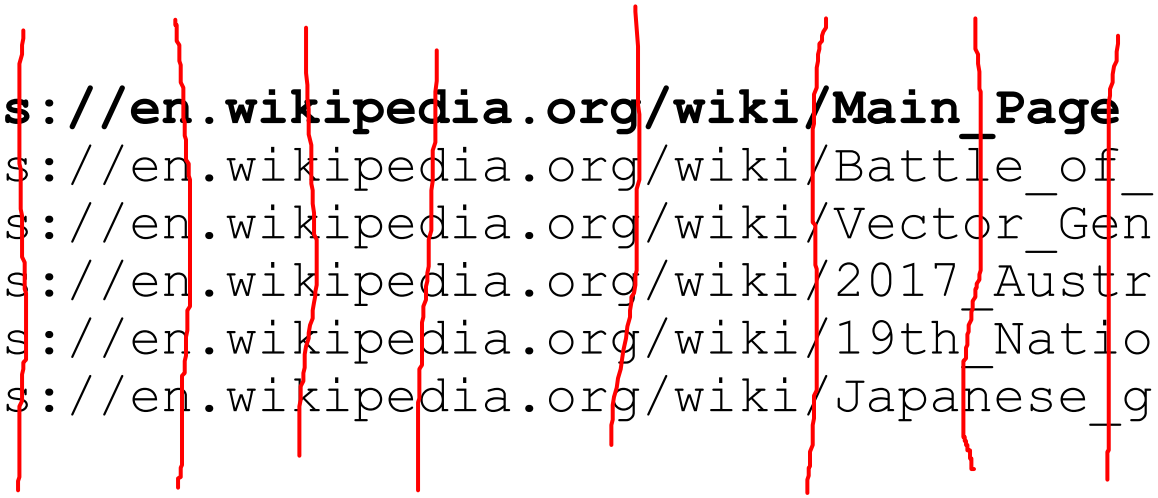
Hash Function

Idea: Map 40 character things to length 8:

```
https://en.wikipedia.org/wiki/Main_Page
https://en.wikipedia.org/wiki/Battle_of_
https://en.wikipedia.org/wiki/Vector_Gen
https://en.wikipedia.org/wiki/2017_Austr
https://en.wikipedia.org/wiki/19th_Natio
https://en.wikipedia.org/wiki/Japanese_g
```

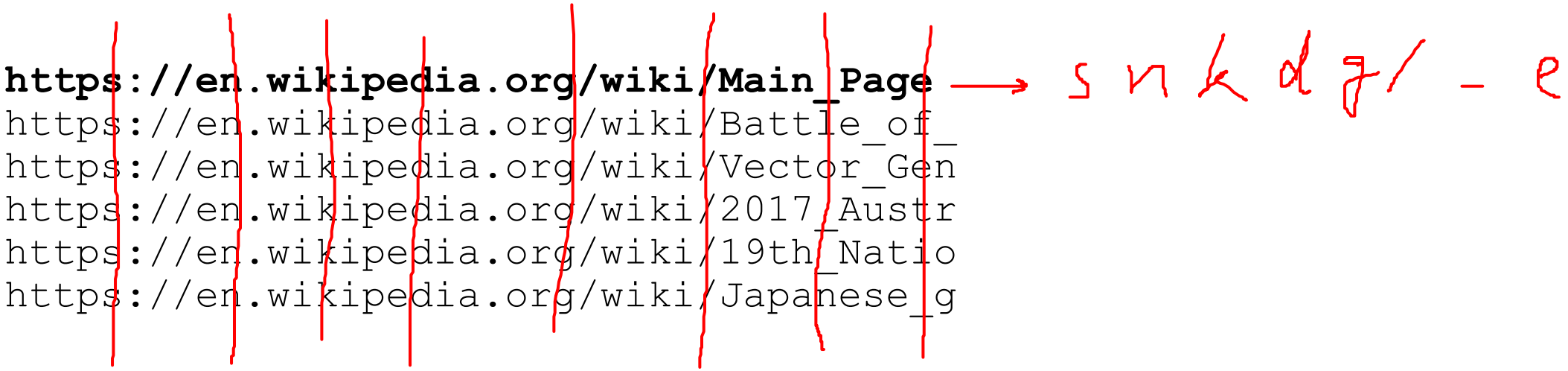
Hash Function

Idea: Map 40 character things to length 8:



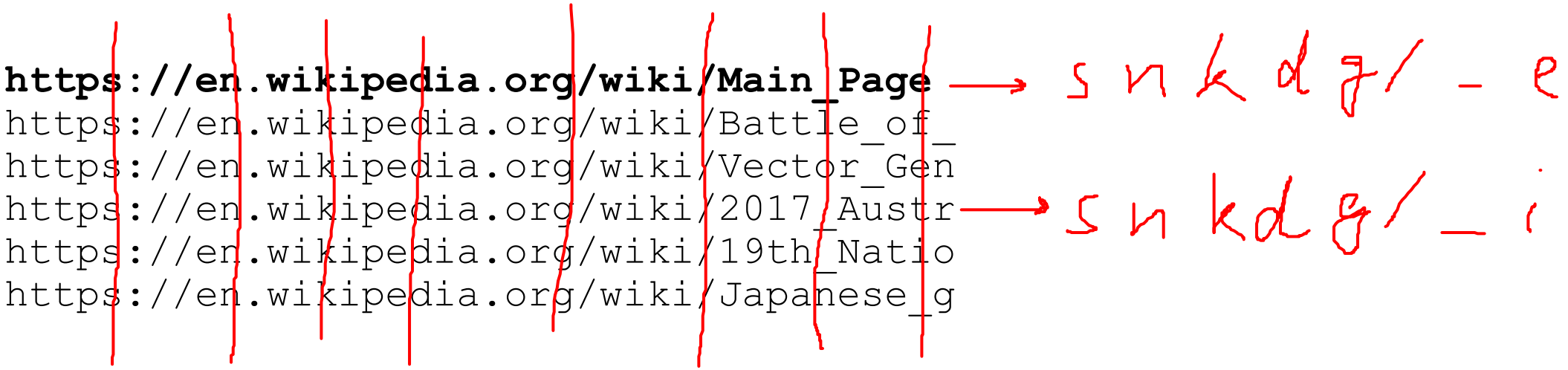
Hash Function

Idea: Map 40 character things to length 8:



Hash Function

Idea: Map 40 character things to length 8:



Hash Function

In CS 225, we focus on **general purpose** hash functions.

Other hash functions exist with different properties
(eg: cryptographic hash functions)

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1.

2.

3.

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1. hash function h(k)
2.
3.

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

1. hash function h(k)
2. array
3.

A Hash Table based Dictionary

Client Code:

```
1  Dictionary<KeyType, ValueType> d;
2  d[k] = v;
```

A **Hash Table** consists of three things:

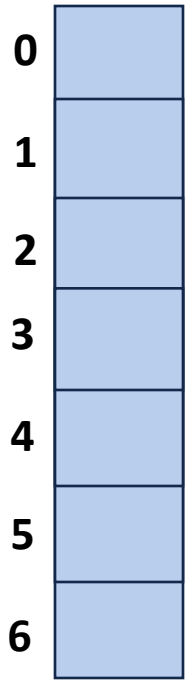
1. hash function h(k)
2. array
3. collision resolution strategies

(Example of open hashing)

Collision Handling: Separate Chaining

$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$ $|S| = n$

$h(k) = k \% 7$ $|Array| = N$



	Worst Case	SUHA
Insert		
Remove/Find		

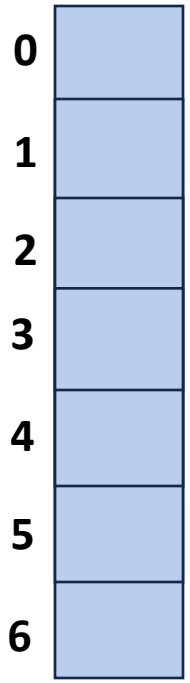
(Example of open hashing)

Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

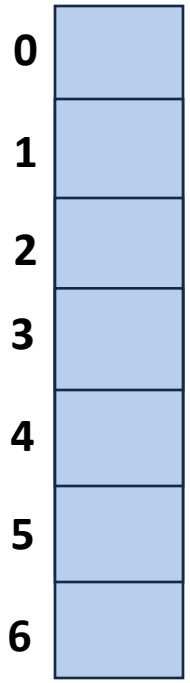
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7=2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

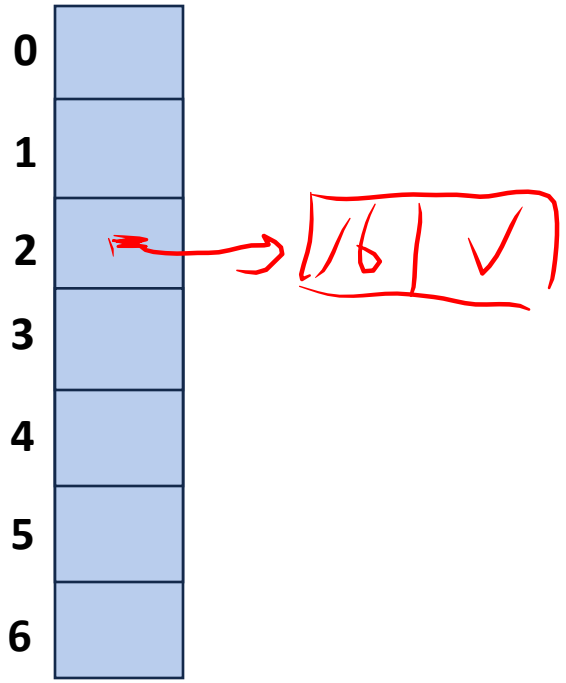
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7=2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

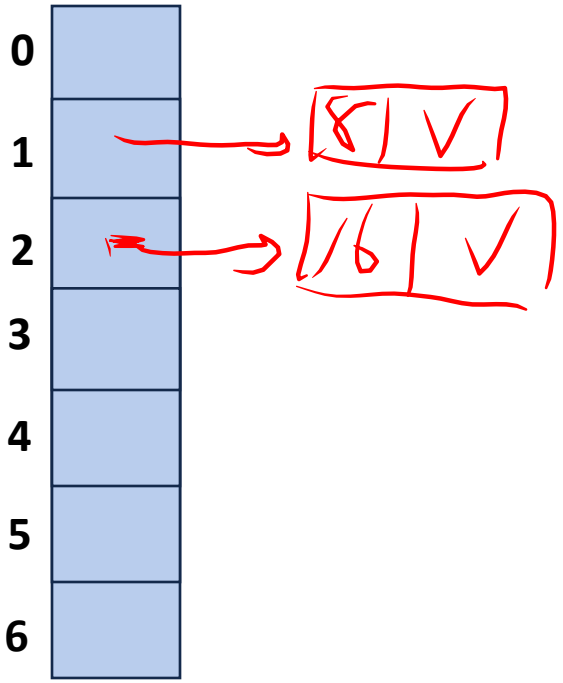
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

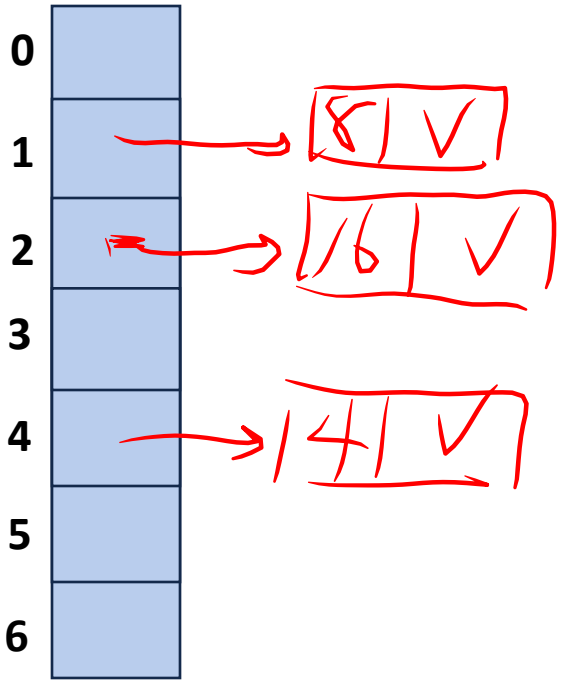
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

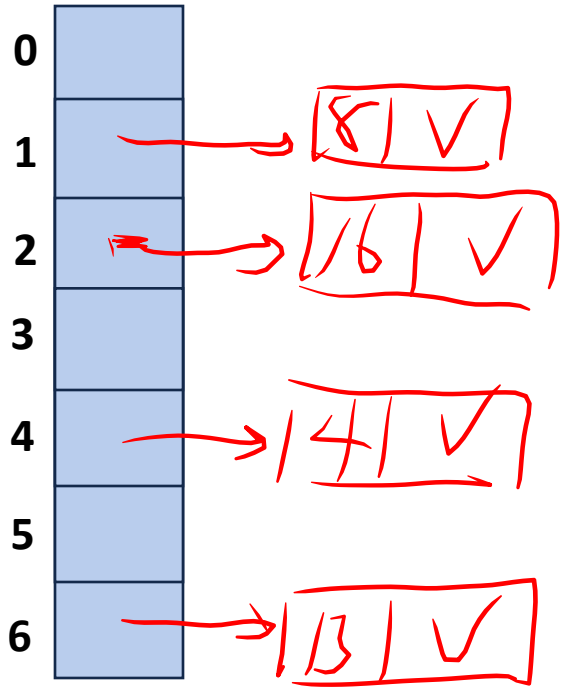
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7=2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

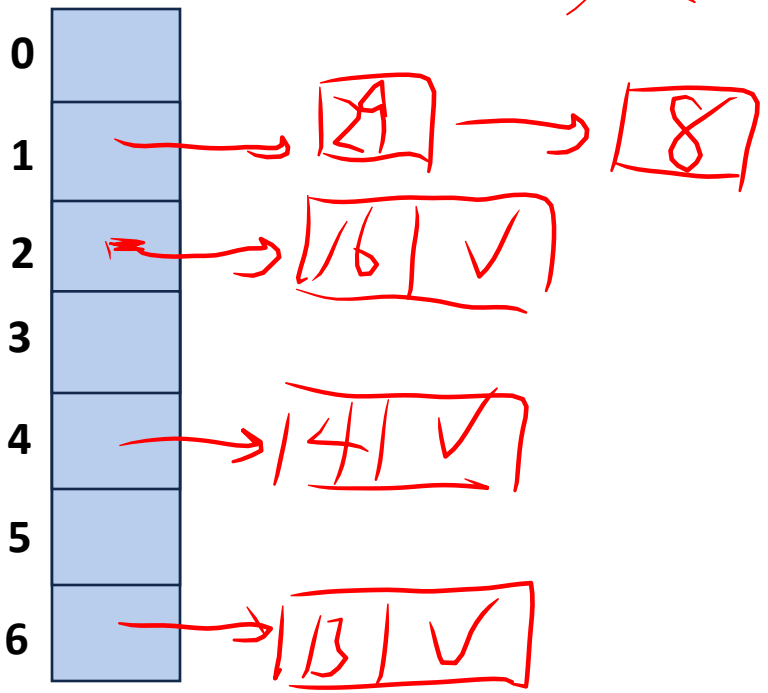
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7=2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

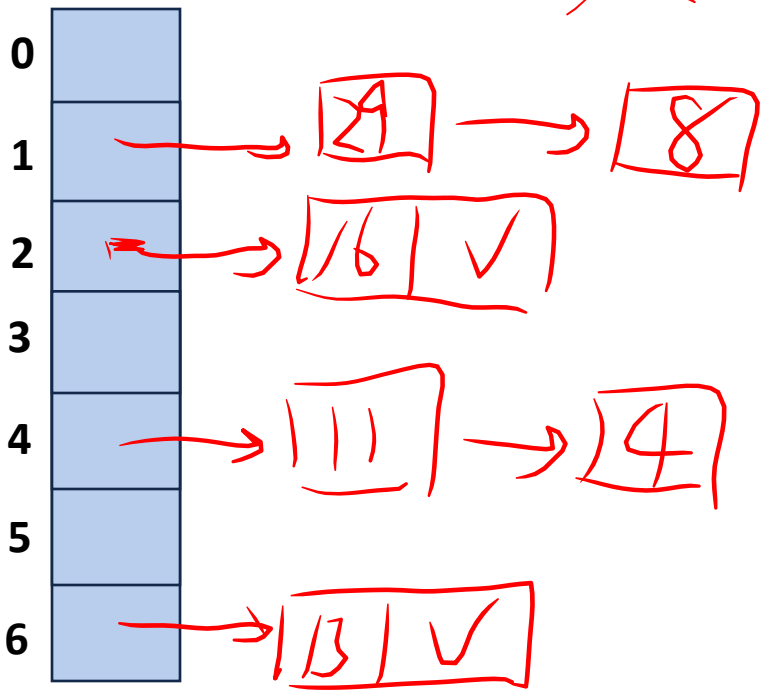
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7=2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

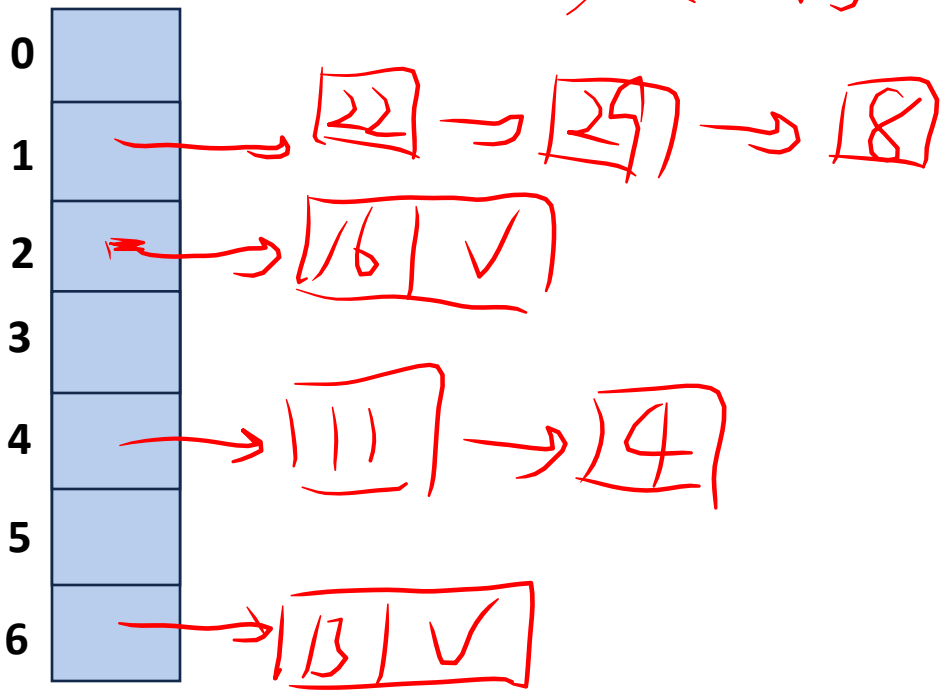
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7=2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

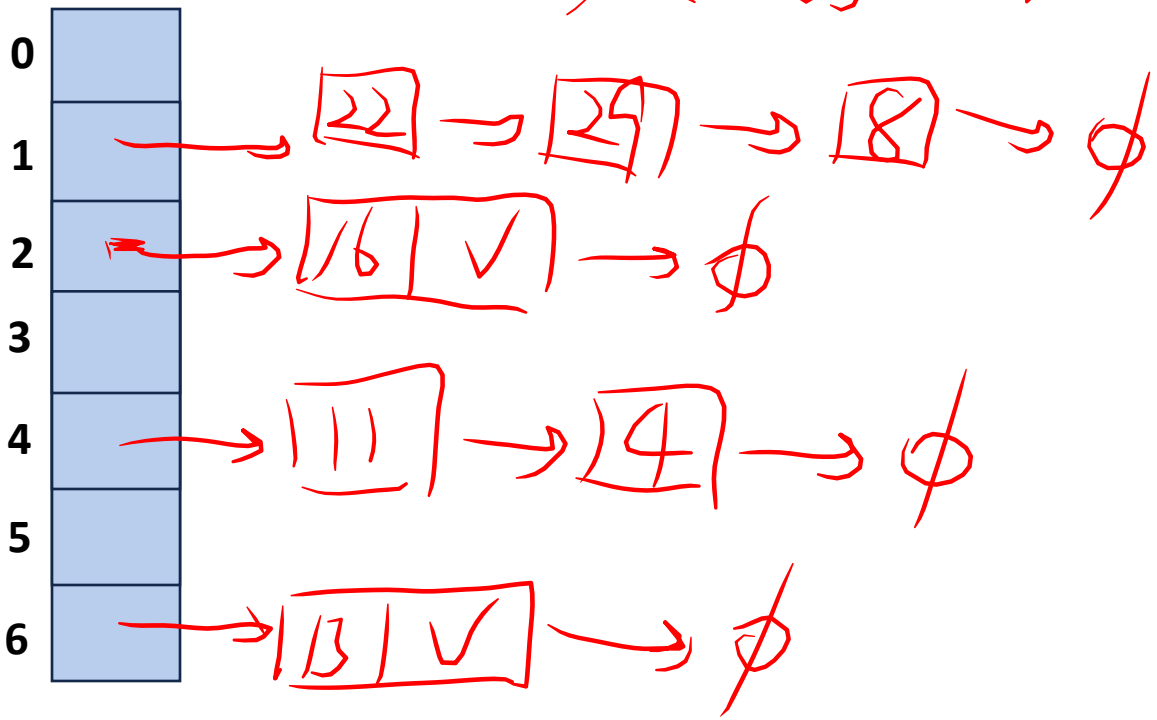
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



	Worst Case	SUHA
Insert		
Remove/Find		

(Example of open hashing)

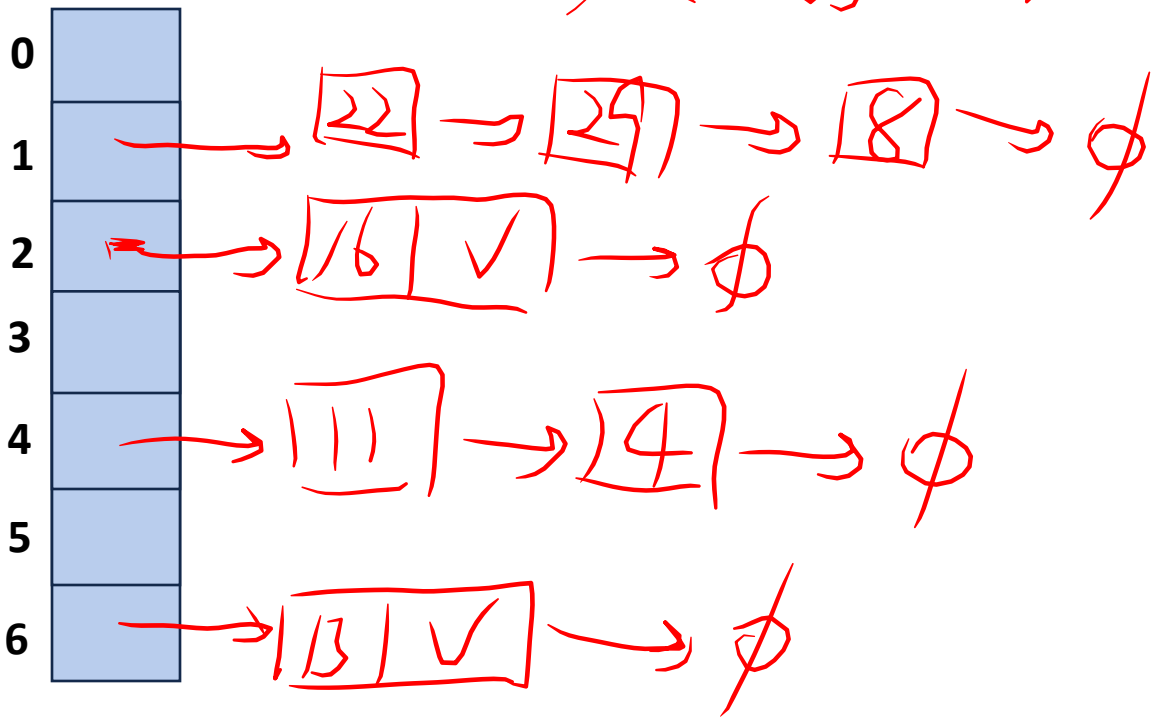
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



	Worst Case	SUHA
Insert	O(1)	
Remove/Find		

(Example of open hashing)

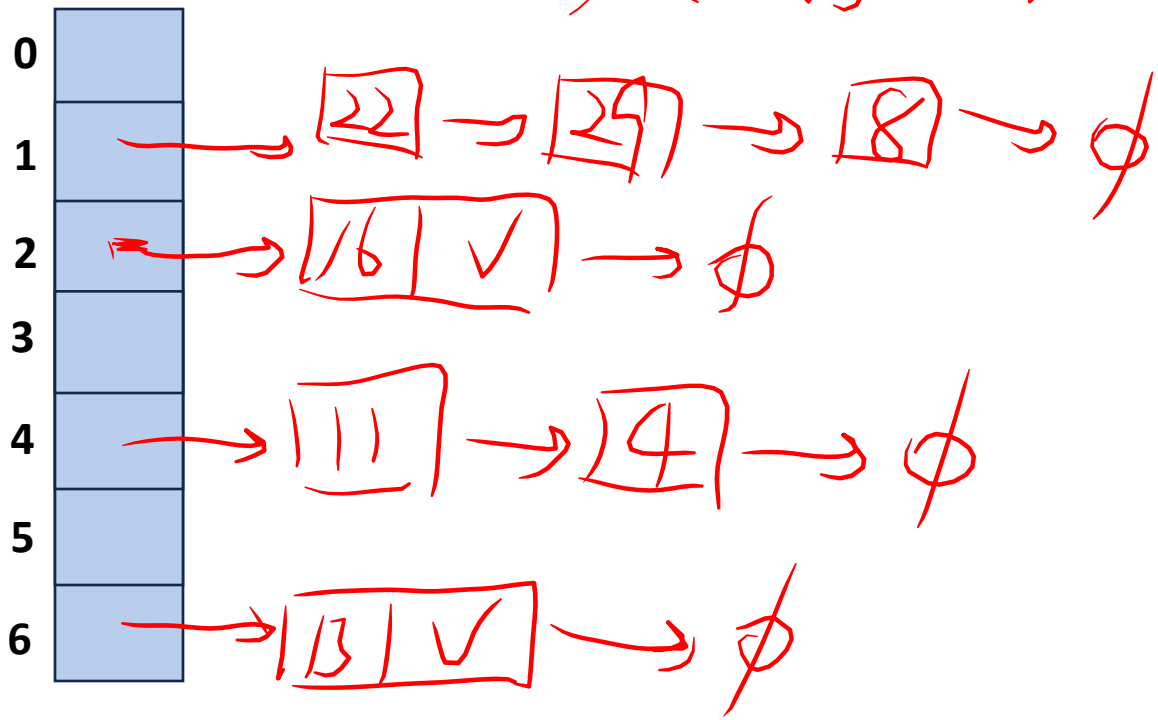
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



	Worst Case	SUHA
Insert	O(1)	
Remove/Find	O(n)	

(Example of open hashing)

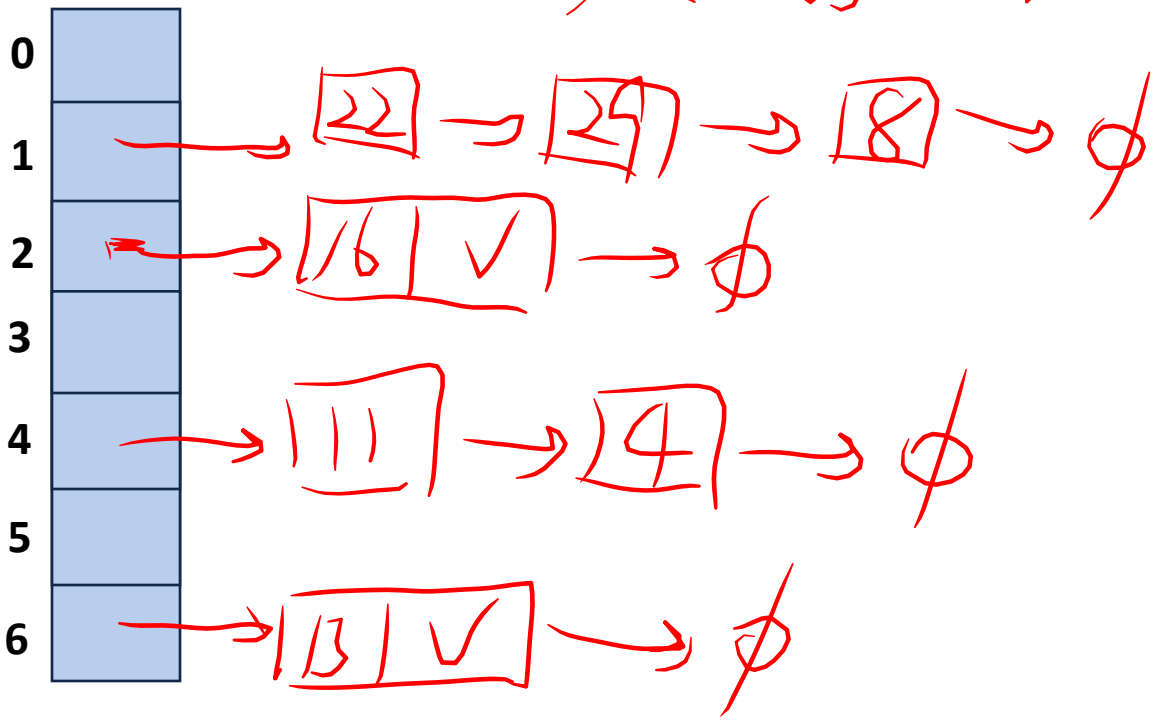
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



	Worst Case	SUHA
Insert	O(1)	
Remove/Find	O(n)	O(n/N)

(Example of open hashing)

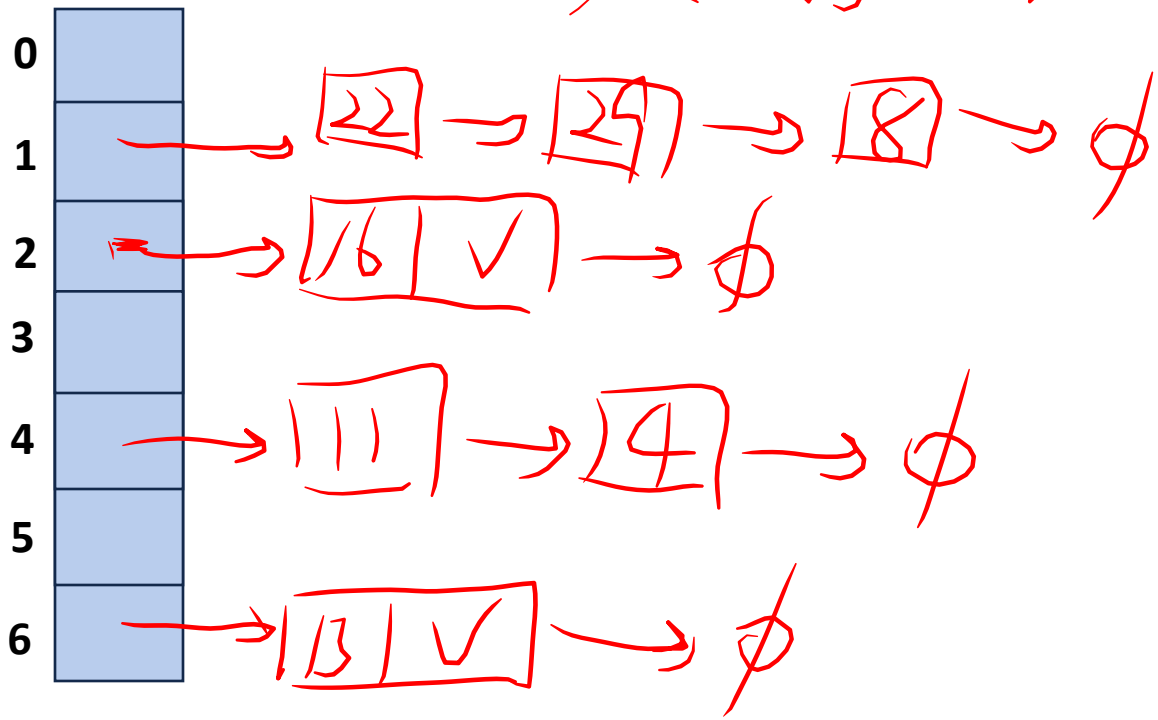
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



	Worst Case	SUHA
Insert	O(1)	
Remove/Find	O(n) ≡ O(n/N)	

(Example of open hashing)

Collision Handling: Separate Chaining

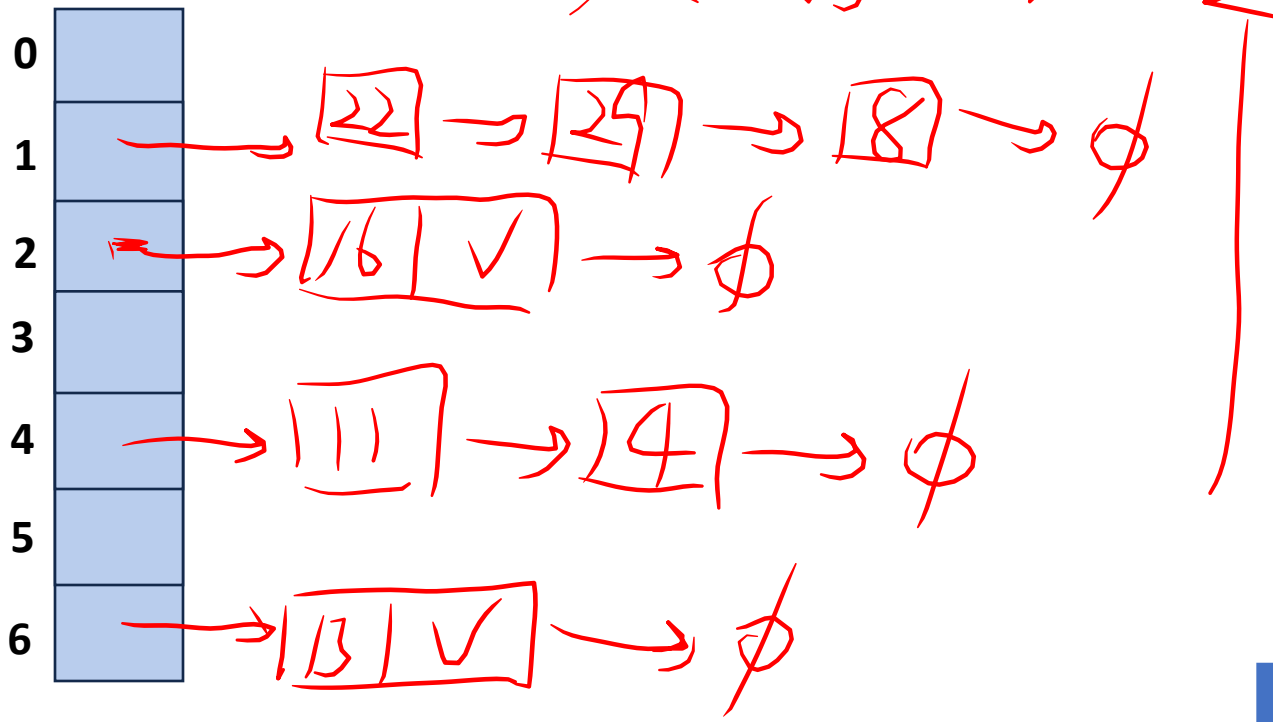
Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2

load factor



	Worst Case	SUHA
Insert	O(1)	
Remove/Find	O(n) ≡ O(n/N)	

(Example of open hashing)

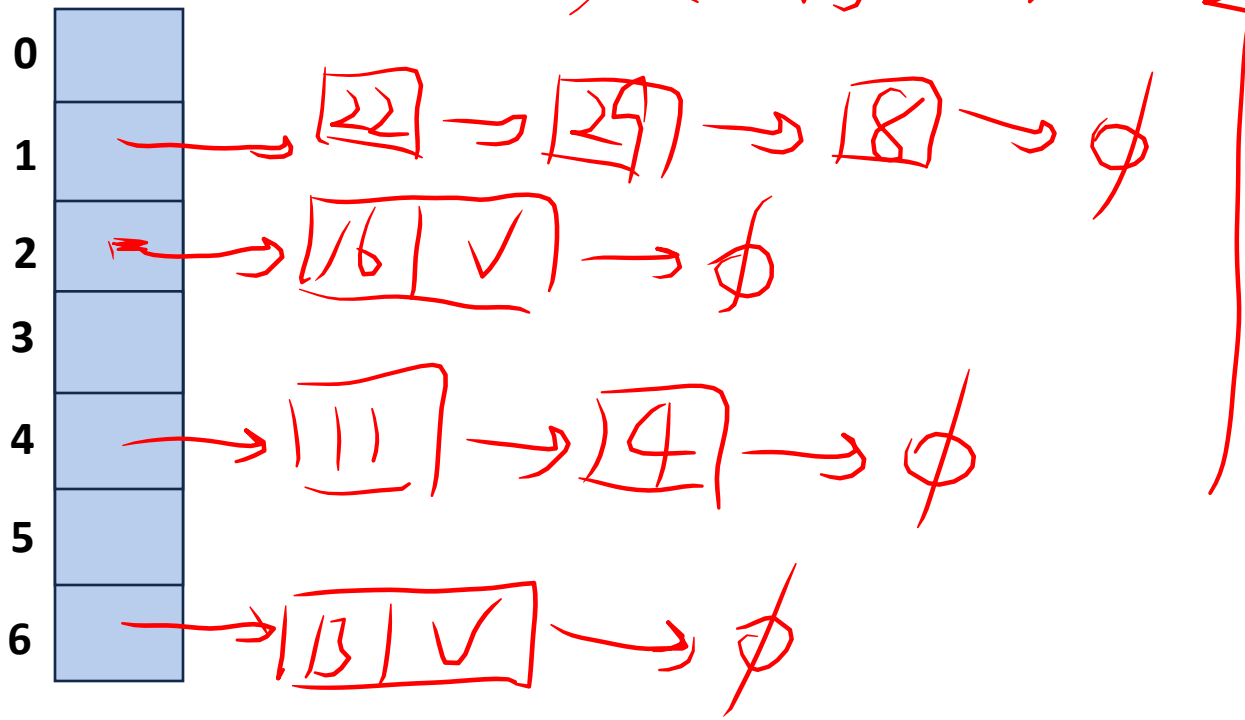
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



load factor

$\alpha = n/N$

	Worst Case	SUHA
Insert	O(1)	
Remove/Find	O(n) ≡ O(n/N)	

(Example of open hashing)

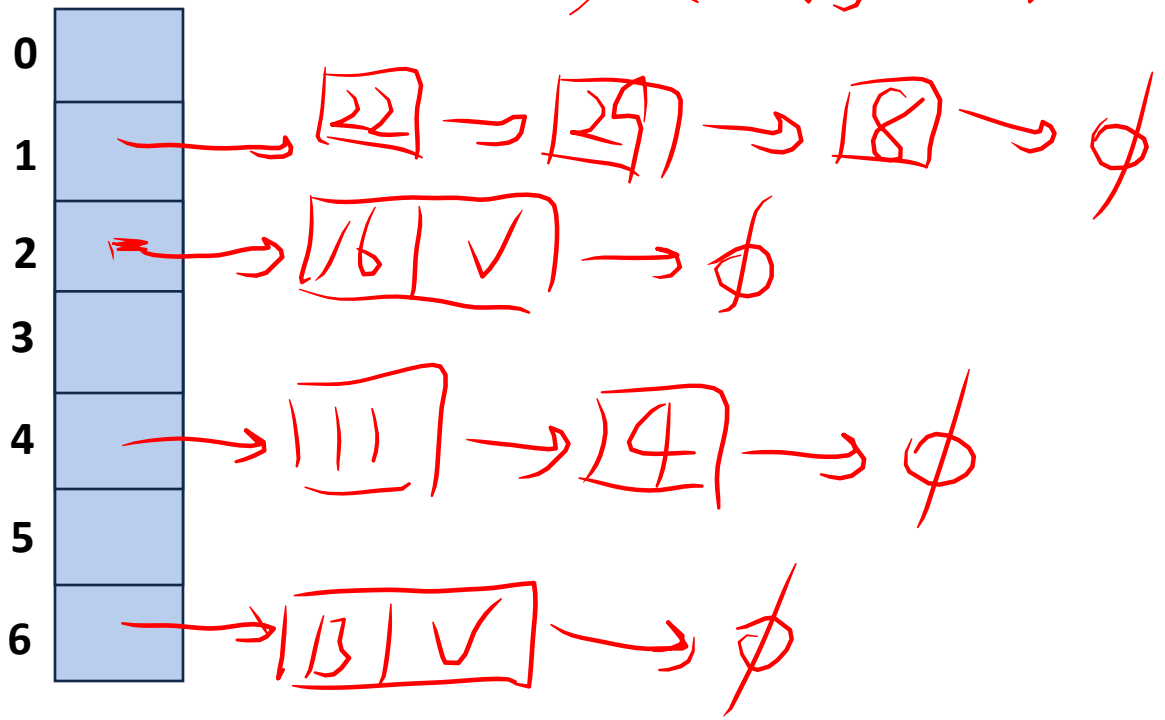
Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**

k=16, h(16) = 16%7 = 2



load factor

$\alpha = n/N$

$\alpha \Rightarrow$ unbounded

	Worst Case	SUHA
Insert	O(1)	
Remove/Find	O(n) ≡ O(n/N)	

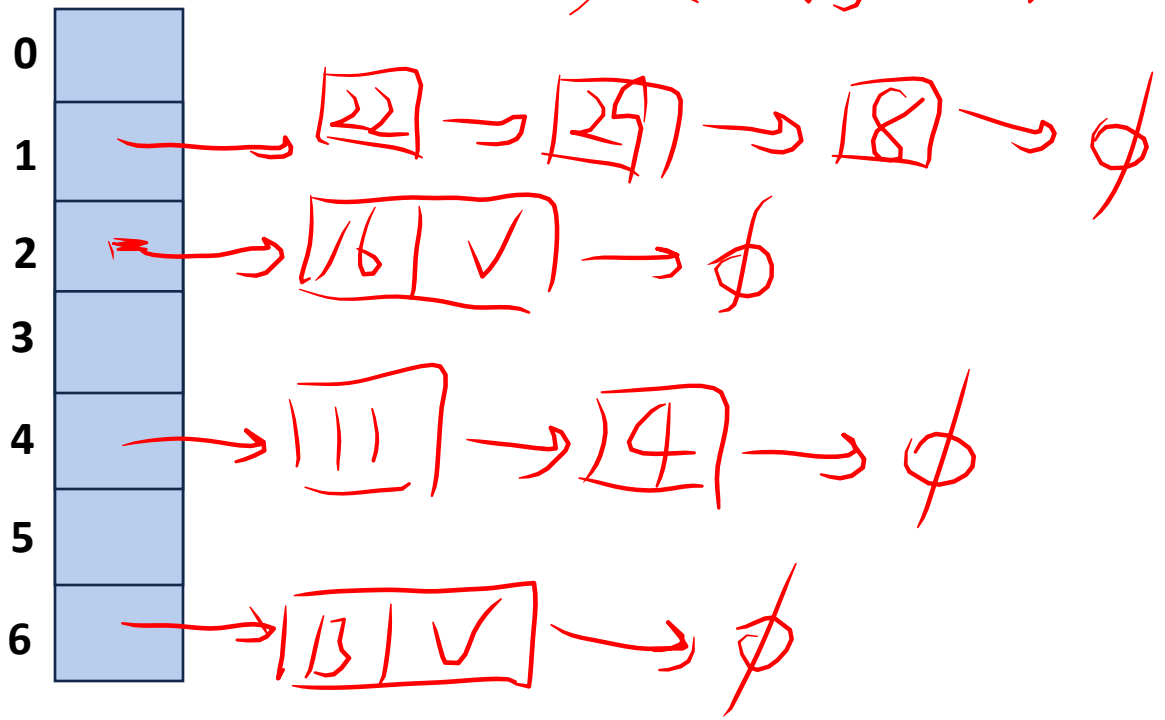
(Example of open hashing)

Collision Handling: Separate Chaining

Linked list of every node

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



load factor

$\alpha = n/N$

$\alpha \Rightarrow$ unbounded

open hashing

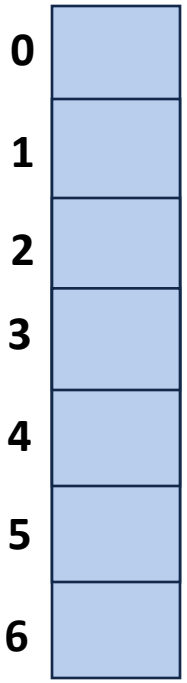
	Worst Case	SUHA
Insert	$O(1)$	
Remove/Find	$O(n) \equiv O(n/N)$	

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

Try h(k) = (k + 2) % 7, if full…

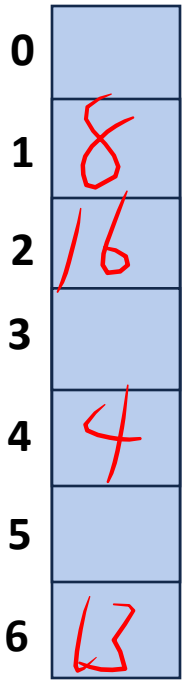
Try …

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

Try h(k) = (k + 2) % 7, if full…

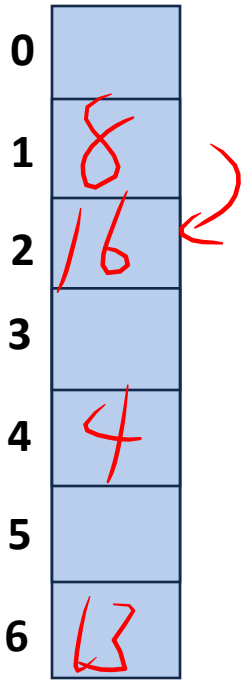
Try …

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

Try h(k) = (k + 2) % 7, if full…

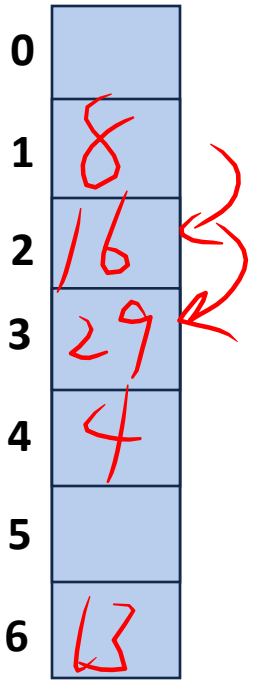
Try …

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

Try h(k) = (k + 2) % 7, if full…

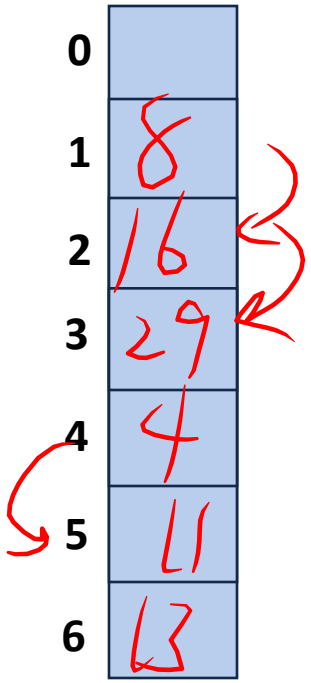
Try …

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

Try h(k) = (k + 2) % 7, if full…

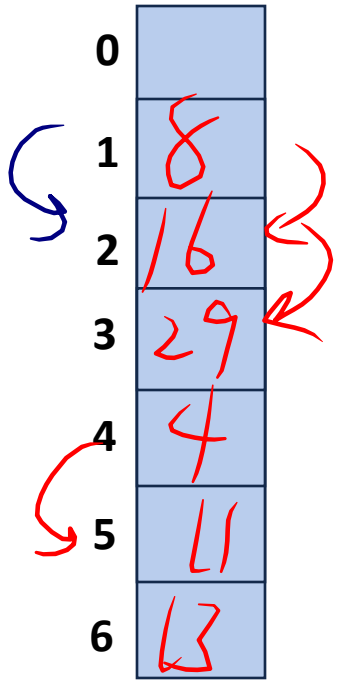
Try …

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

Try h(k) = (k + 2) % 7, if full…

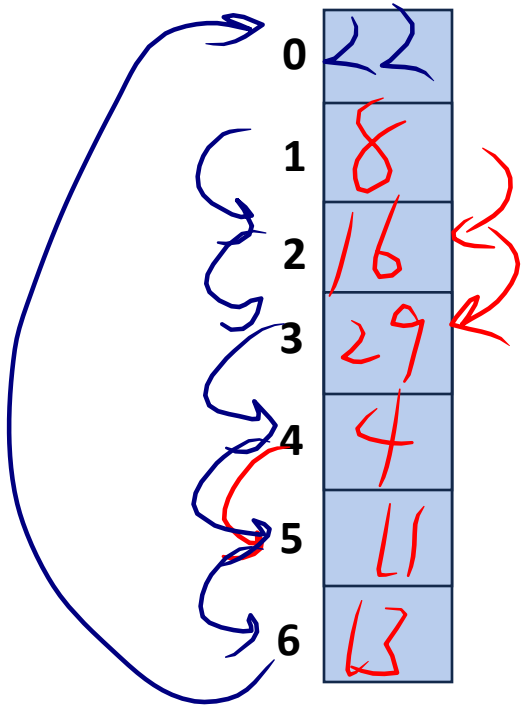
Try …

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

Try h(k) = (k + 2) % 7, if full…

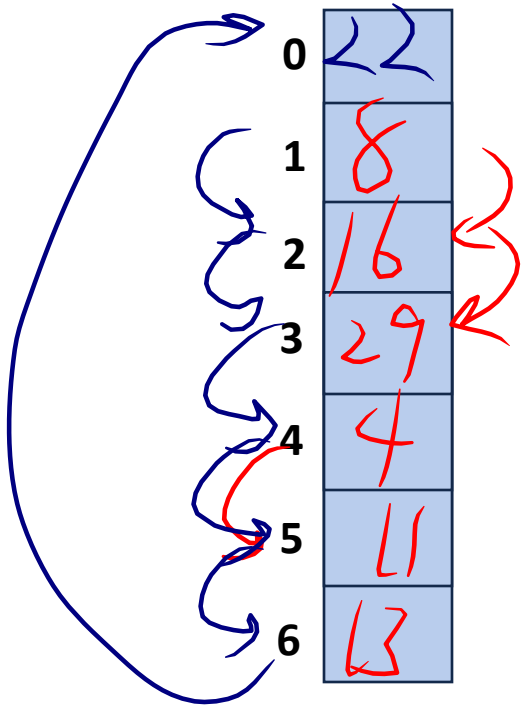
Try …

(Example of closed hashing)

Collision Handling: Probe-based Hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

h(k) = k % 7 **|Array| = N**



Try h(k) = (k + 0) % 7, if full…

Try h(k) = (k + 1) % 7, if full…

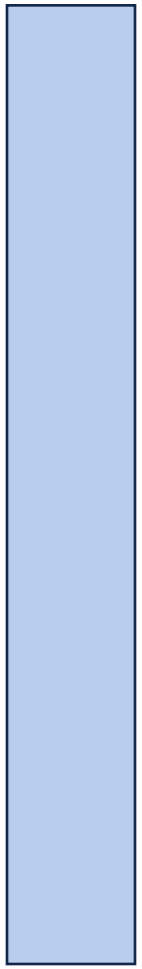
Try h(k) = (k + 2) % 7, if full…

Try …

$2 \leq 1.0 \quad \frac{n}{N} = \frac{7}{7}$

A Problem w/ Linear Probing

Primary clustering:

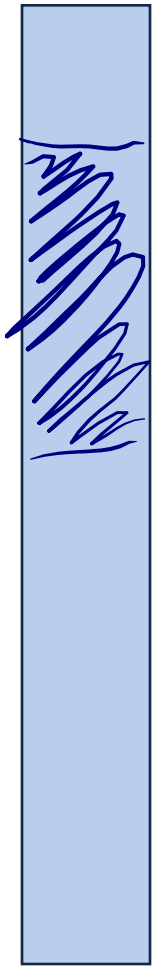


Description:

Remedy:

A Problem w/ Linear Probing

Primary clustering:

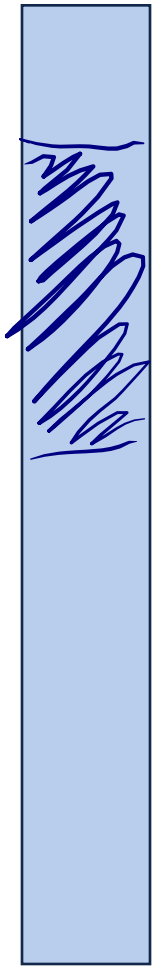


Description:

Remedy:

A Problem w/ Linear Probing

Primary clustering:



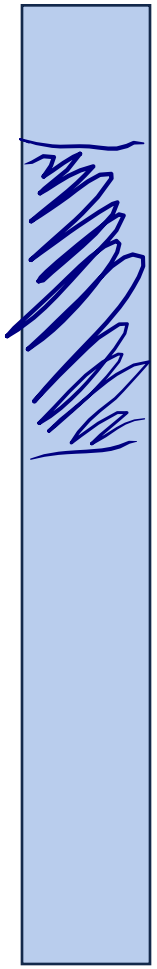
Description:

hash locations tend to cluster in primary clusters even under low load 2

Remedy:

A Problem w/ Linear Probing

Primary clustering:



Description:

hash locations tend to cluster in primary clusters even under low load 2

Remedy:

double hashing

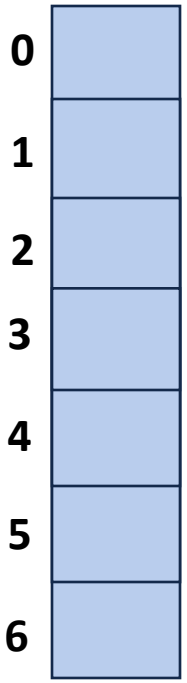
(Example of closed hashing)

Collision Handling: Double hashing

$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$ $|S| = n$

$h_1(k) = k \% 7$ $|Array| = N$

$h_2(k) = 5 - k \% 5$



Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

$$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$$

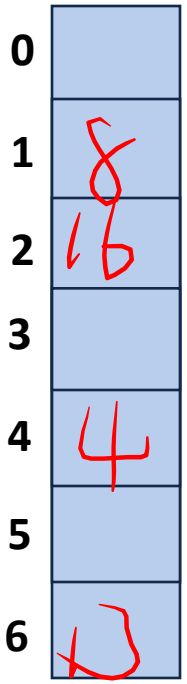
(Example of closed hashing)

Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$



Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

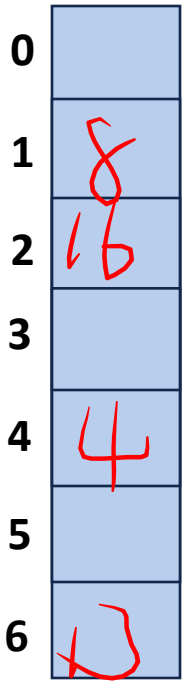
(Example of closed hashing)

Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$



Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

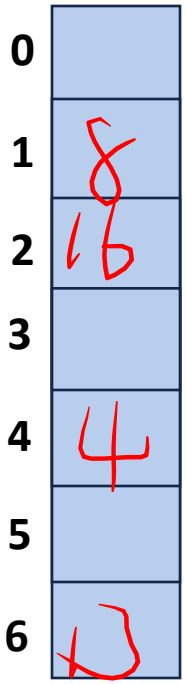
(Example of closed hashing)

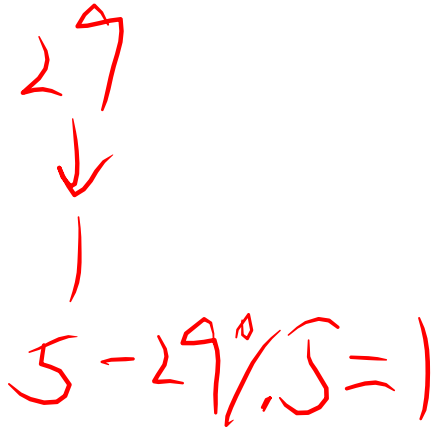
Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$





Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

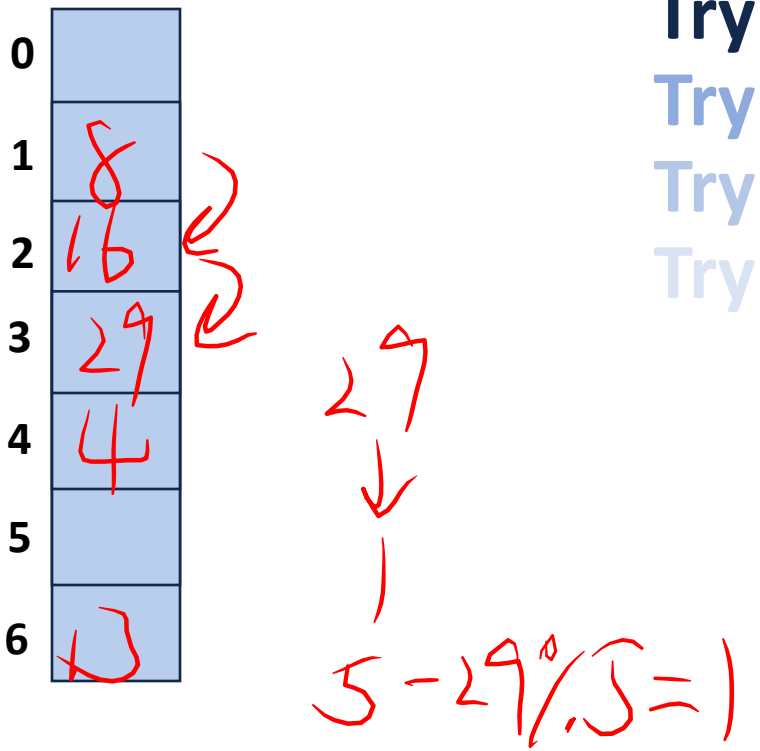
(Example of closed hashing)

Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$



Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

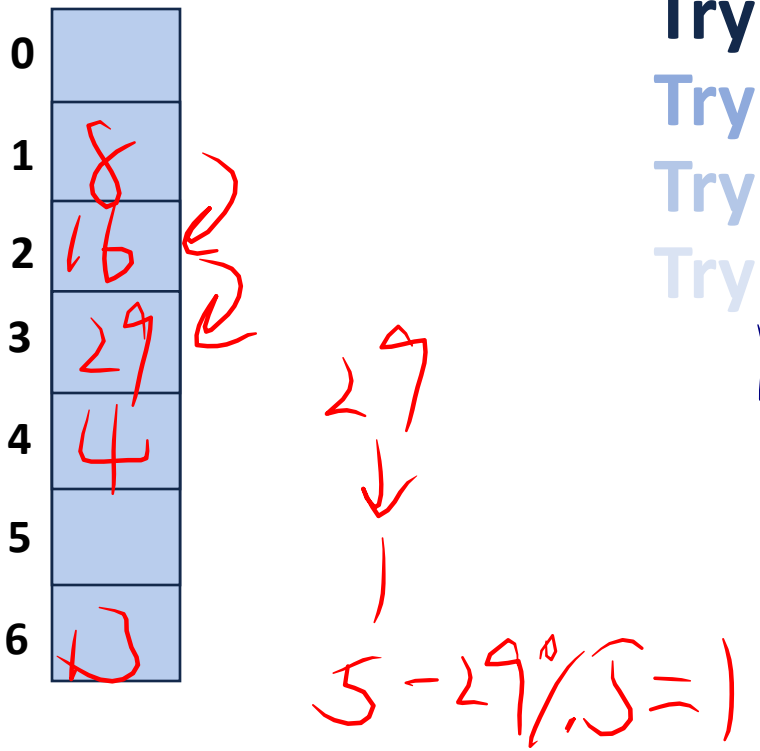
(Example of closed hashing)

Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$



Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

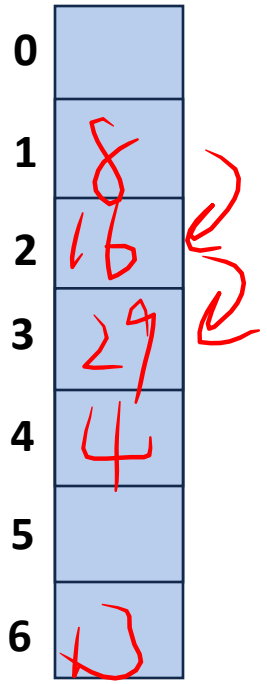
(Example of closed hashing)

Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$



$h_2(11) = 4$

Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

29 → 1 → $5 - 29\%5 = 1$

11 → 4

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

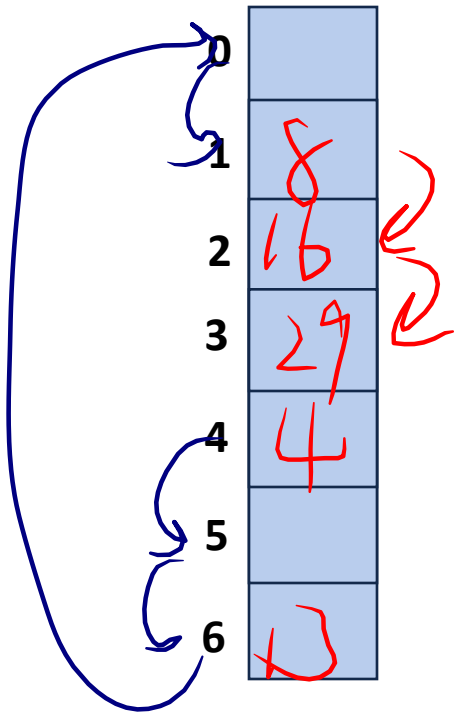
(Example of closed hashing)

Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$



$h_2(11) = 4$

Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

29 → 1

$5 - 29\%5 = 1$

11 → 4

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

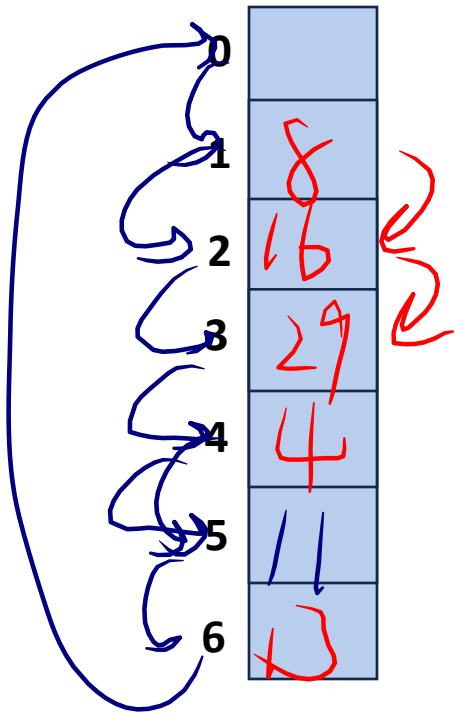
(Example of closed hashing)

Collision Handling: Double hashing

S = { 16, 8, 4, 13, 29, 11, 22 } **|S| = n**

$h_1(k) = k \% 7$ **|Array| = N**

$h_2(k) = 5 - k \% 5$



$h_2(11) = 4$

Try $h(k) = (k + 0*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 1*h_2(k)) \% 7$, if full…

Try $h(k) = (k + 2*h_2(k)) \% 7$, if full…

Try …

29 → 1 → $5 - 29\%5 = 1$

11 → 4

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

Running Times

The expected number of probes for find(key) under SUHA

Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**

Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

Separate Chaining:

- Successful: **1 + α/2**
- Unsuccessful: **1 + α**

(Don't memorize these equations, no need.)

Instead, observe:

- As α increases:

- If α is constant:

Running Times

The expected number of probes for find(key) under SUHA

Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**

Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

Separate Chaining:

- Successful: **1 + α/2**
- Unsuccessful: **1 + α**

(Don't memorize these equations, no need.)

Instead, observe:

- As α increases:

We have slower hash table operations

- If α is constant:

Running Times

The expected number of probes for find(key) under SUHA

Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**

Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

Separate Chaining:

- Successful: **1 + α/2**
- Unsuccessful: **1 + α**

(Don't memorize these equations, no need.)

Instead, observe:

- As α increases:

We have slower hash table operations

- If α is constant:

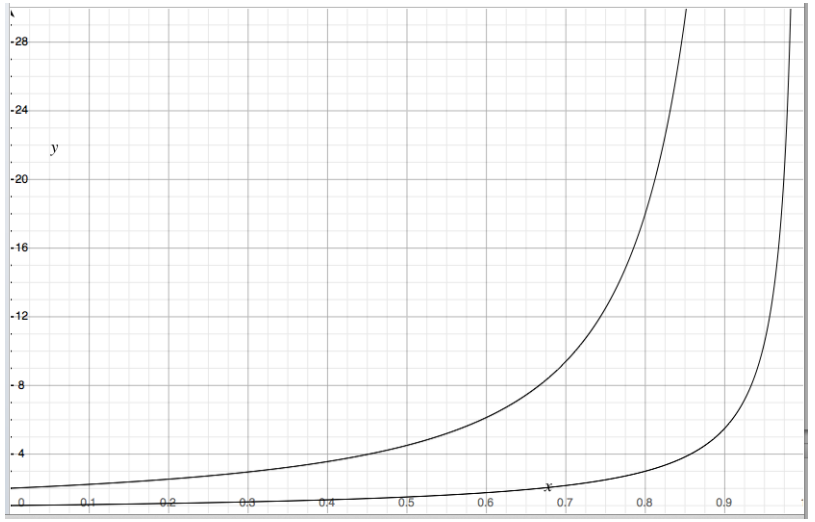
Running time does not change

Running Times

The expected number of probes for find(key) under SUHA

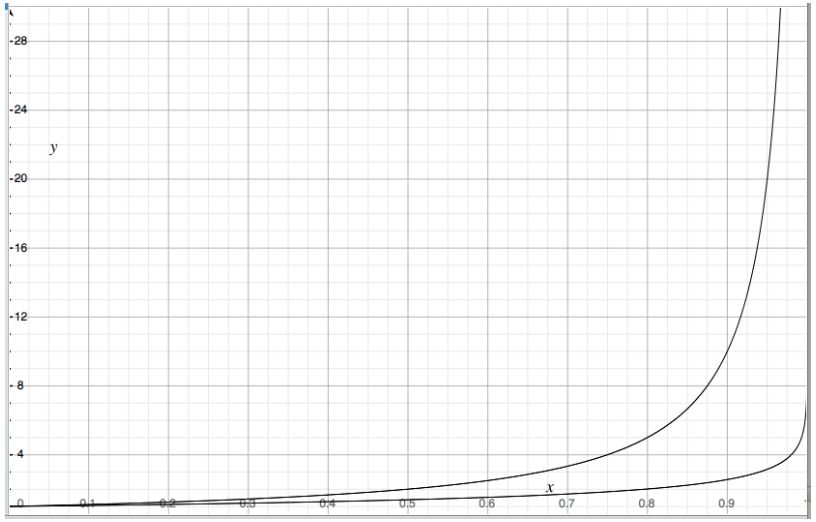
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

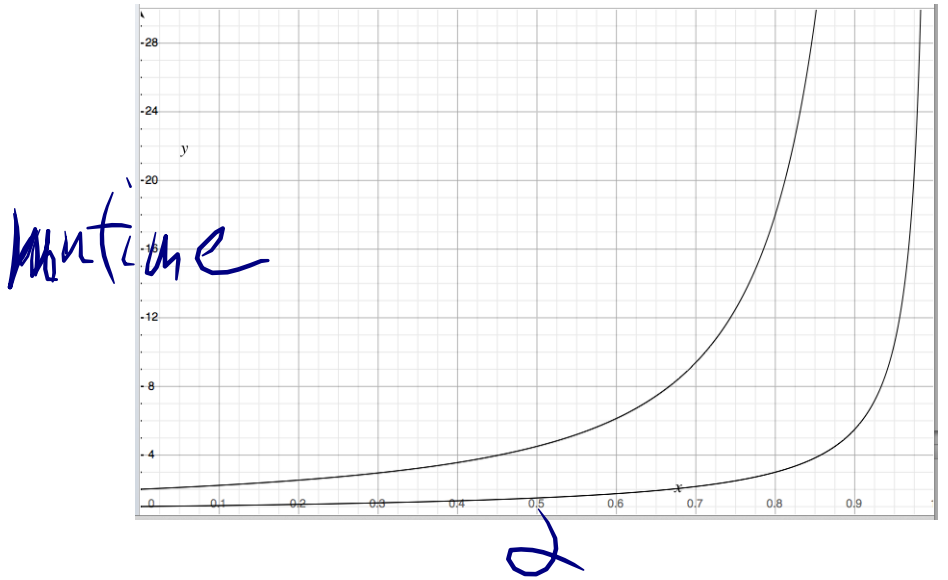


Running Times

The expected number of probes for find(key) under SUHA

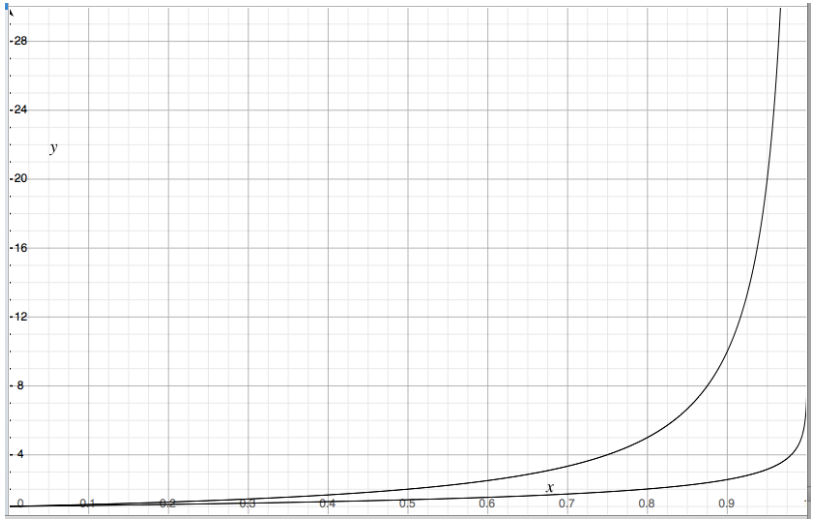
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

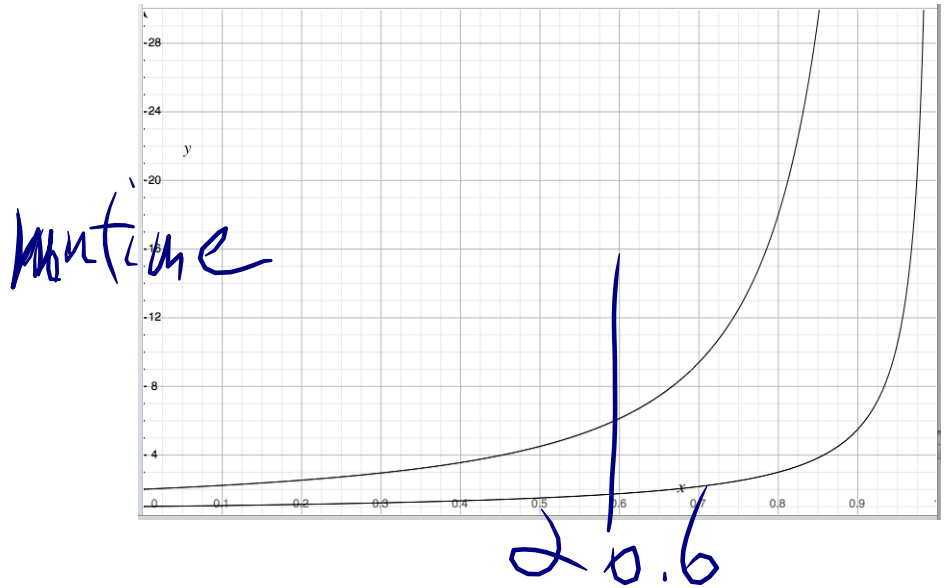


Running Times

The expected number of probes for find(key) under SUHA

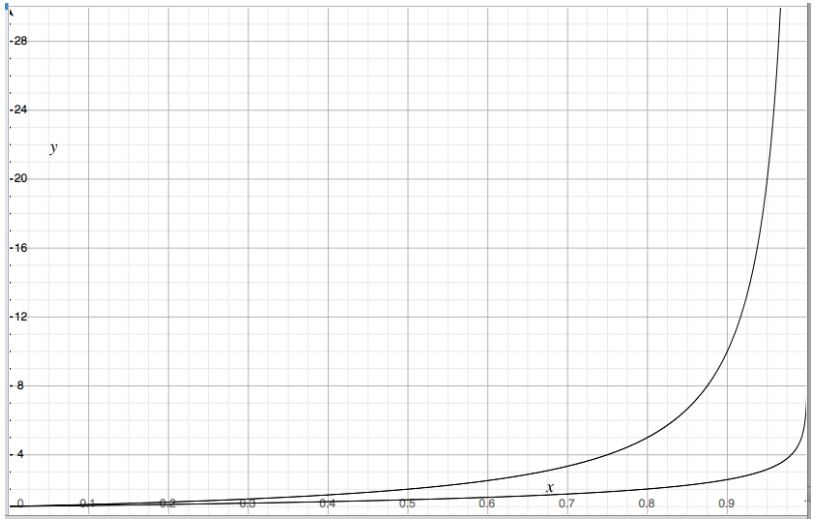
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

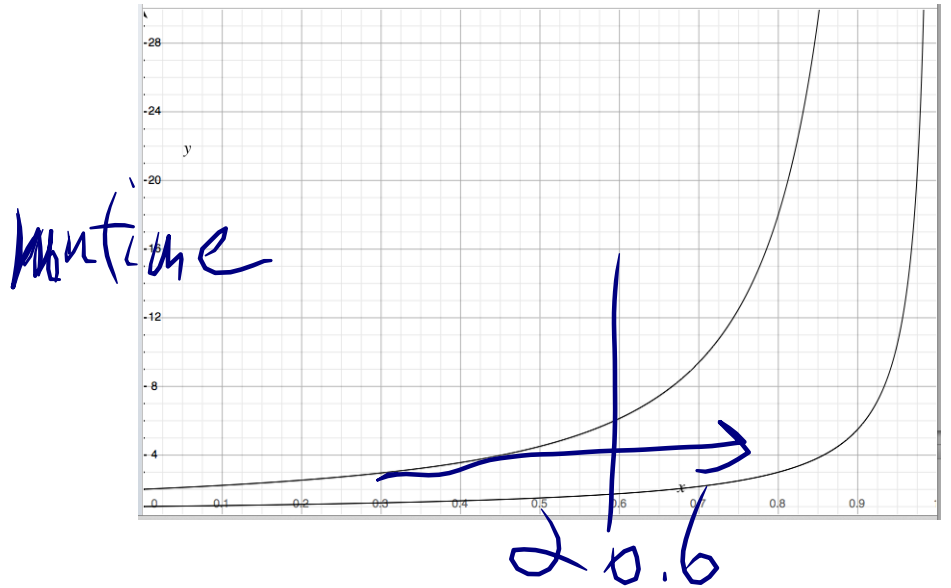


Running Times

The expected number of probes for find(key) under SUHA

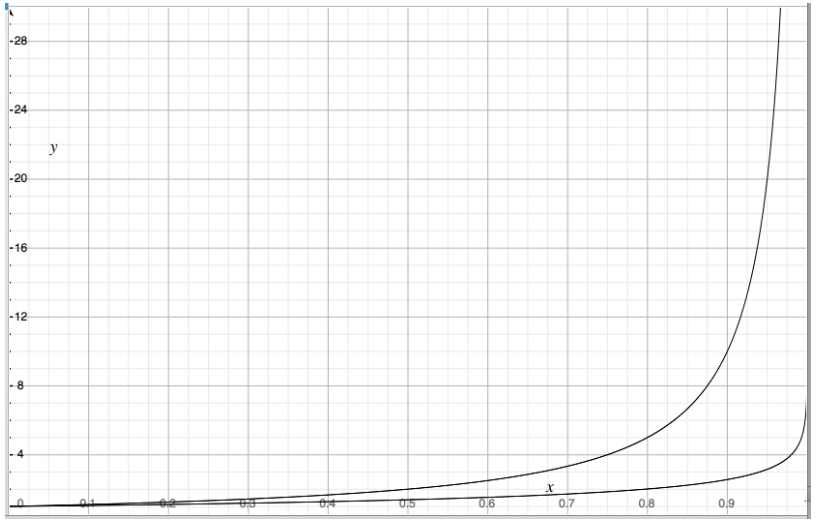
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

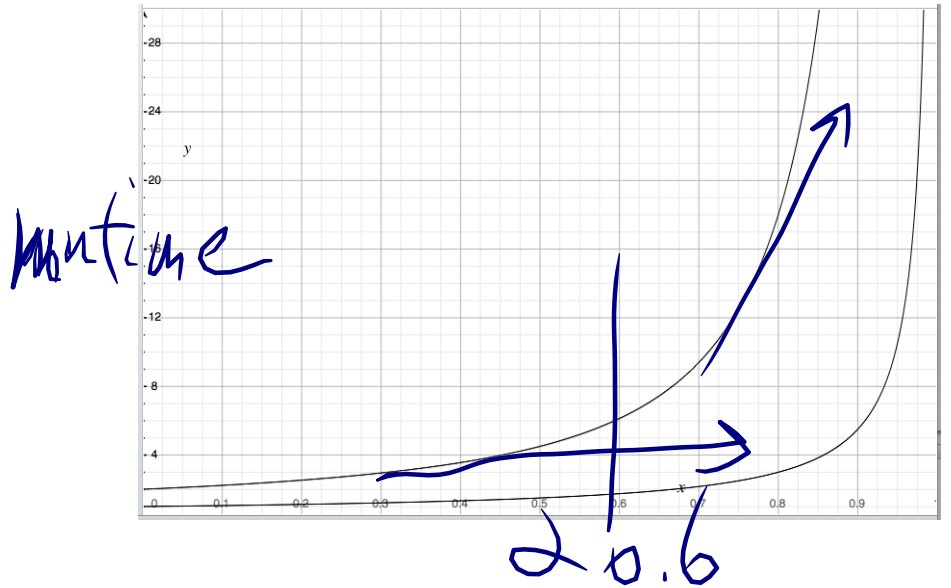


Running Times

The expected number of probes for find(key) under SUHA

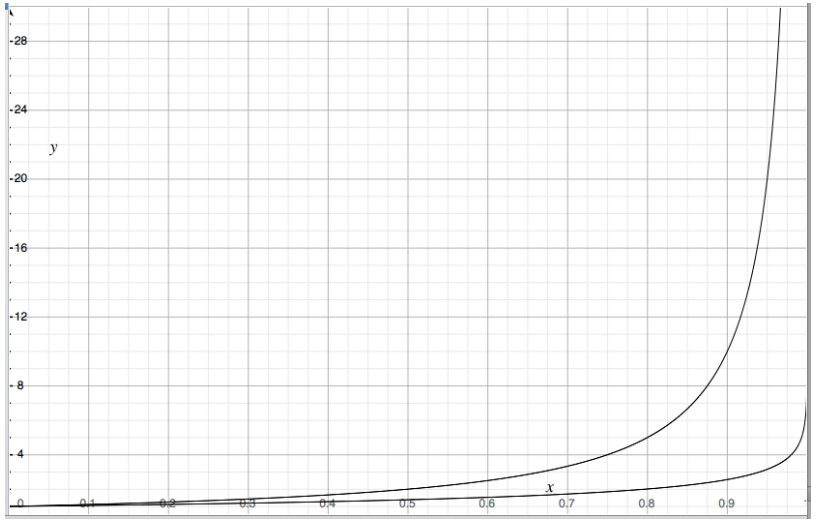
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**



CS 225

Data Structures

Volodymyr Kindratenko

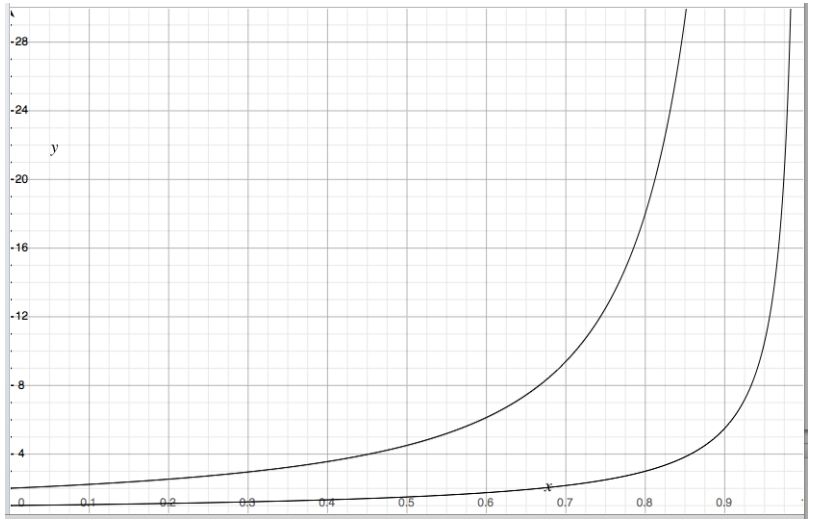
Slides courtesy of Wade Fagen-Ulmschneider

Running Times

The expected number of probes for find(key) under SUHA

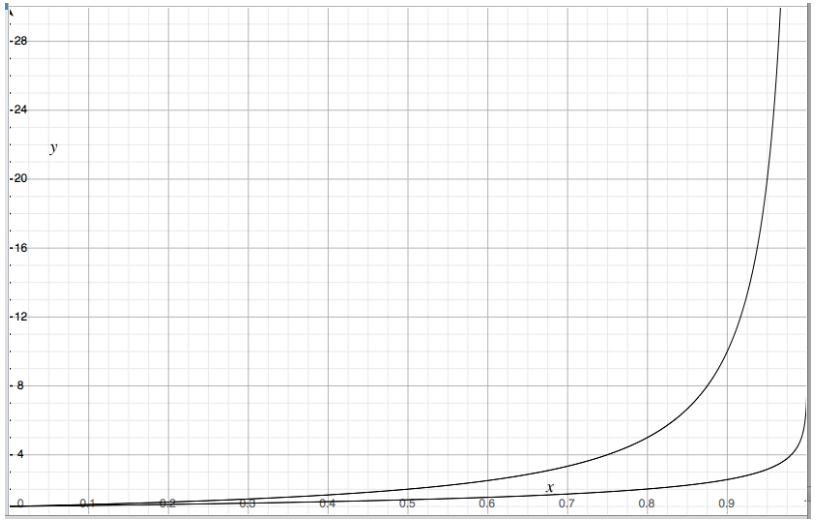
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

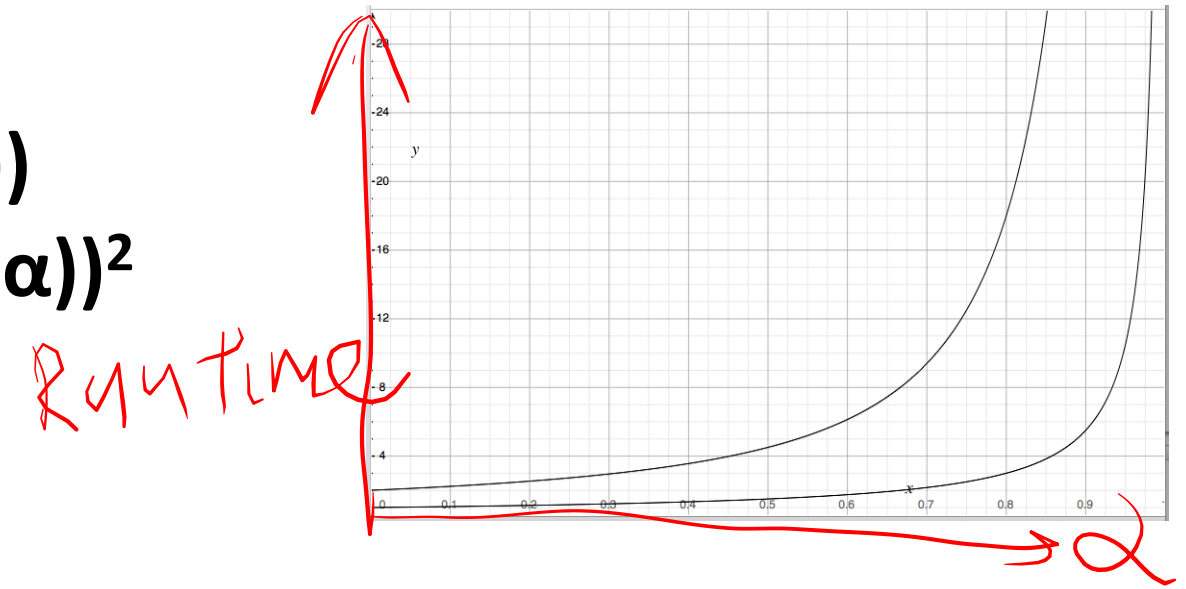


Running Times

The expected number of probes for find(key) under SUHA

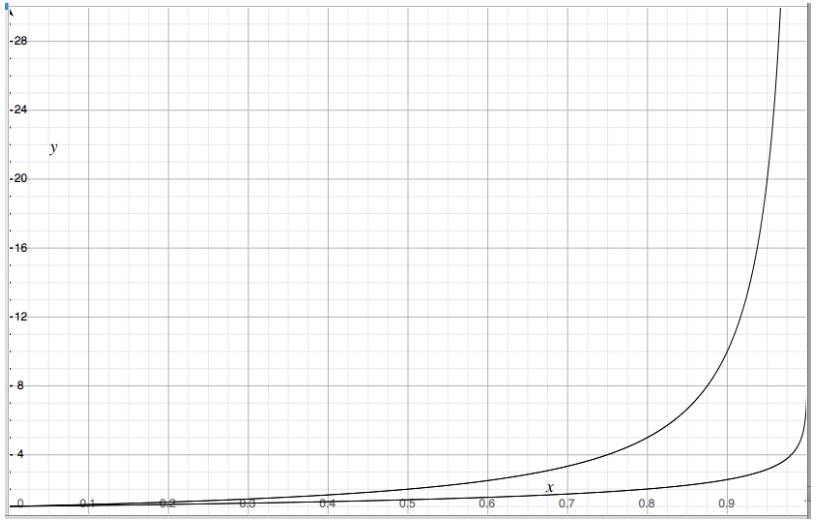
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

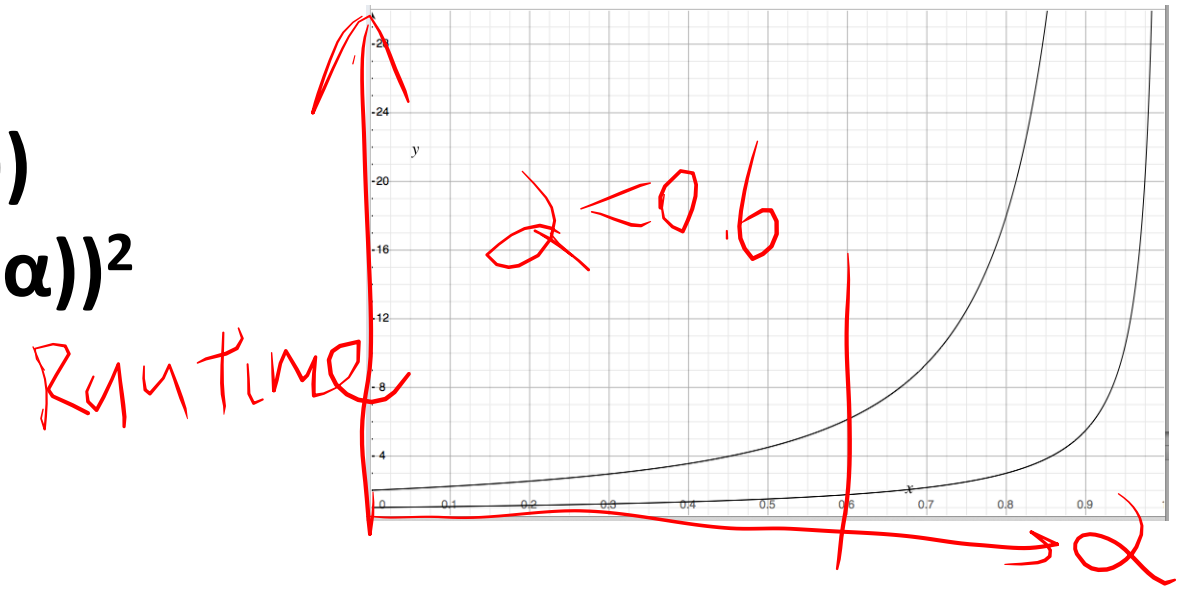


Running Times

The expected number of probes for find(key) under SUHA

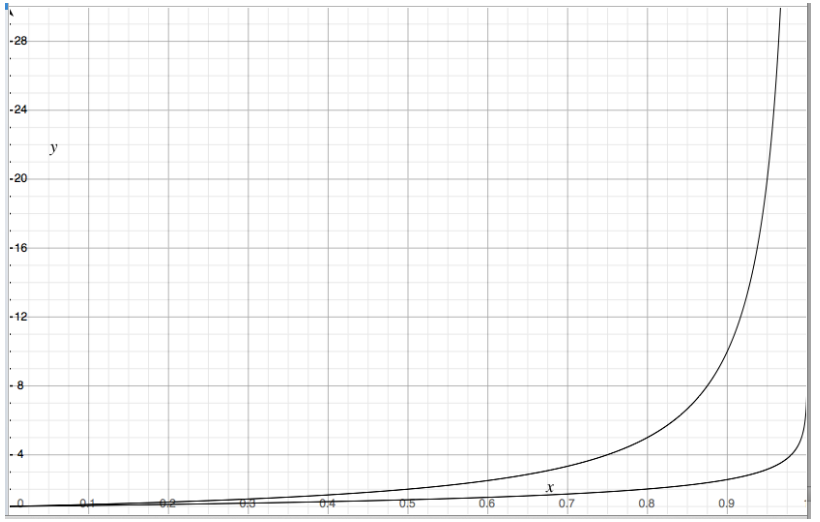
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

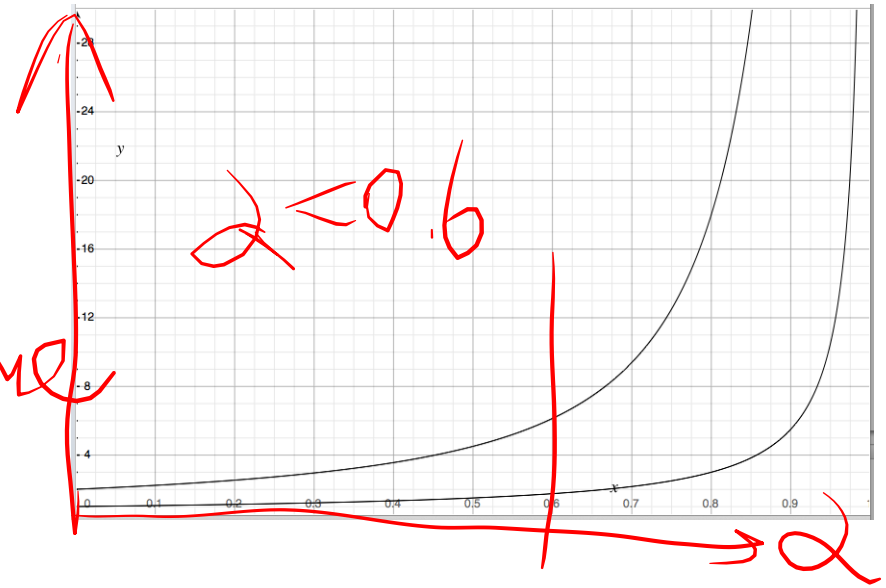


Running Times

The expected number of probes for find(key) under SUHA

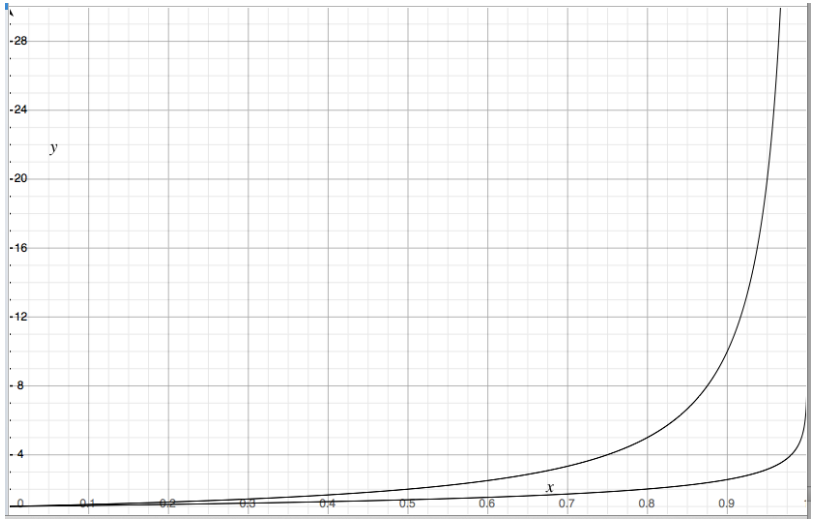
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



Double Hashing:

- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**

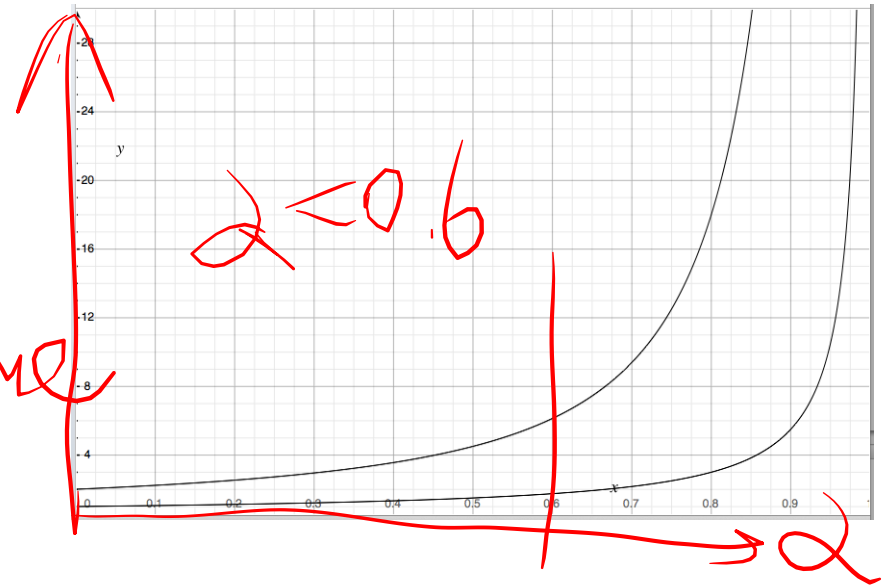


Running Times

The expected number of probes for find(key) under SUHA

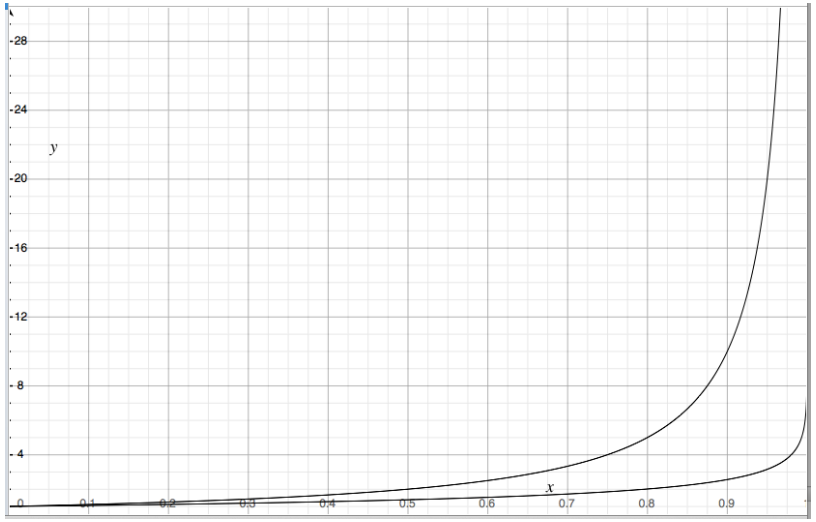
Linear Probing:

- Successful: **½(1 + 1/(1-α))**
- Unsuccessful: **½(1 + 1/(1-α))²**



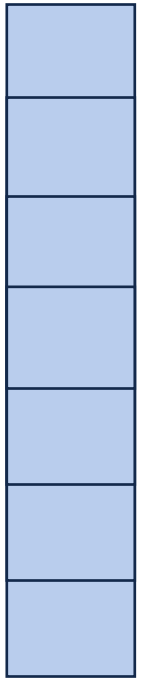
Double Hashing:

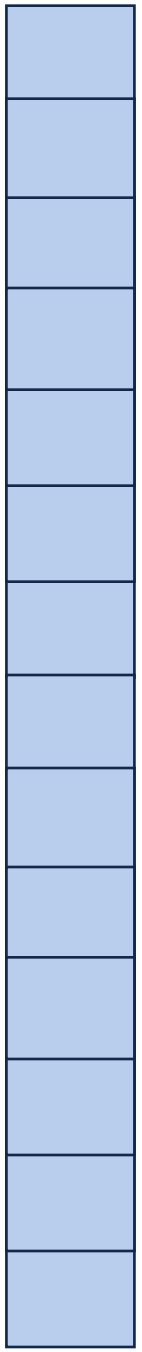
- Successful: **1/α * ln(1/(1-α))**
- Unsuccessful: **1/(1-α)**



ReHashing

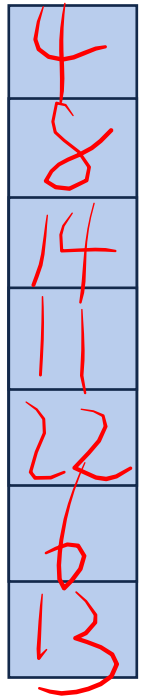
What if the array fills?

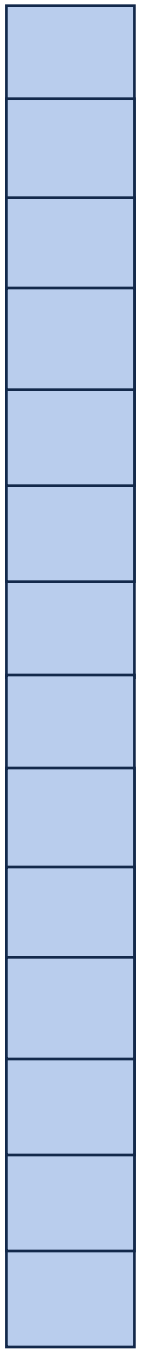




ReHashing

What if the array fills?

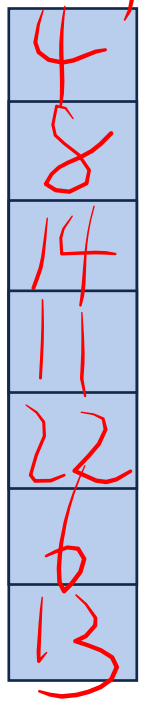


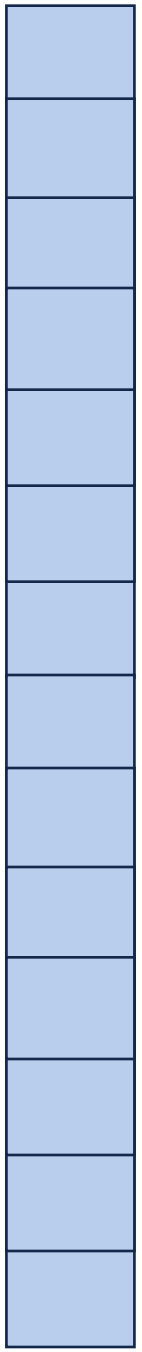


ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?

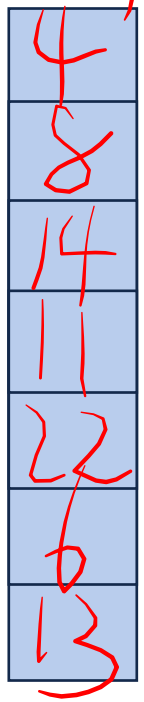


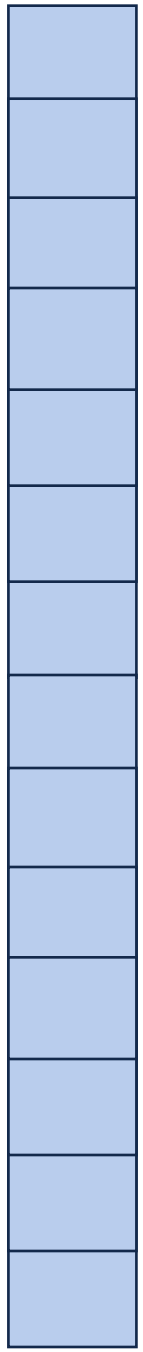


ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?





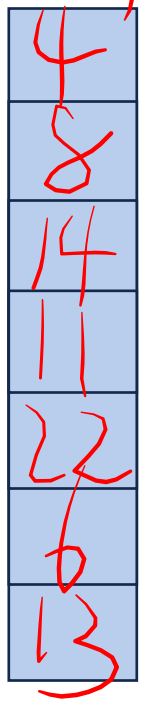
Double size

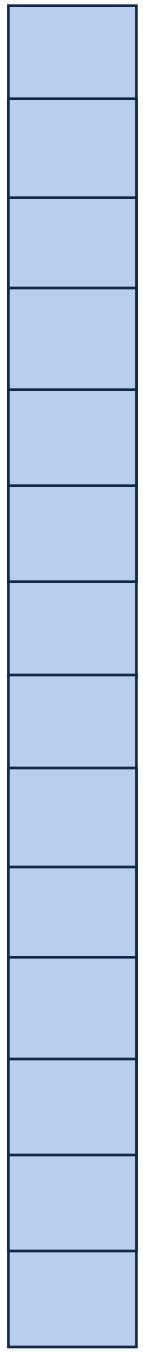
Copy elements

ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?



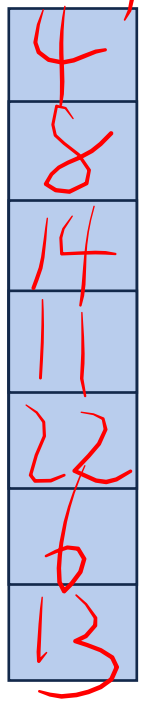


Double size
update hash f
Copy elements

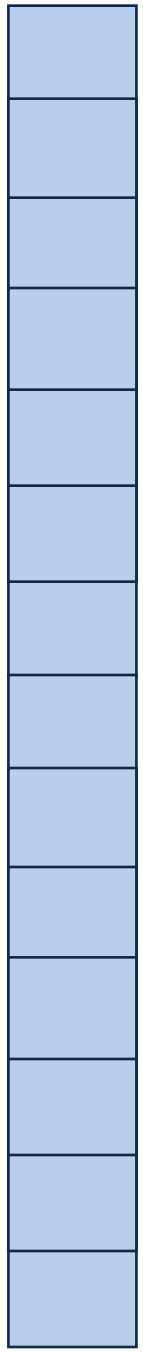
ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?



$h(k) = k \% 7$

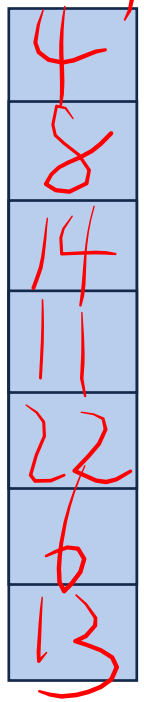


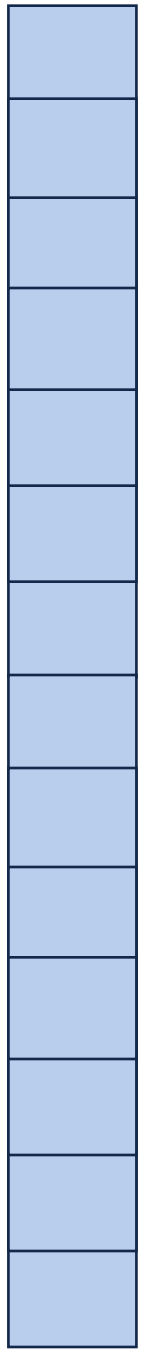
Double size
update hash f
Copy elements

ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?





Double size
update hash f
Copy elements

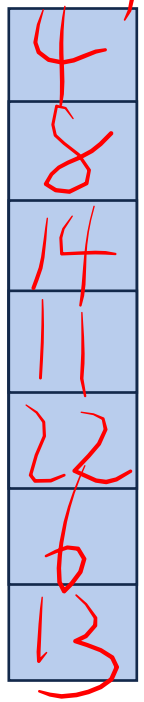
$h(k) = k\%7$

$k\%11$

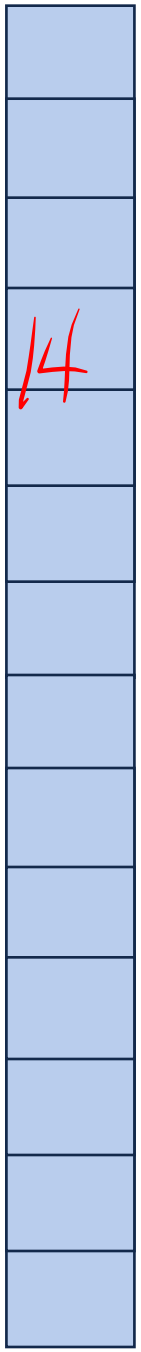
ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?



$h(k) = k\%7$



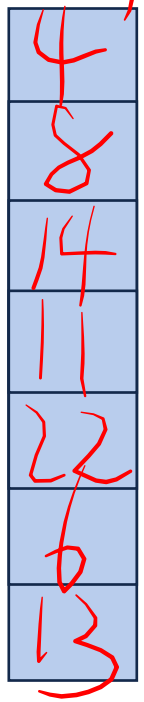
$k\%11$

Double size
update hash f
Copy elements

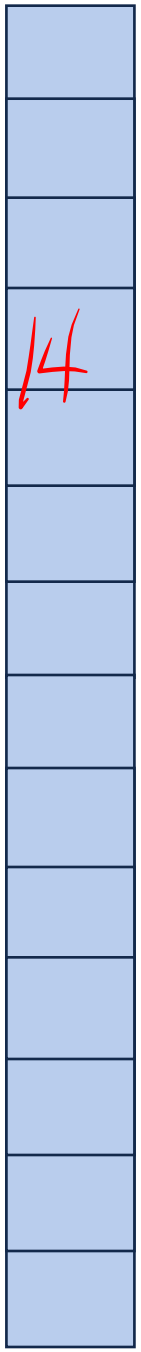
ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?



$h(k) = k \% 7$



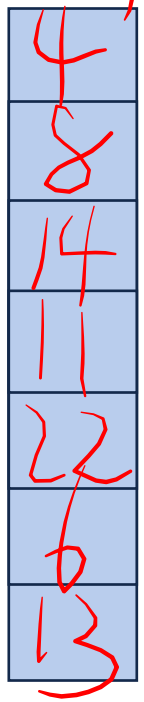
$k \% 11$

Double size
: update hash f
Copy elements
- Rehash elements

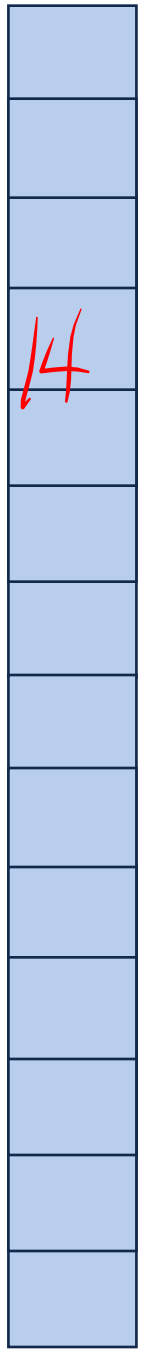
ReHashing

~~What if the array fills?~~

What should happen as $\alpha \to 0.6$?



$h(k) = k \% 7$



Double size
: update hash f

Copy elements

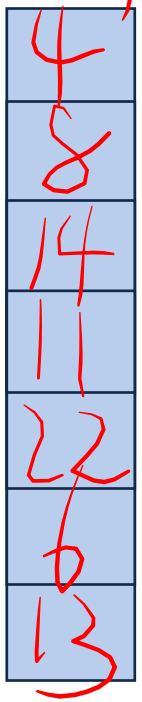
- Rehash elements
- Make to new location

$k \% 11$

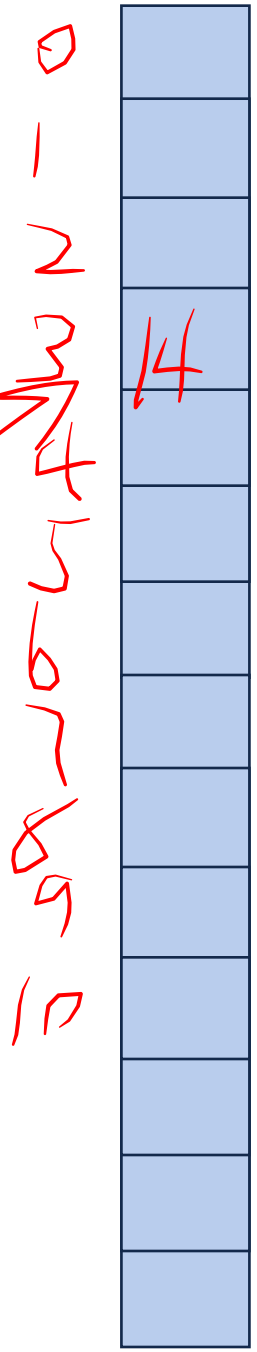
ReHashing

~~What if the array fills?~~

What should happen as $2 \to 0.6$?



$h(k) = k\%7$



Double size
: update hash f

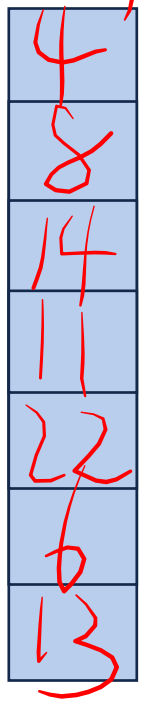
Copy elements
- Rehash elements
- Make to new location

$k\%11$

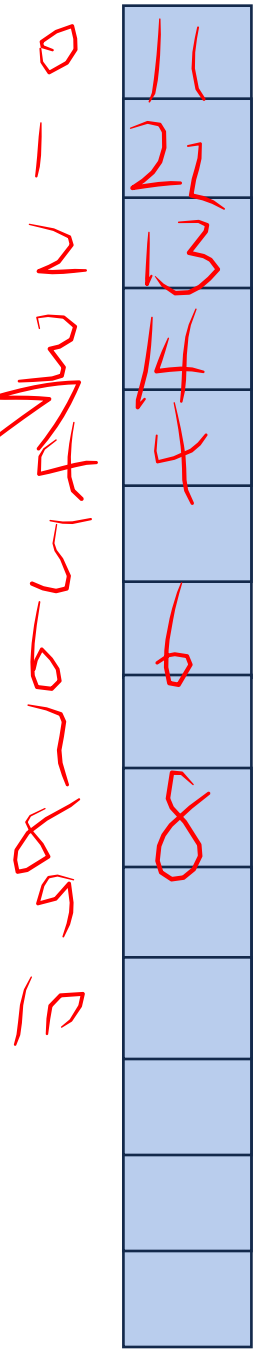
ReHashing

~~What if the array fills?~~

What should happen as 2 → 0.6?



$h(k) = k\%7$



Double size
: update hash f

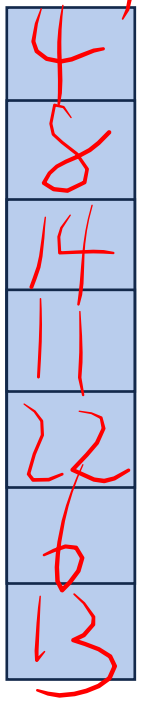
Copy elements
- Rehash elements
- Make to new location

$k\%11$

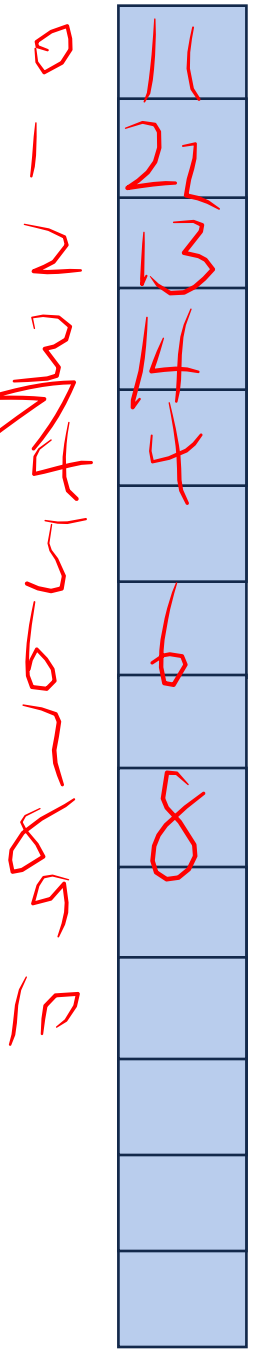
ReHashing

~~What if the array fills?~~

What should happen as a → 0.6?



$h(k) = k\%7$



Double size
: update hash f

Copy elements
- Rehash elements
- Make to new location O(1)

$k\%11$

O(1)

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: Worst Case:		
Insert	Amortized: Worst Case:		
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: O(1) Worst Case:		
Insert	Amortized: Worst Case:		
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: O(1) Worst Case: O(n)		
Insert	Amortized: Worst Case:		
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: Worst Case:		
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case:		
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$		
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space			

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space			$O(n)$

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space		$O(n)$	$O(n)$

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space		$O(n)$	$O(n)$ 1 xptr/elem

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space		$O(n)$ 2 x ptrs / elem	$O(n)$ 1 x ptr / elem

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space	$N = \frac{n}{0.6} \approx 1.5n$	$O(n)$ 2 x ptrs/elem	$O(n)$ 1 x ptr/elem

Running Times

	Hash Table	AVL	Linked List
Find	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(n)$
Insert	Amortized: $O(1)$ Worst Case: $O(n)$	$O(\lg(n))$	$O(1)$
Storage Space	$N = \frac{n}{0.6} \approx 1.5n$ $O(n)$	$O(n)$ 2 xptrs/elem	$O(n)$ 1 xptr/elem

Which collision resolution strategy is better?

- Big Records:

- Structure Speed:

What structure do hash tables replace?

What constraint exists on hashing that doesn't exist with BSTs?

Why talk about BSTs at all?

Which collision resolution strategy is better?

- Big Records: Sc → "Open Hashing"

- Structure Speed:

What structure do hash tables replace?

What constraint exists on hashing that doesn't exist with BSTs?

Why talk about BSTs at all?

Which collision resolution strategy is better?

- Big Records:
- Structure Speed:

Sc → "Open Hashing"

Double Hashing → "closed Hashing"

What structure do hash tables replace?

What constraint exists on hashing that doesn't exist with BSTs?

Why talk about BSTs at all?

Which collision resolution strategy is better?

- Big Records:
- Structure Speed:

SC → "Open Hashing"

Double Hashing → "closed Hashing"

What structure do hash tables replace?

List, AVL

What constraint exists on hashing that doesn't exist with BSTs?

Why talk about BSTs at all?

Which collision resolution strategy is better?

- Big Records: Sc → "Open Hashing"
- Structure Speed: Double Hashing → "closed Hashing"

What structure do hash tables replace?

List, AVL

What constraint exists on hashing that doesn't exist with BSTs?

We need the key, AVL can find nearby keys

Why talk about BSTs at all?

Which collision resolution strategy is better?

- Big Records: Sc → "Open Hashing"
- Structure Speed: Double Hashing → "closed Hashing"

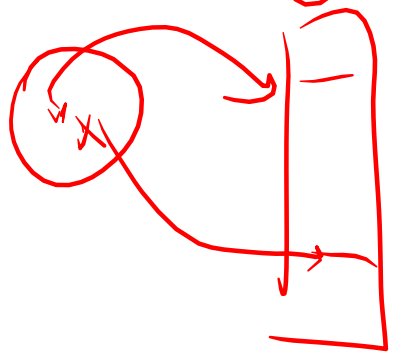
What structure do hash tables replace?

List, AVL

What constraint exists on hashing that doesn't exist with BSTs?

We need the key, AVL can find nearby keys

Why talk about BSTs at all?



Which collision resolution strategy is better?

- Big Records:
- Structure Speed:

SC → "Open Hashing"

Double Hashing → "closed Hashing"

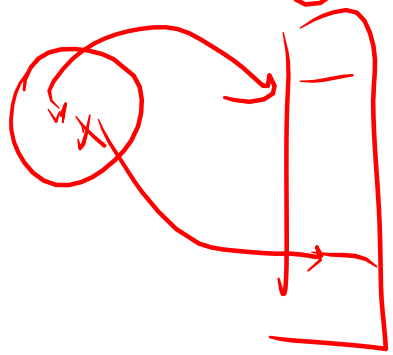
What structure do hash tables replace?

List, AVL

What constraint exists on hashing that doesn't exist with BSTs?

We need the key, AVL can find nearby keys

Why talk about BSTs at all?



★ Nearest neighbor

Ranges [10, 100]

std data structures

std::map

std data structures

std::map

operator[]

insert

evase

std data structures

std::map

::operator[]
::insert
::erase

::lower_bound(key) ➔ Iterator to first element ≤ key
::upper_bound(key) ➔ Iterator to first element > key

std data structures

→ Impl w/ a balanced BST

std::map

::operator[]

::insert

::erase

::lower_bound(key) ➔ Iterator to first element ≤ key

::upper_bound(key) ➔ Iterator to first element > key

std data structures

std::unordered_map

::operator[]

::insert

::erase

~~– ::lower_bound(key) → Iterator to first element ≤ key~~

~~– ::upper_bound(key) → Iterator to first element > key~~

std data structures

std::unordered_map

::operator[]

::insert

::erase

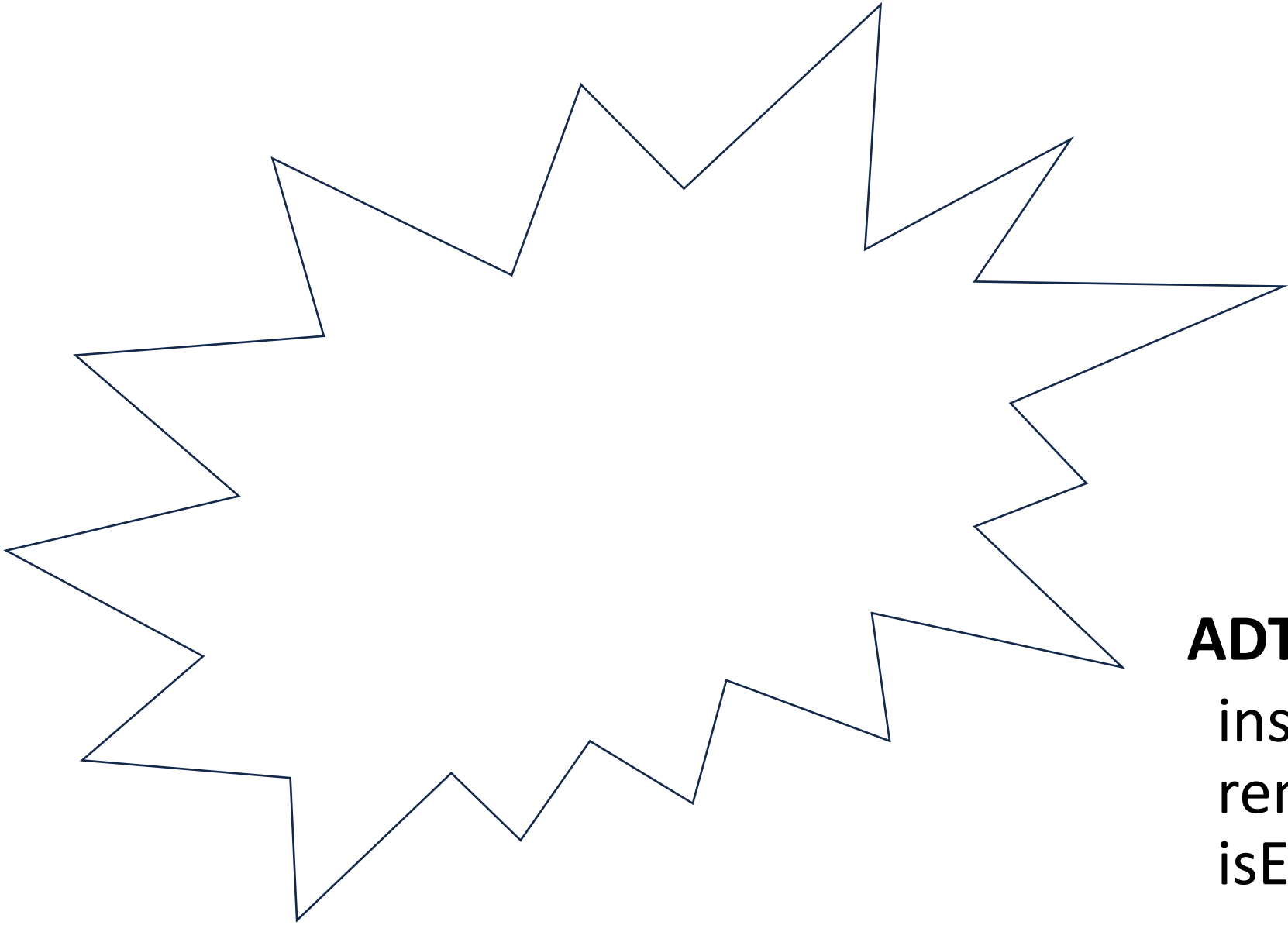
~~::lower_bound(key) → Iterator to first element ≤ key~~

~~::upper_bound(key) → Iterator to first element > key~~

::load_factor()

::max_load_factor(ml) → Sets the max load factor

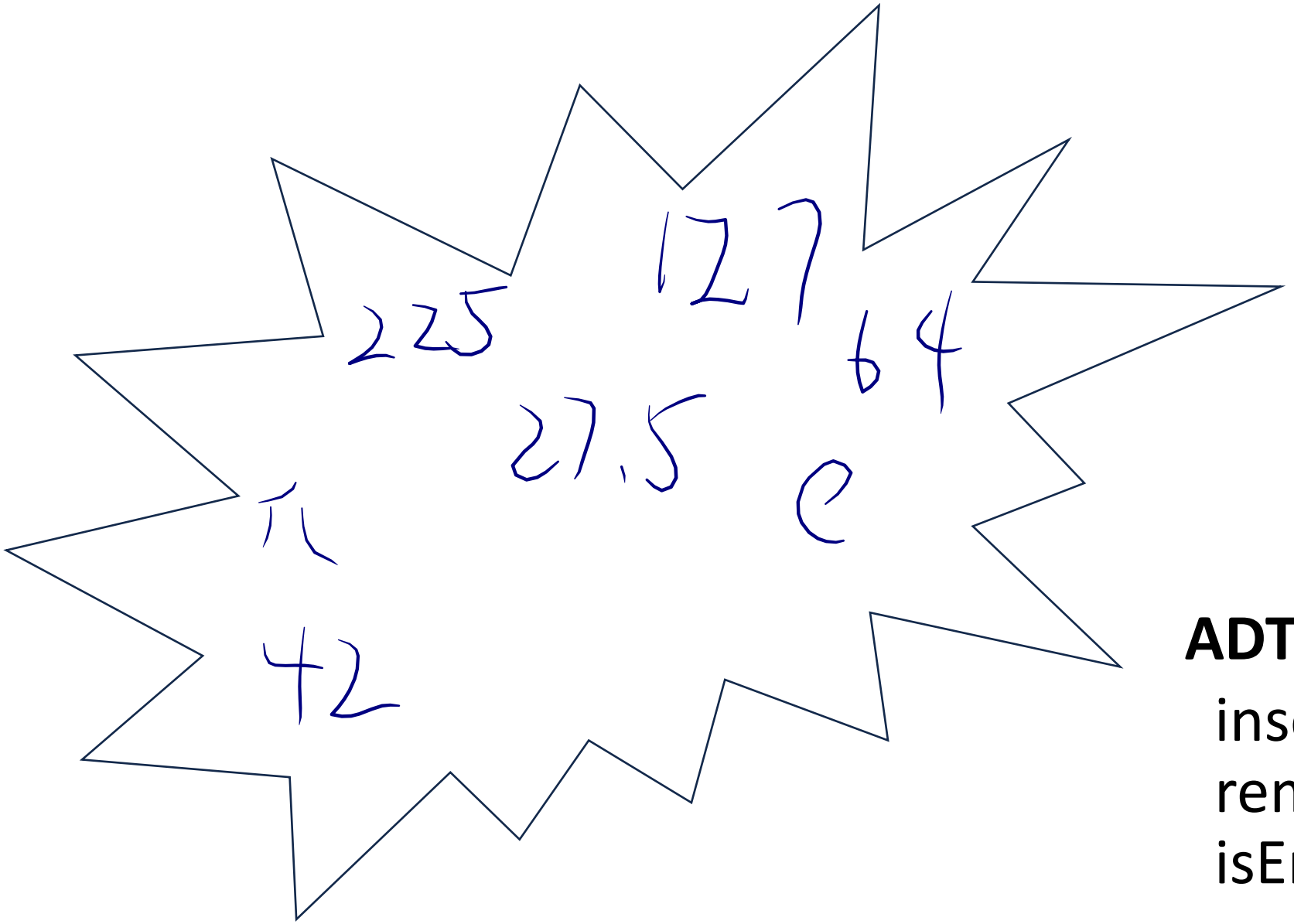
Secret, Mystery Data Structure



ADT:

insert
remove
isEmpty

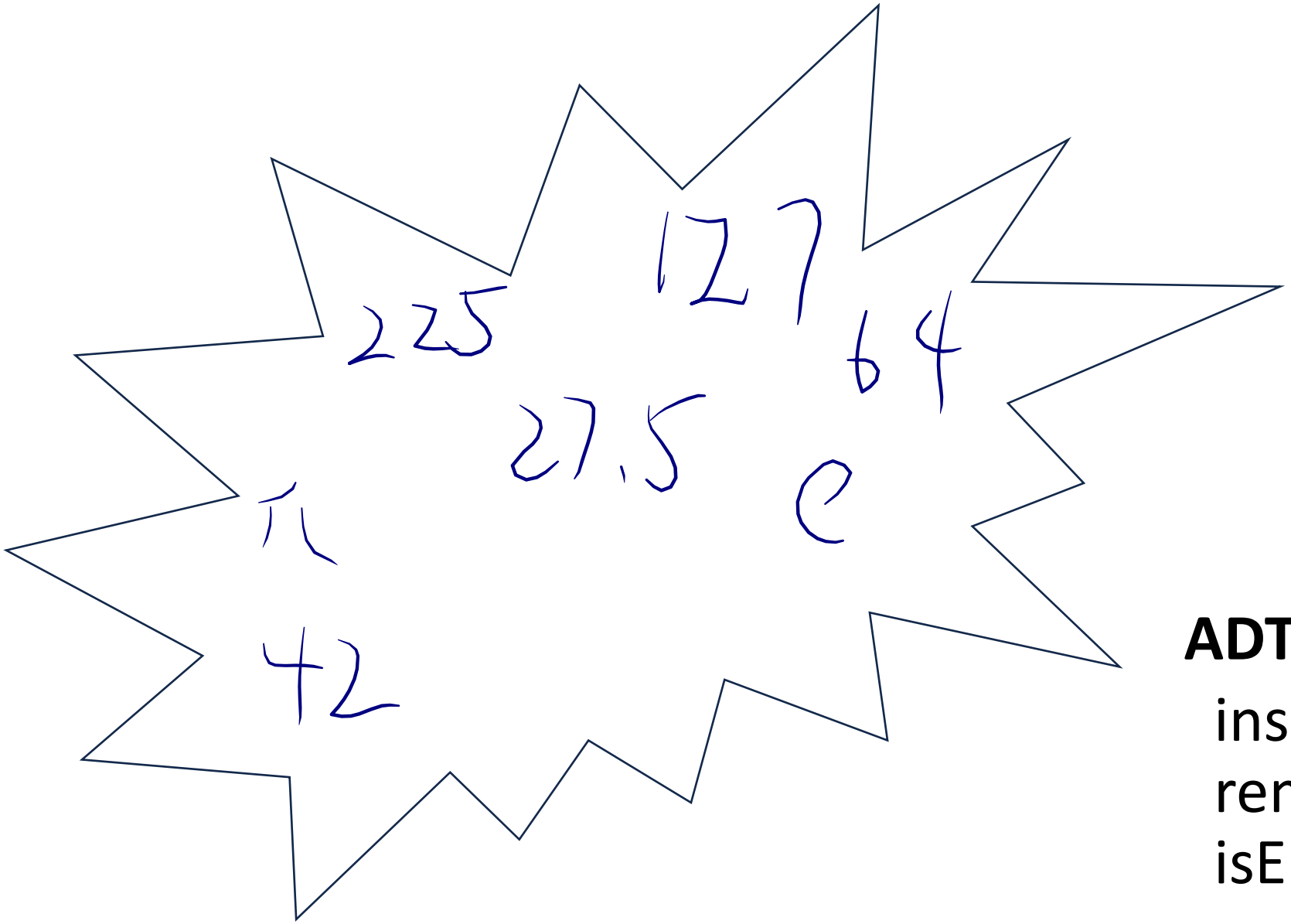
Secret, Mystery Data Structure



ADT:

insert
remove
isEmpty

Secret, Mystery Data Structure



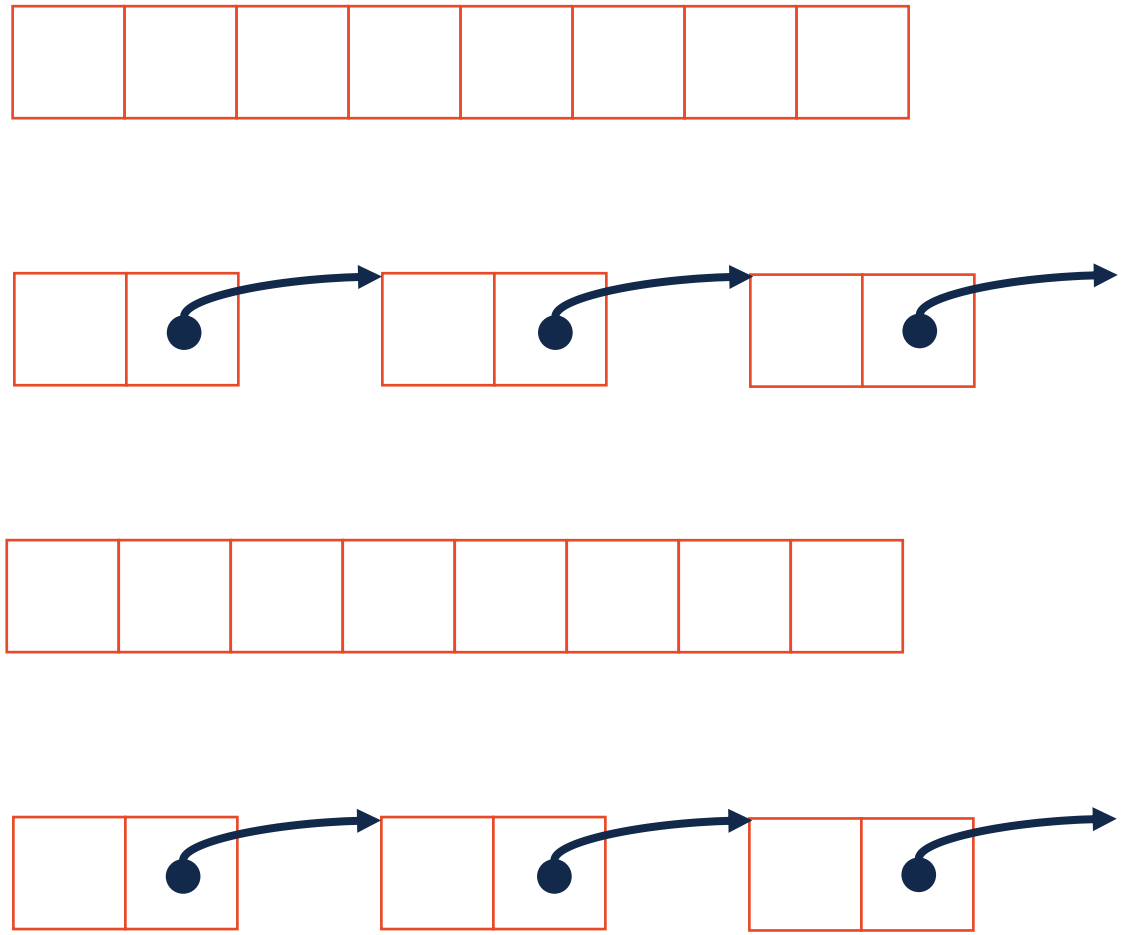
ADT:

insert
remove
isEmpty

min()

Priority Queue Implementation

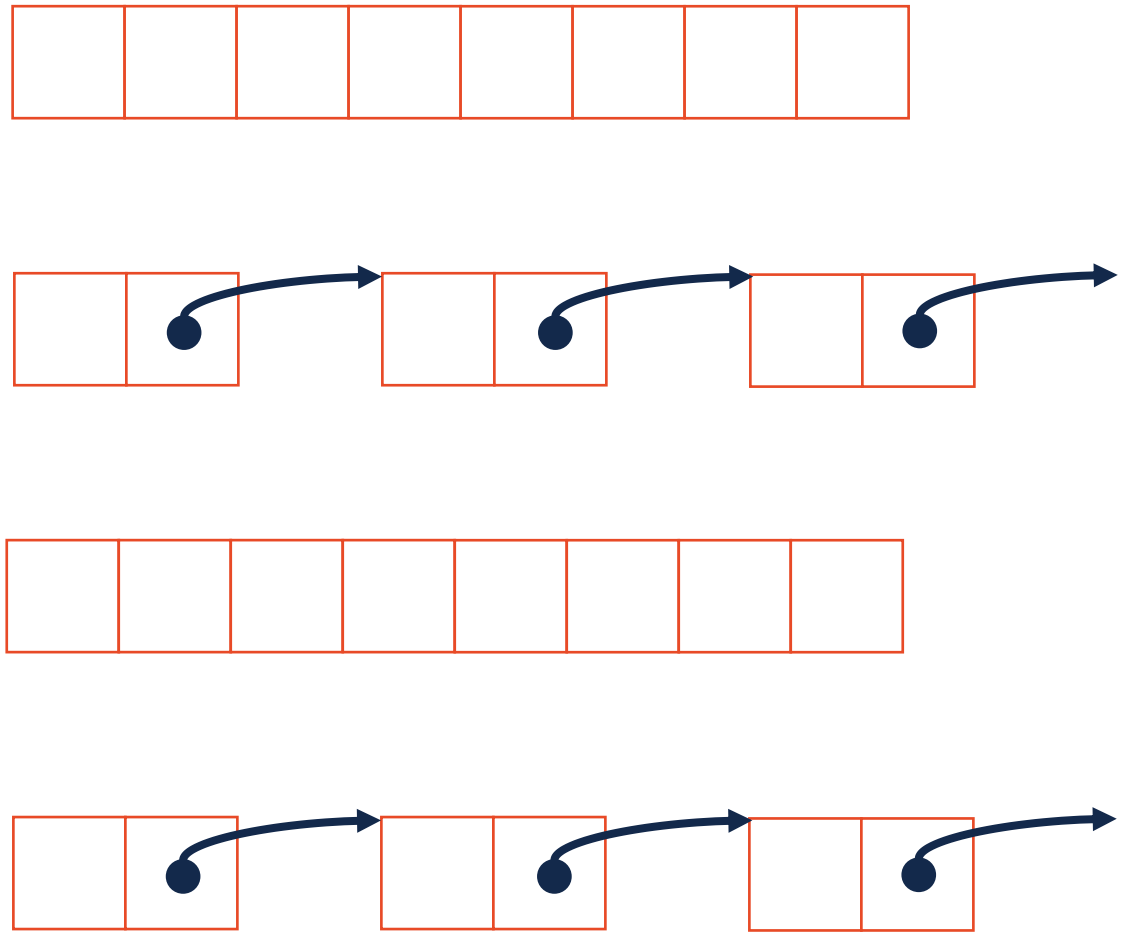
insert	removeMin
O(n)	O(n)
O(1)	O(n)
O(lg(n))	O(1)
O(lg(n))	O(1)



Priority Queue Implementation

insert	removeMin
O(n)	O(n)
O(1)	O(n)
~~O(lg(n))~~ O(4)	O(1)
O(lg(n))	O(1)

find O(lgn)
insert push

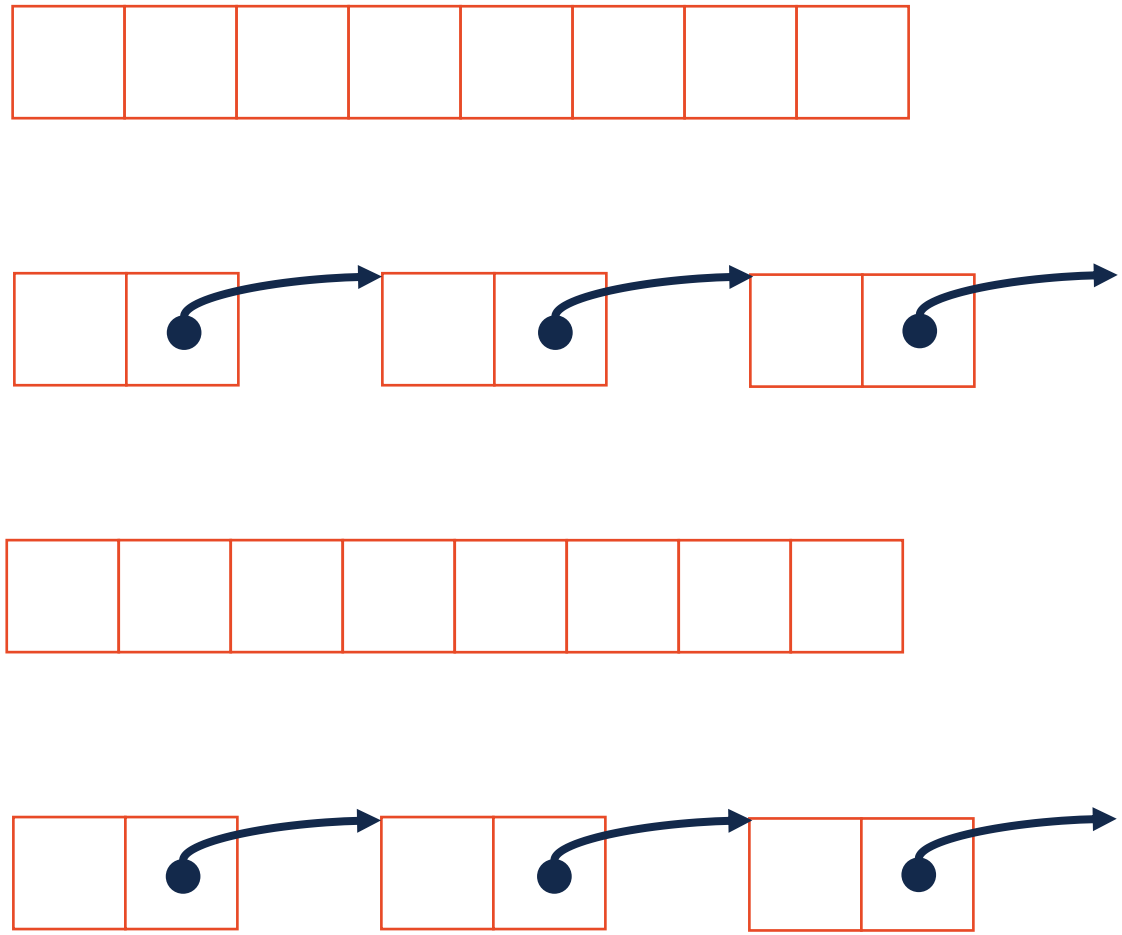


Priority Queue Implementation

insert	removeMin
O(n)	O(n)
O(1)	O(n)
~~O(lg(n))~~ O(n)	O(1)
~~O(lg(n))~~ O(n)	O(1)

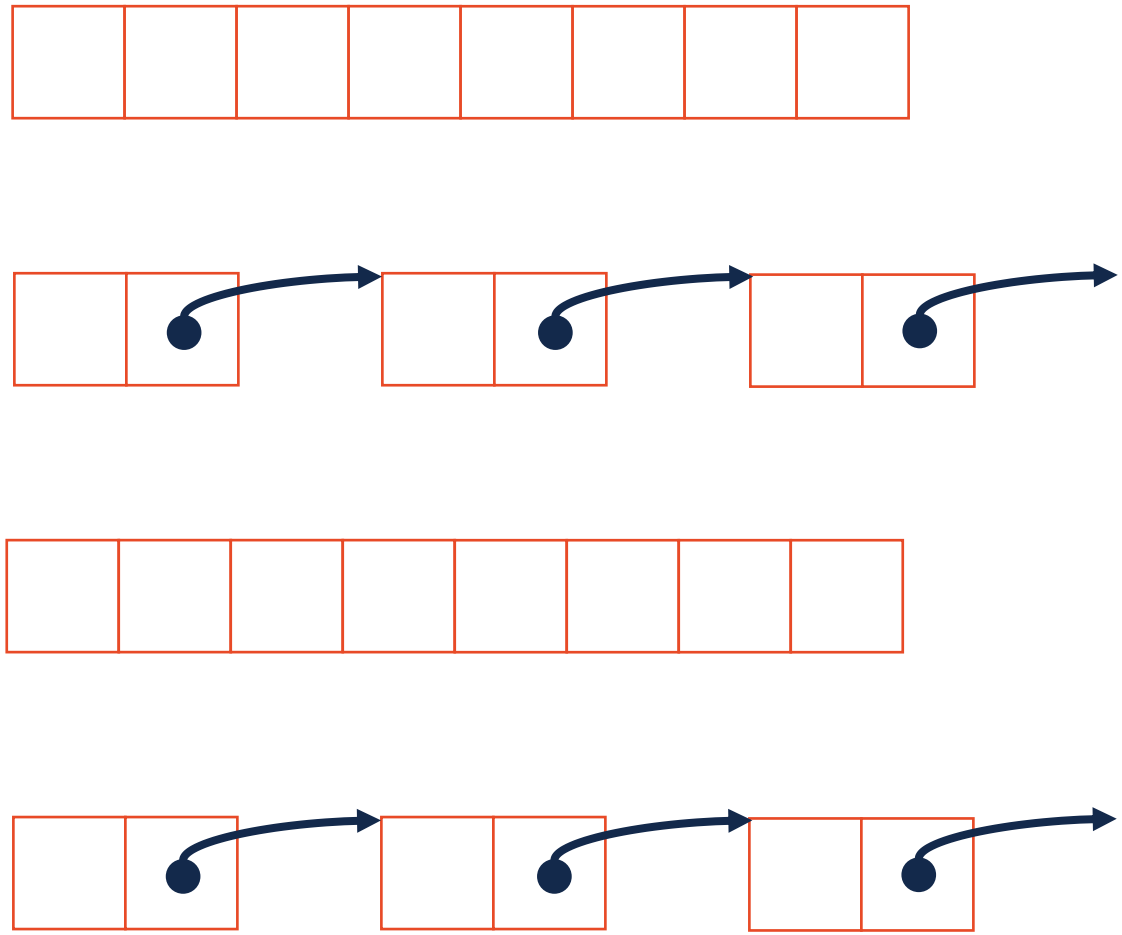
find O(lgn)
insert push O(1)

find O(n)
insert O(1)
+ O(n)



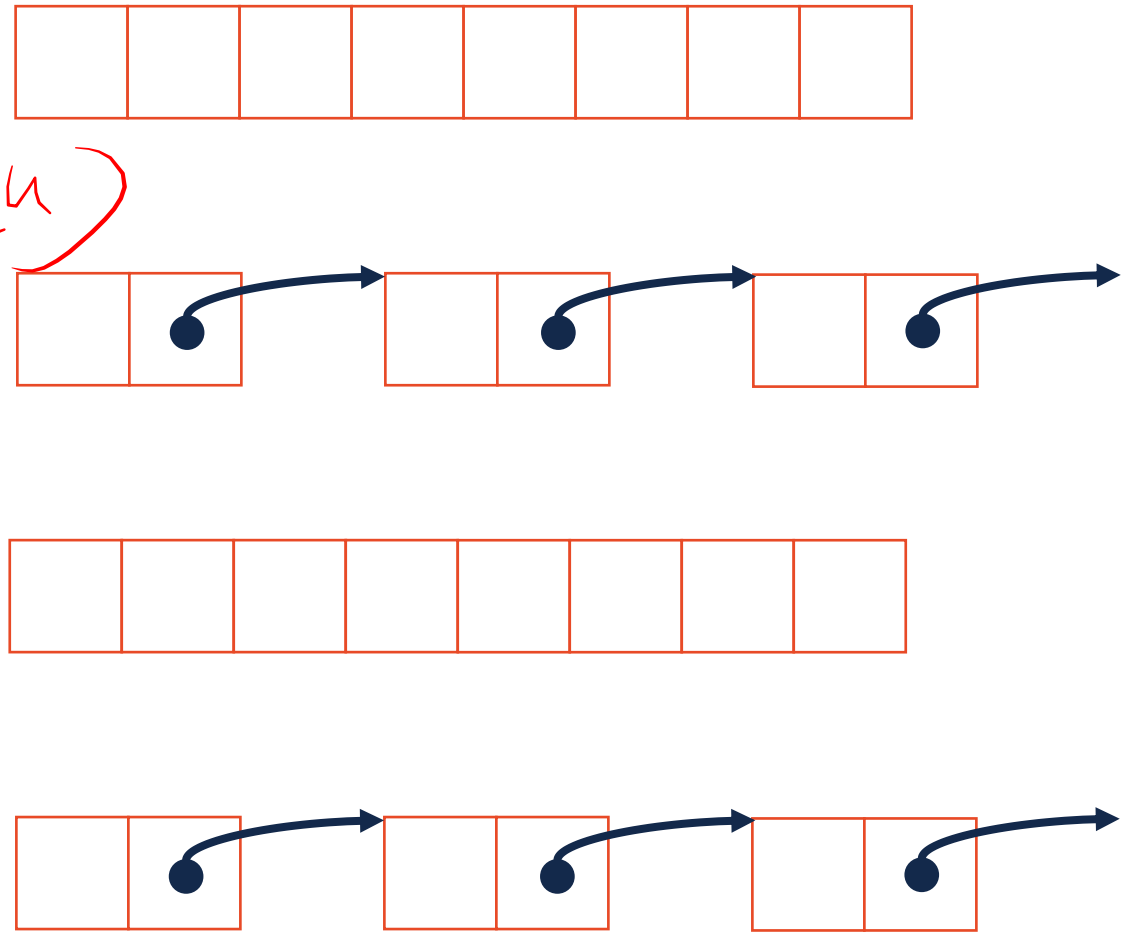
Priority Queue Implementation

insert	removeMin
~~O(n)~~ O(1)	O(n)
O(1)	O(n)
~~O(lg(n))~~ O(4) find O(lgn) insert push	O(1)
~~O(lg(n))~~ find O(n) insert O(1) + O(n)	O(1)

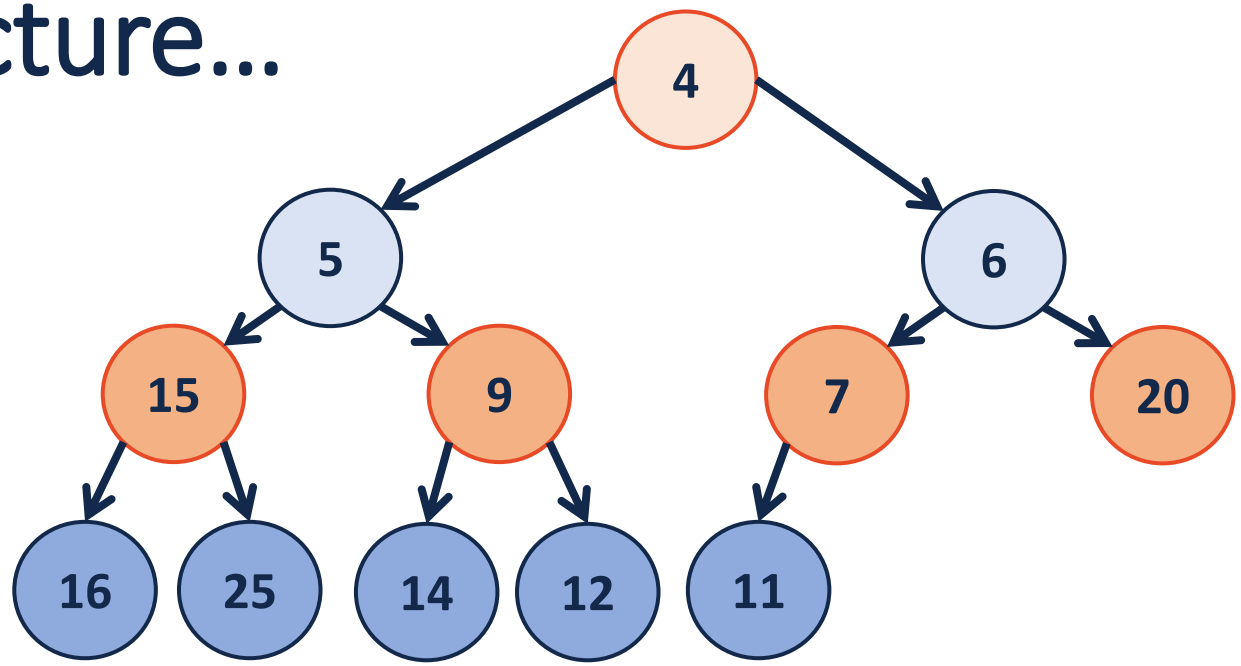


Priority Queue Implementation

insert	removeMin
O(1)	O(n)
O(1)	O(n)
O(n)	O(1)
O(n)	O(1)

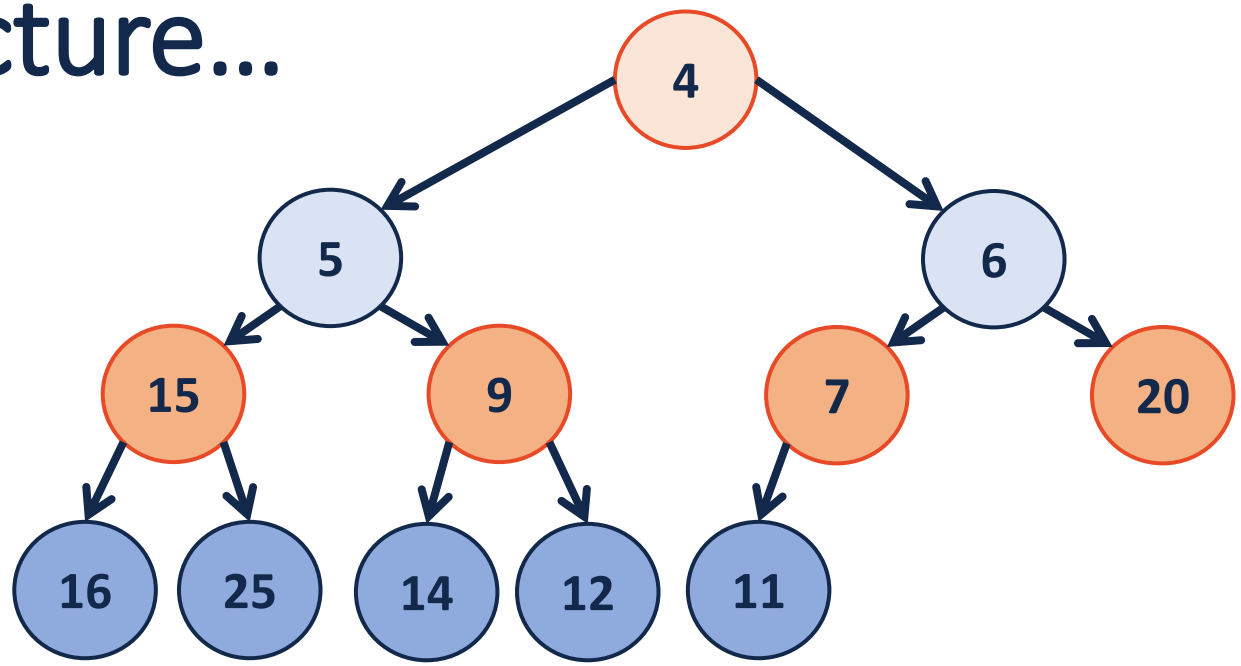


Another possible structure…



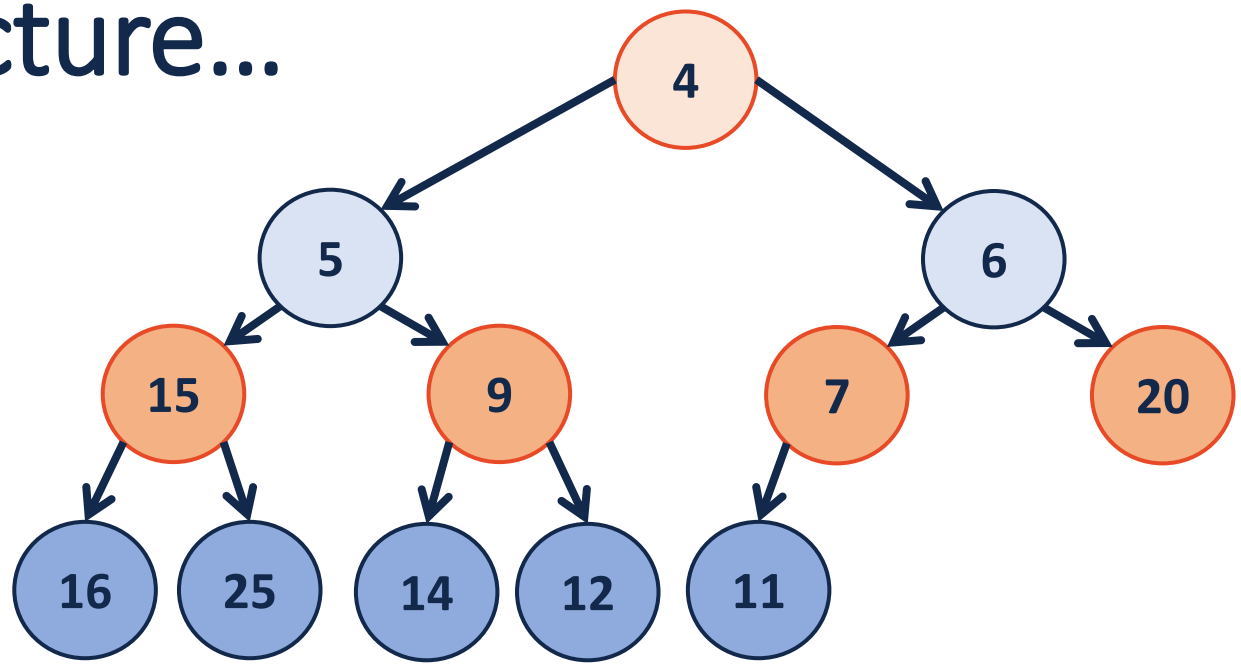
Another possible structure…

Binary Tree



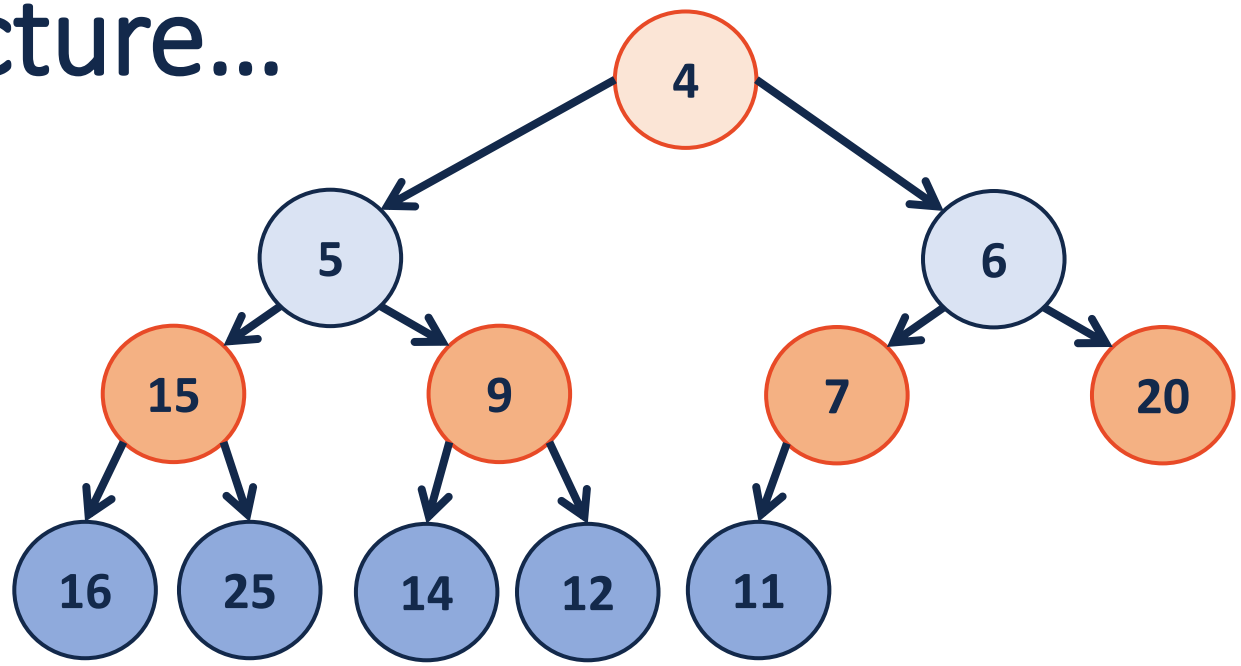
Another possible structure…

Binary Tree
- Complete



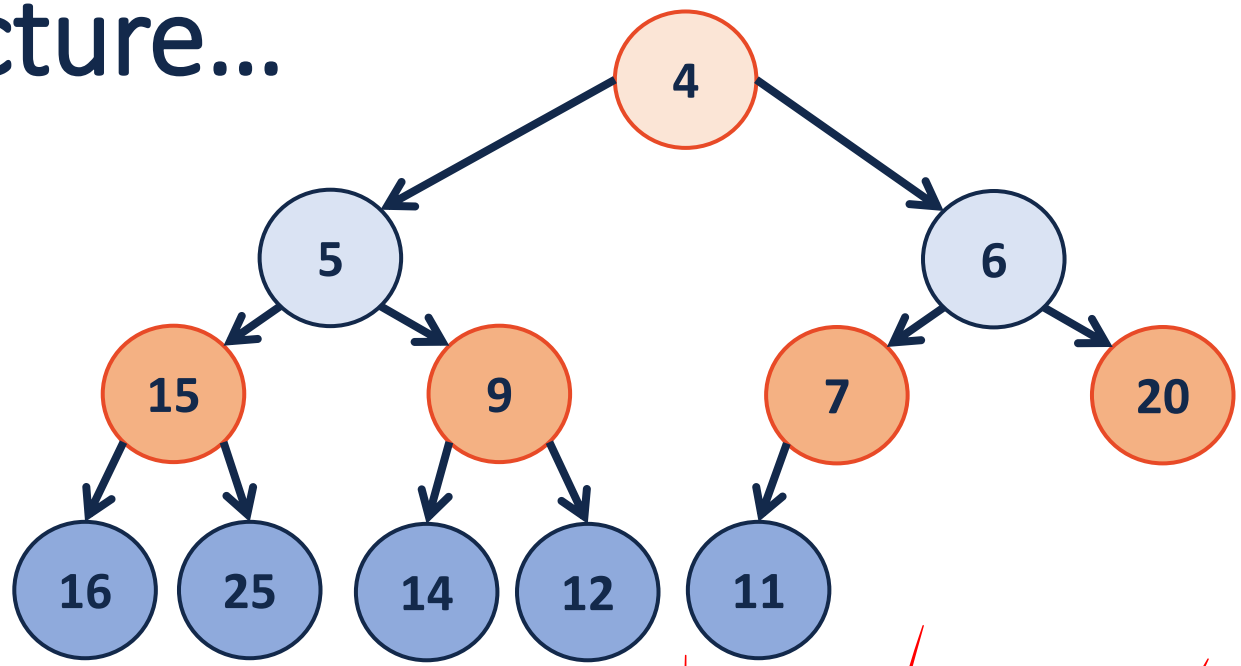
Another possible structure…

Binary Tree
- Complete
- !BST



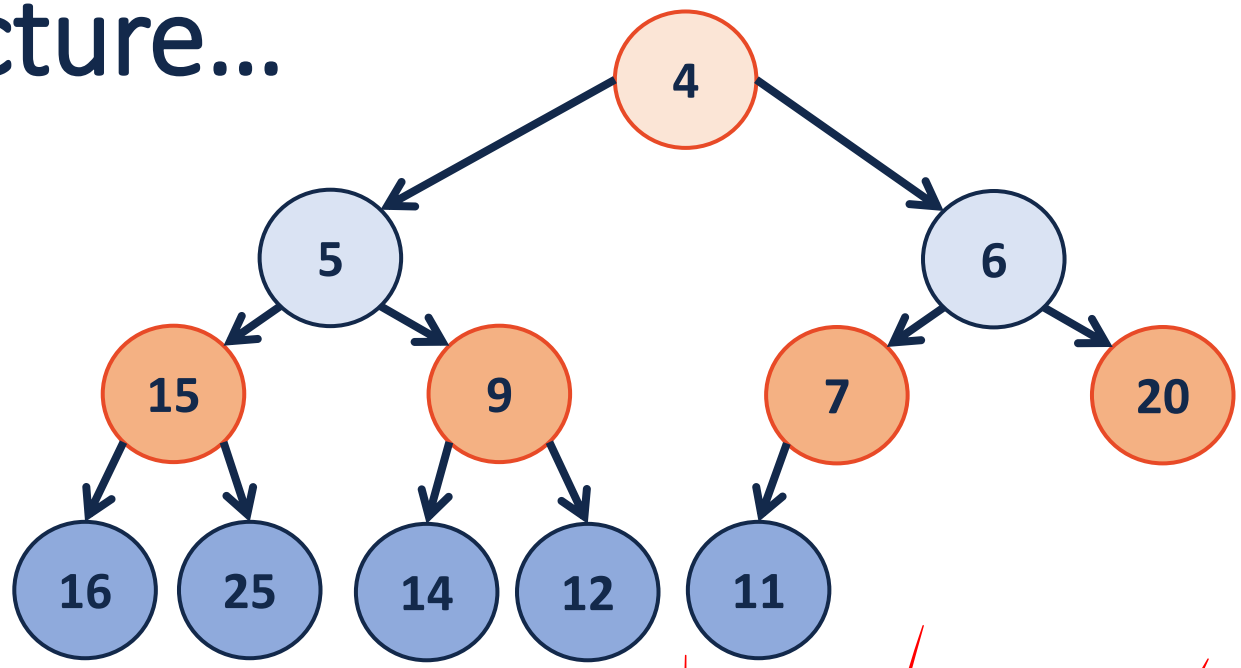
Another possible structure…

Binary Tree
- Complete
- !BST



- All descendents of a node > the value of the node

Another possible structure...



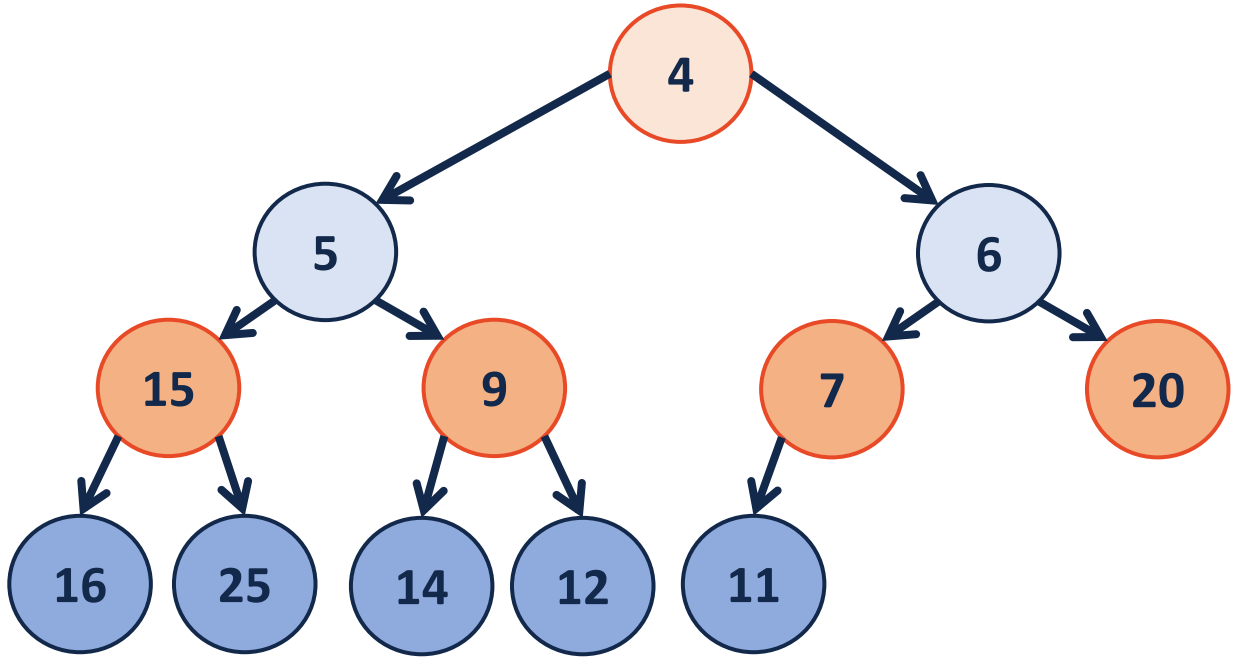
Binary Tree
- Complete
- !BST

- All descendents of a node > the value of the node
  - ↳ Root is the smallest element

(min)Heap

A complete binary tree T is a min-heap if:

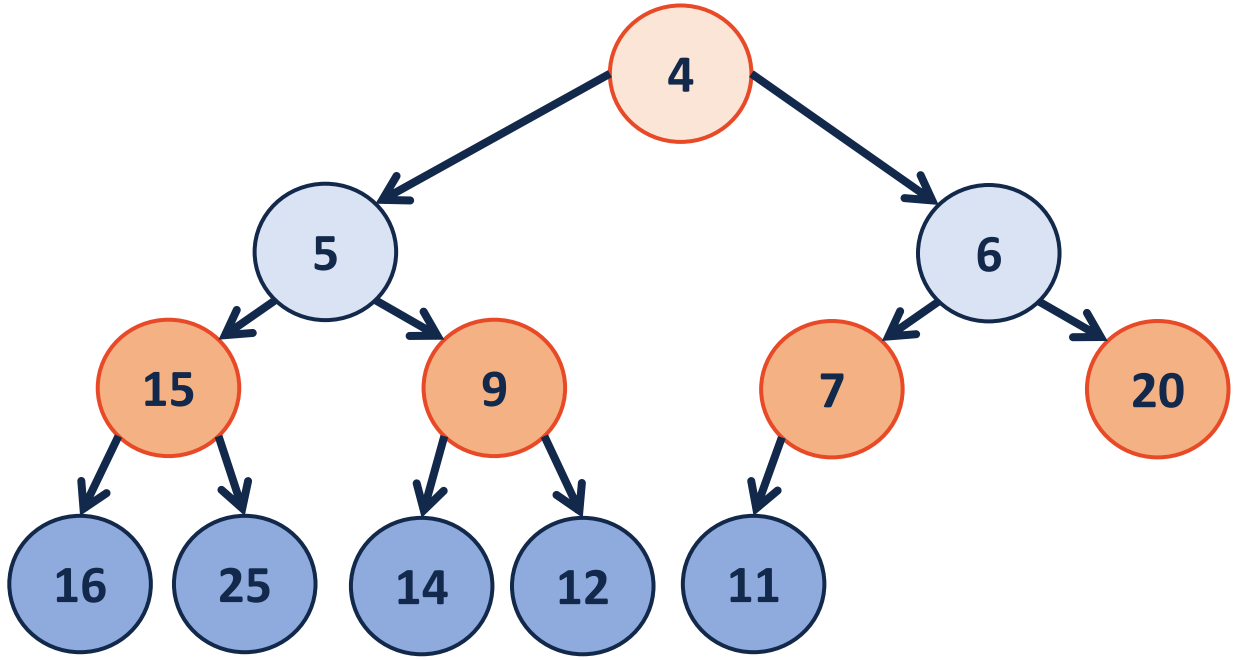
- **T = {}** or
- **T = {r, T_L, T_R}**, where **r** is less than the roots of {T_L, T_R} and {T_L, T_R} are min-heaps.



(min)Heap

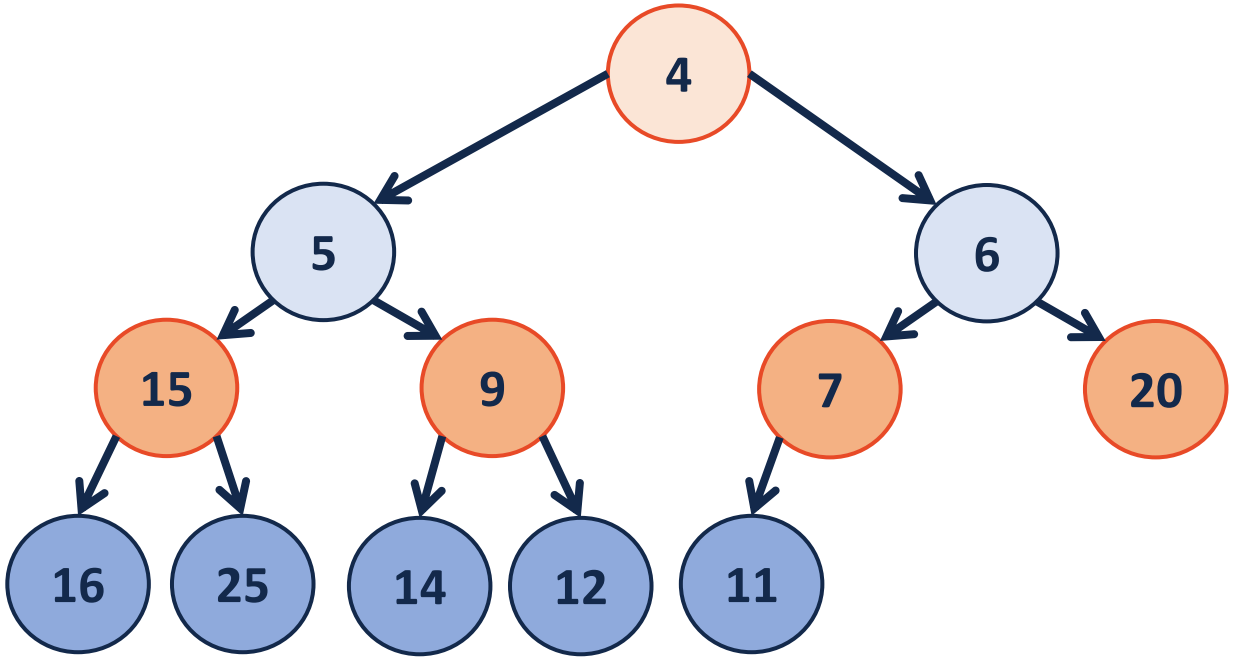
A complete binary tree T is a min-heap if:

- **T = {}** or
- **T = {r, T_L, T_R}**, where **r** is less than the roots of {T_L, T_R} and {T_L, T_R} are min-heaps.



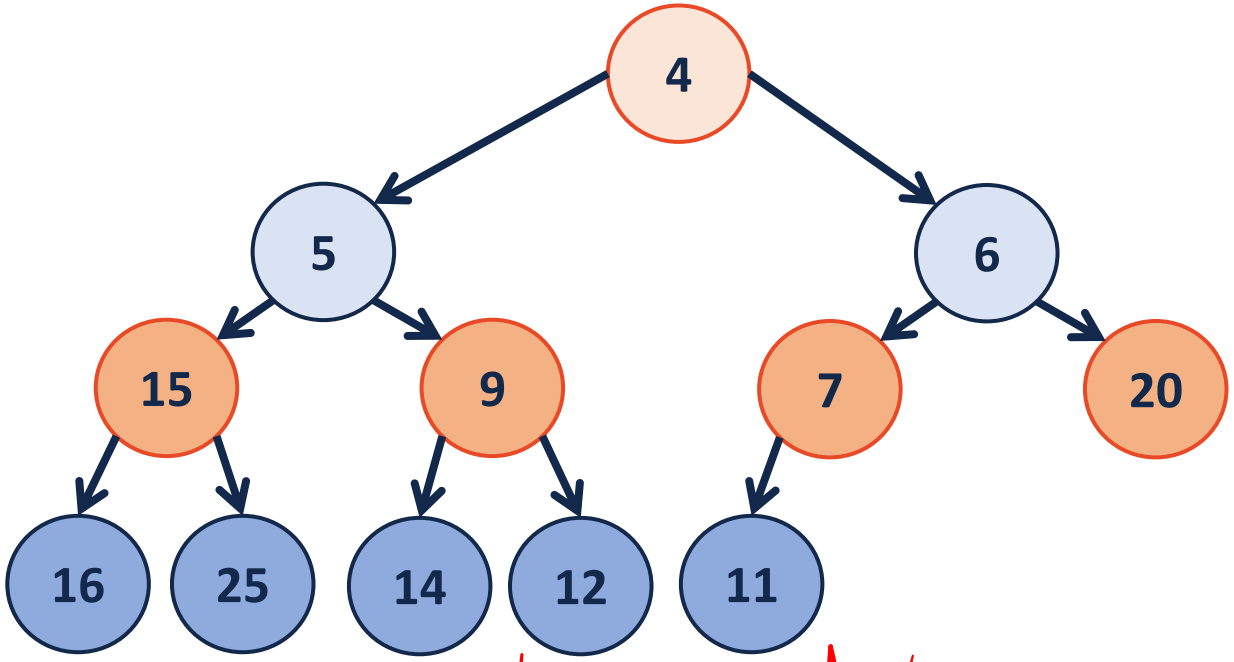
visual model of implementation

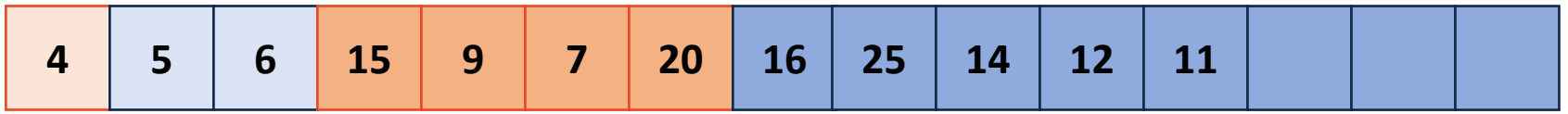
(min)Heap



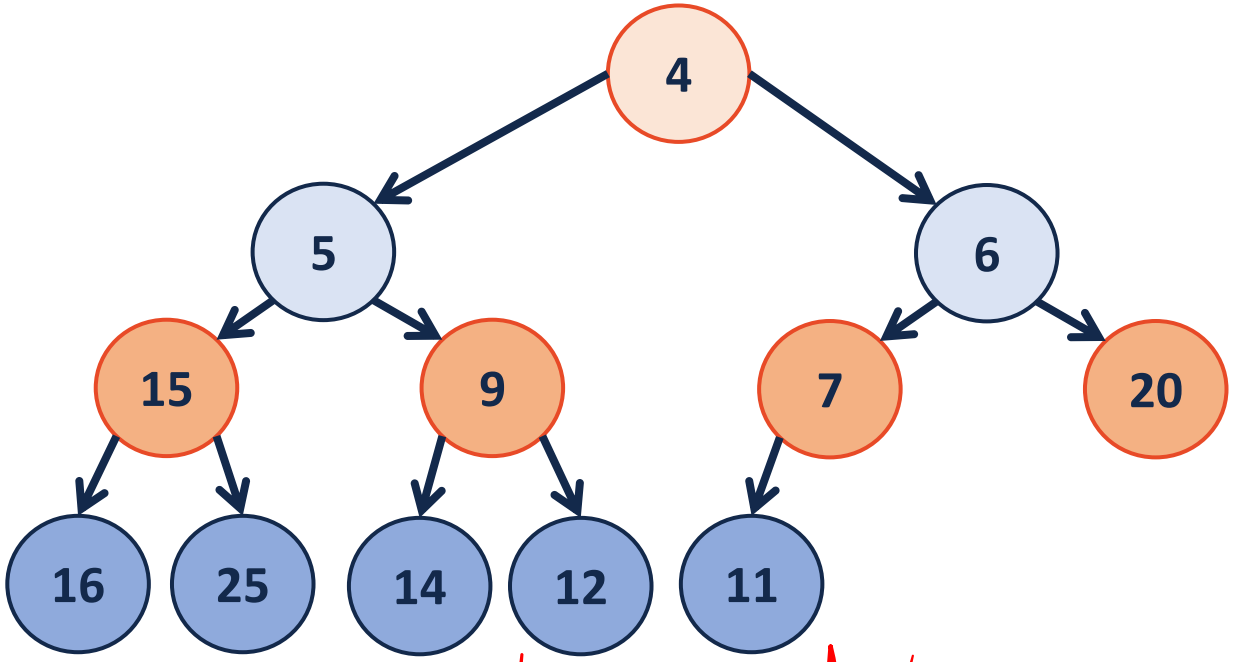
4	5	6	15	9	7	20	16	25	14	12	11			

(min)Heap





(min)Heap

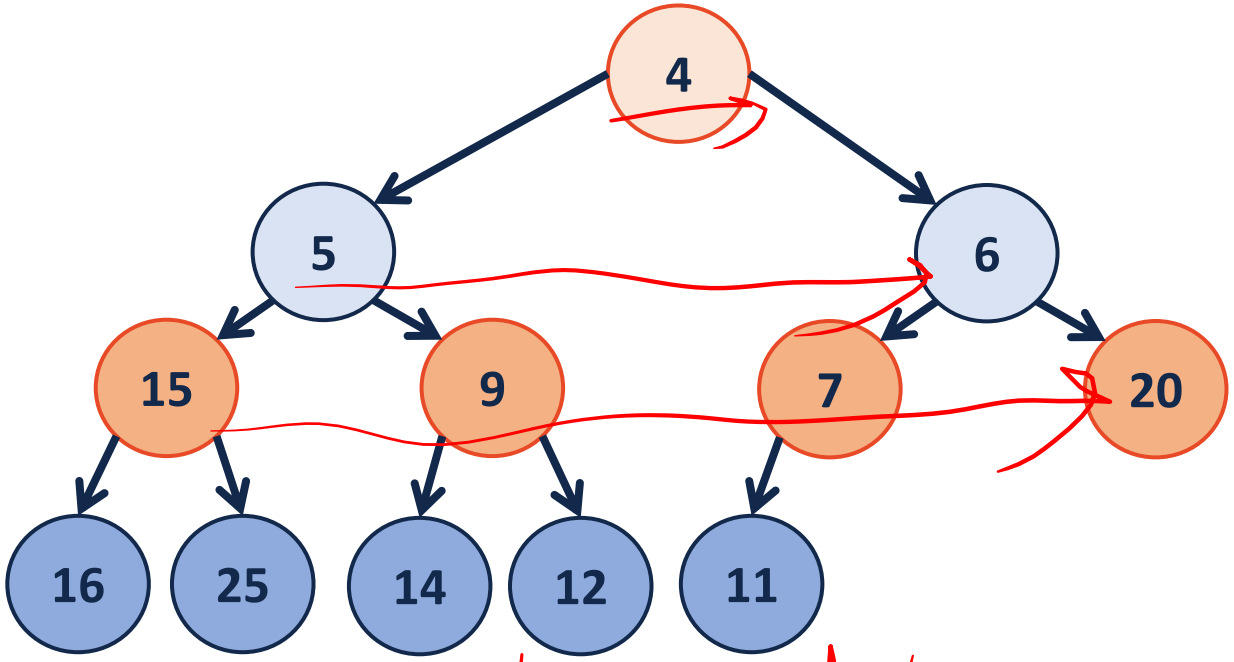


visual model

4	5	6	15	9	7	20	16	25	14	12	11			

impl

(min)Heap



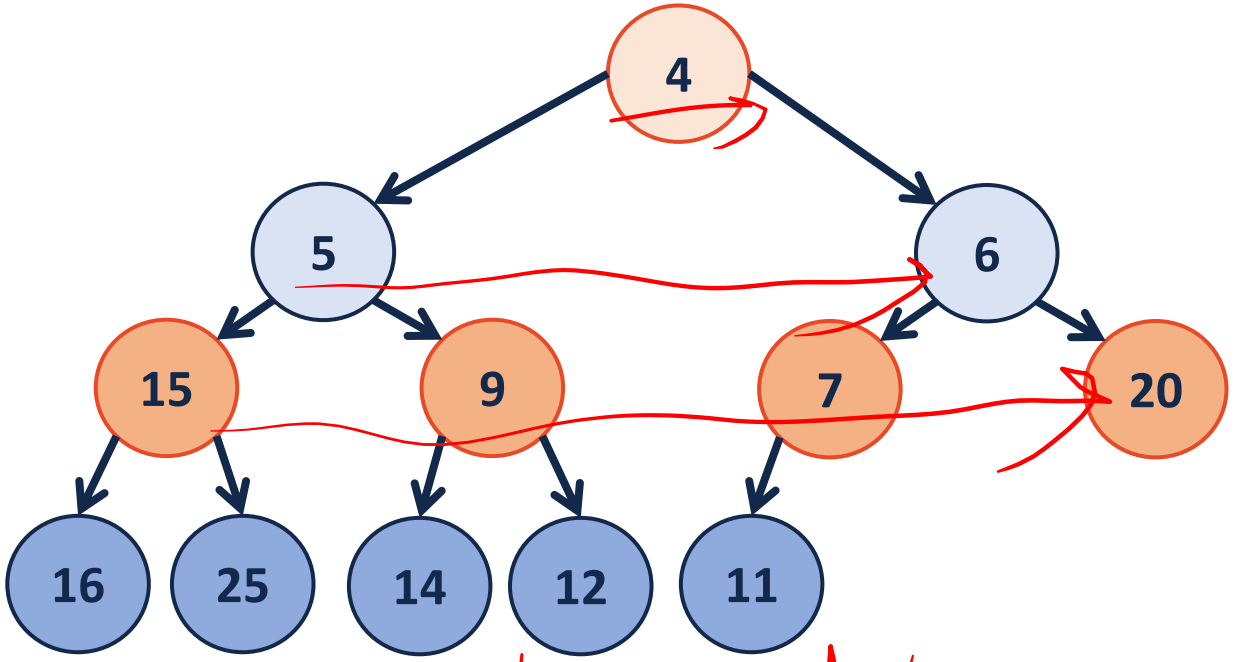
visual model

4	5	6	15	9	7	20	16	25	14	12	11			

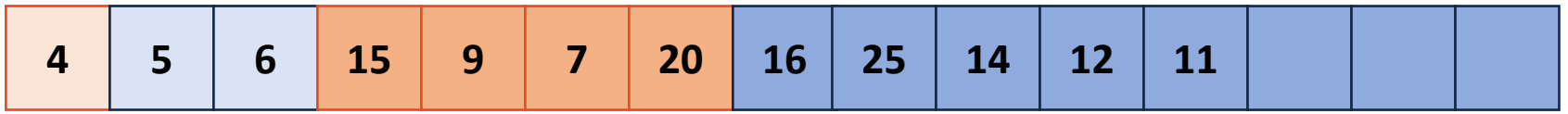
impl

(min)Heap

Level order TRAV



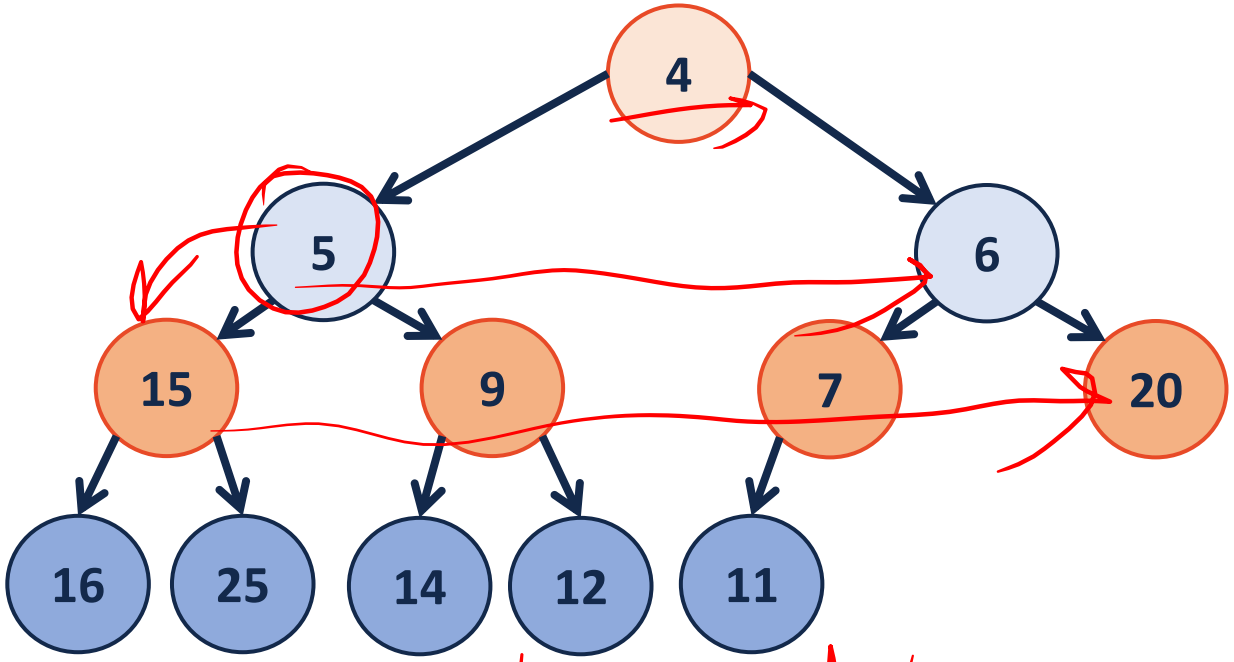
visual model



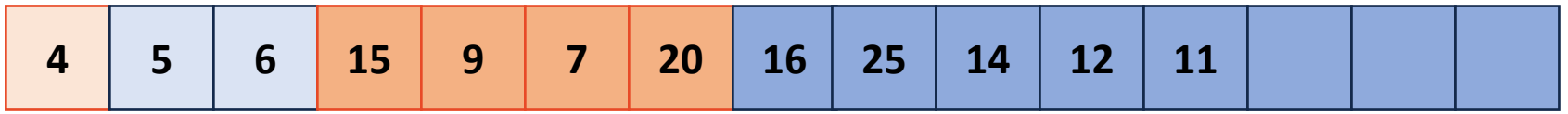
impl

(min)Heap

Level order TRAV



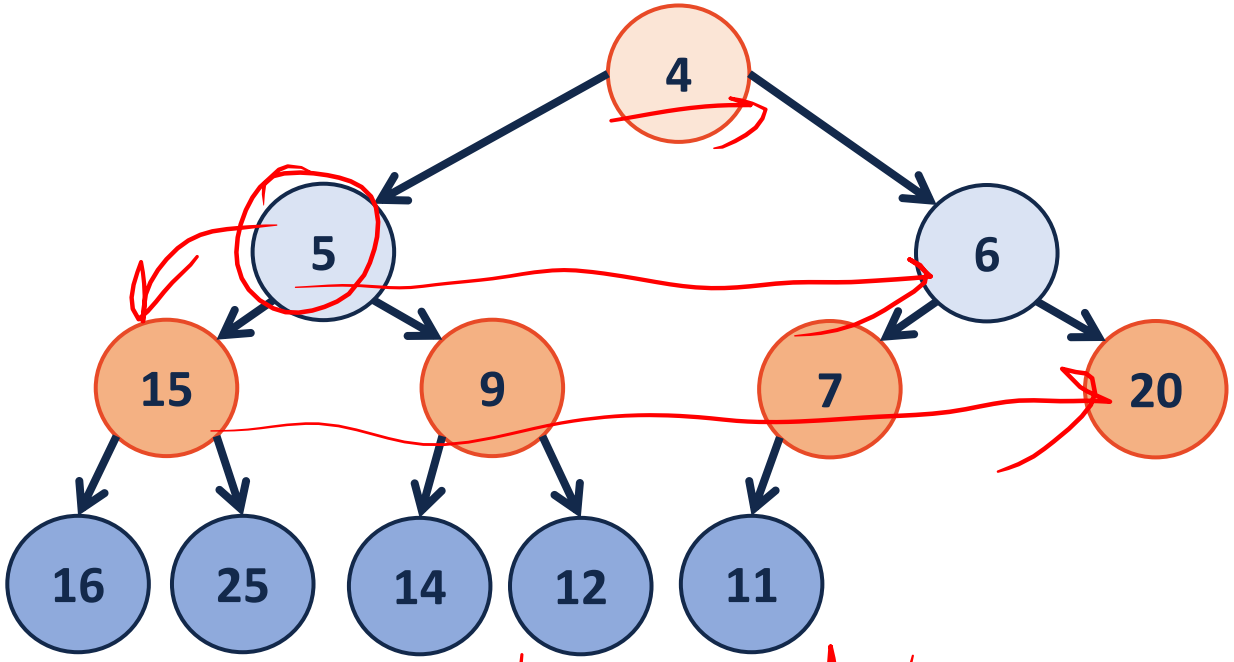
visual model



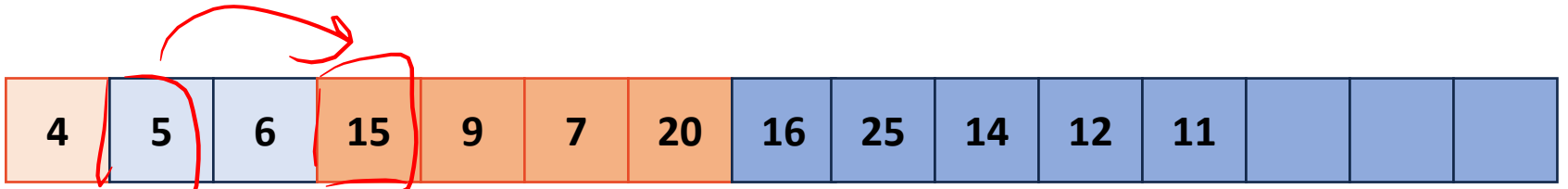
impl

(min)Heap

Level order TRAV



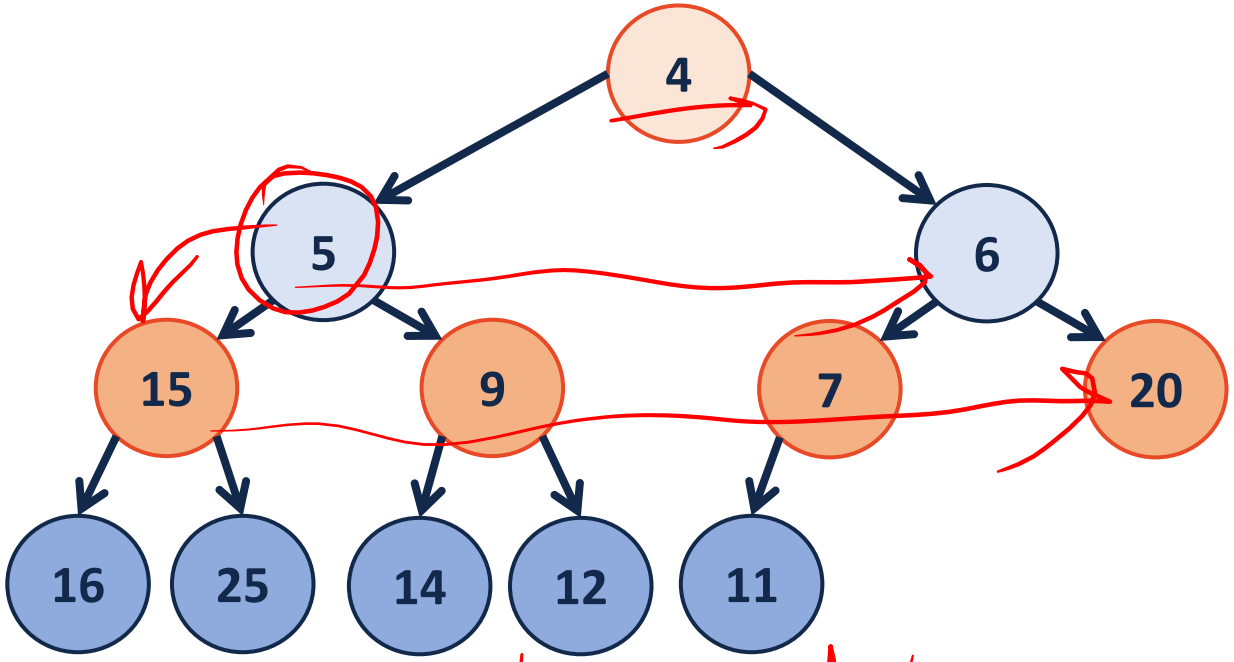
visual model



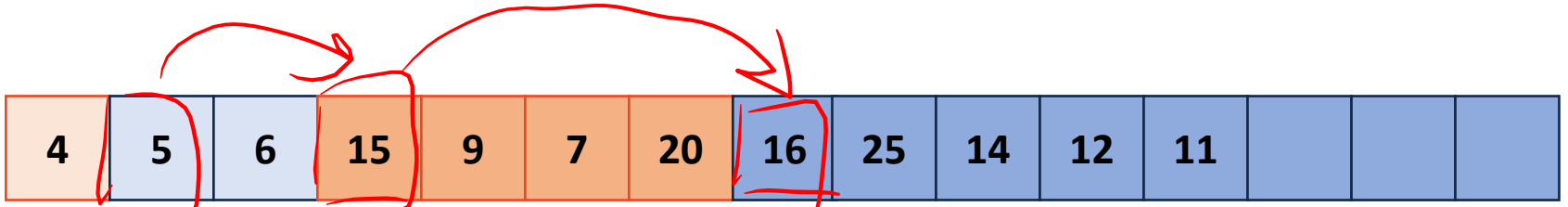
impl

(min)Heap

Level order TRAV



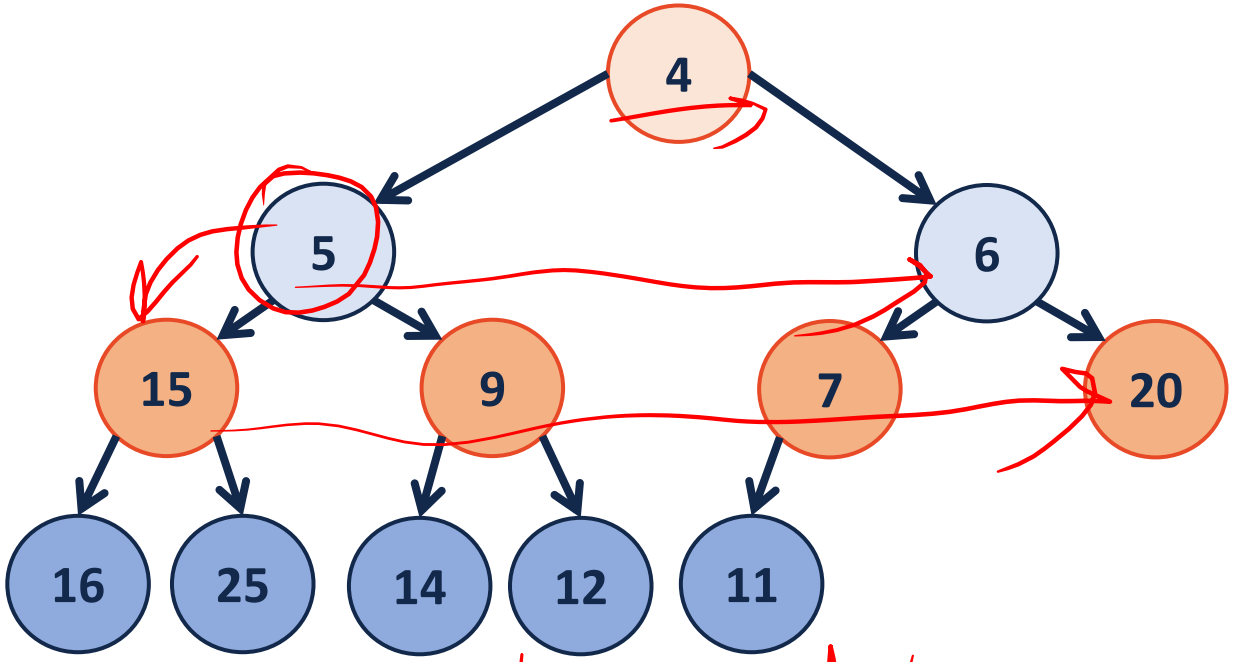
visual model



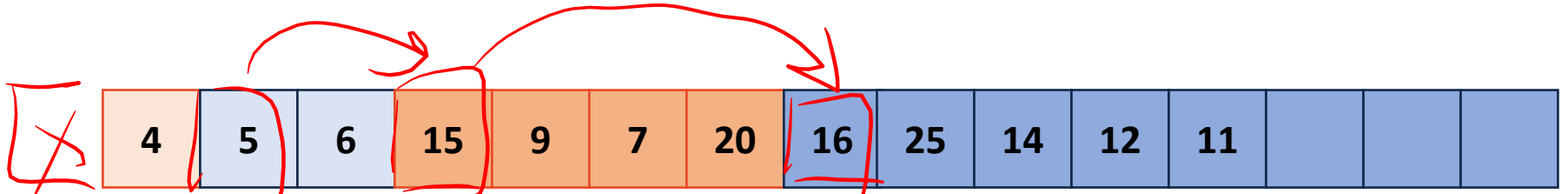
impl

(min)Heap

Level order TRAV



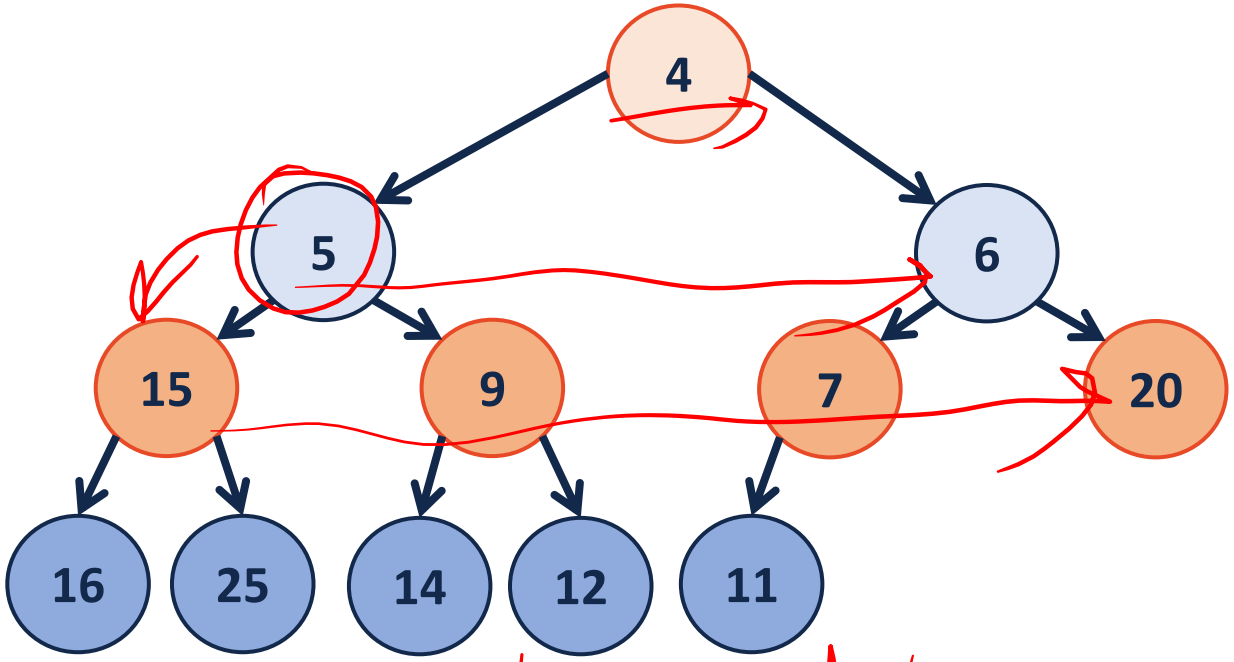
visual model



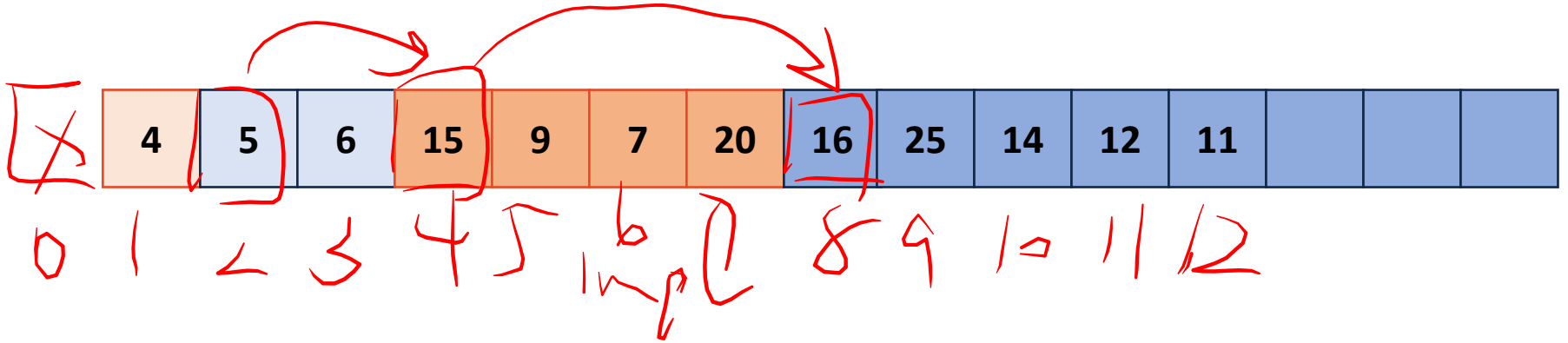
impl

(min)Heap

Level order TRAV



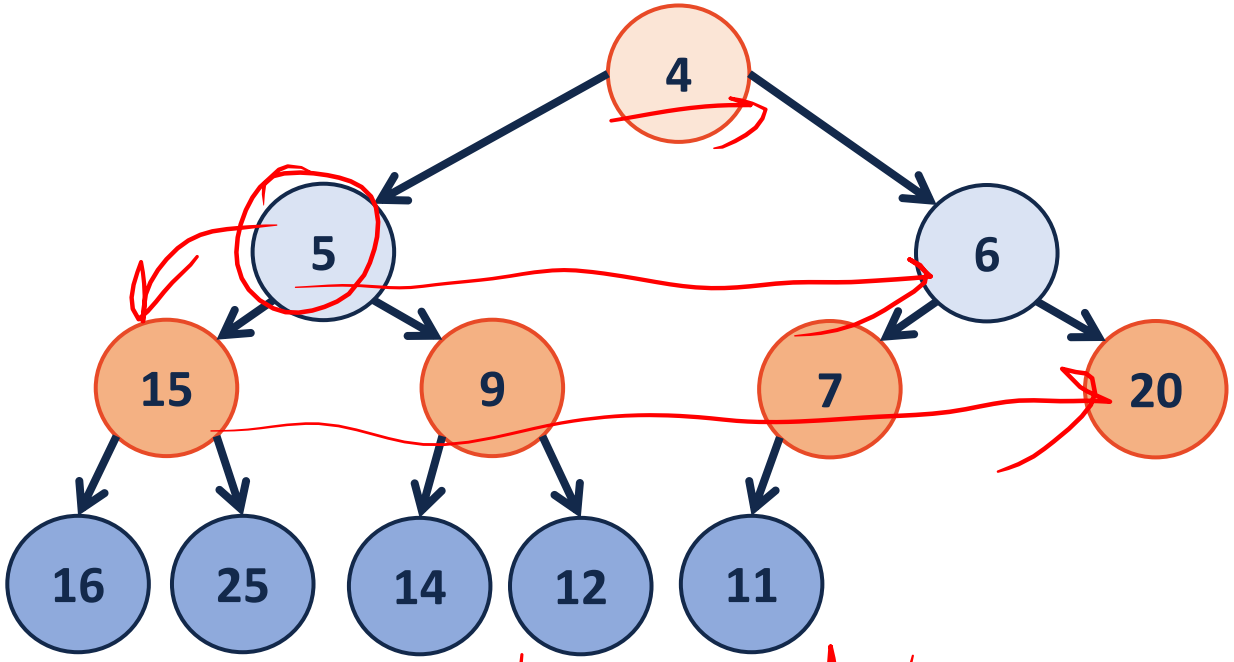
visual model



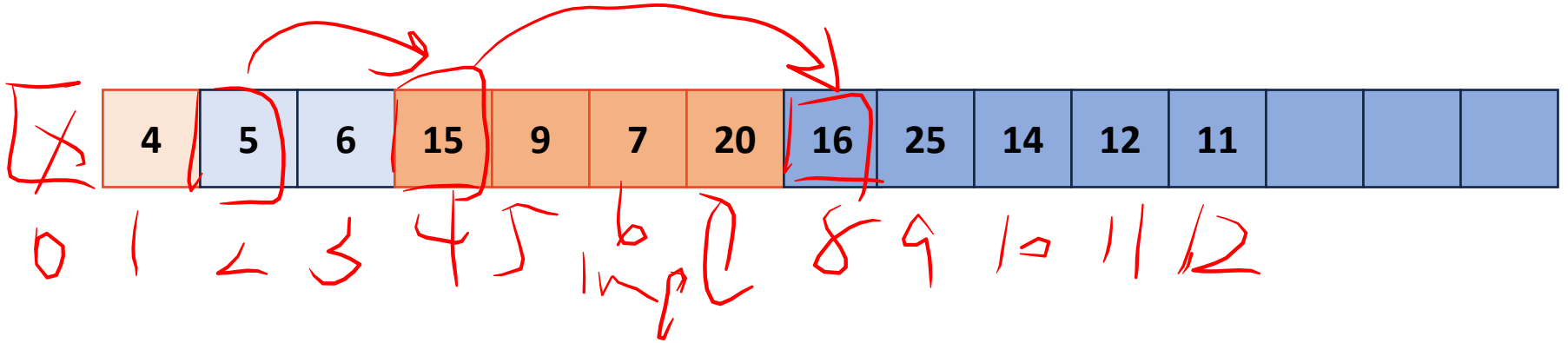
(min)Heap

Level order Trav

* In a heap impl [0] is not used



visual model

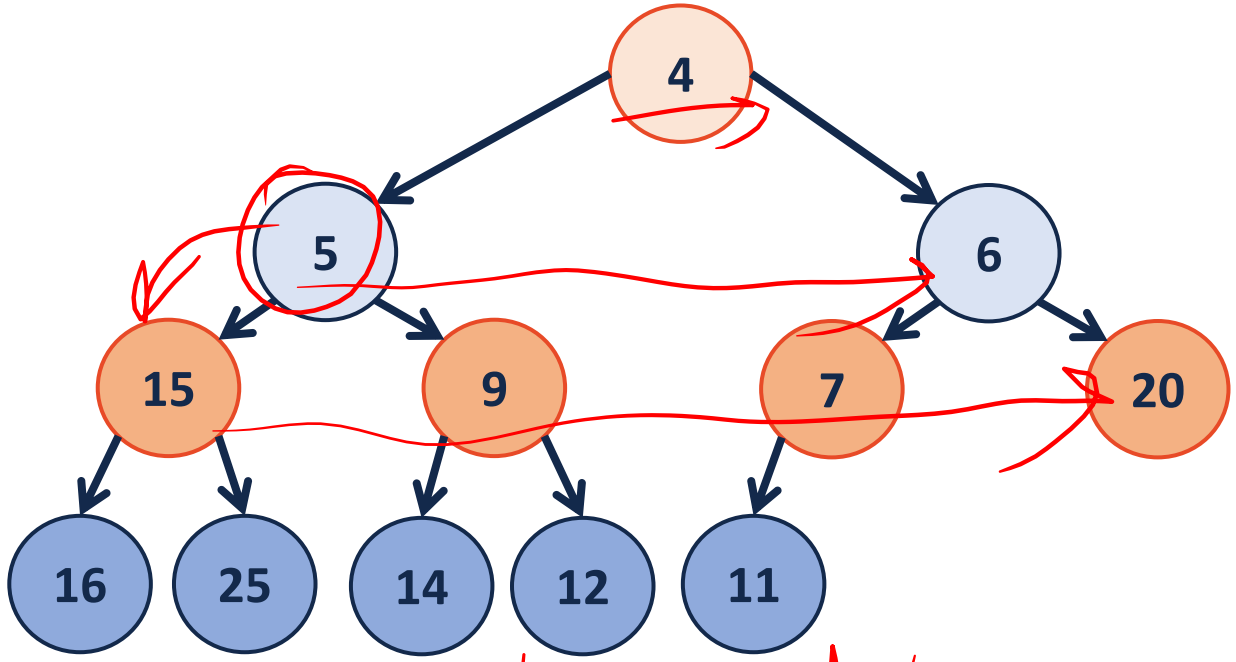


(min)Heap

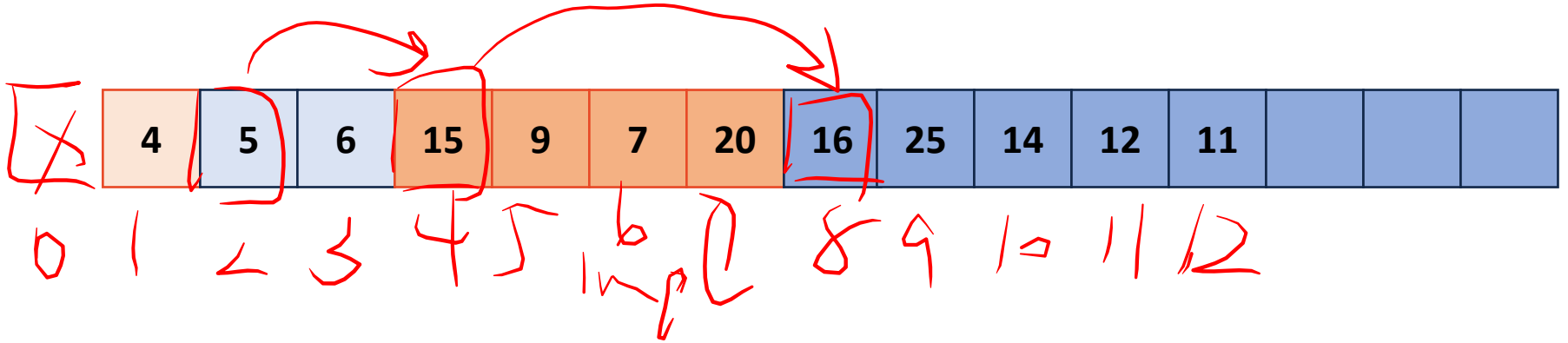
Level order Trav

* In a heap impl [0] is not used

Left child = 2 * index



visual model



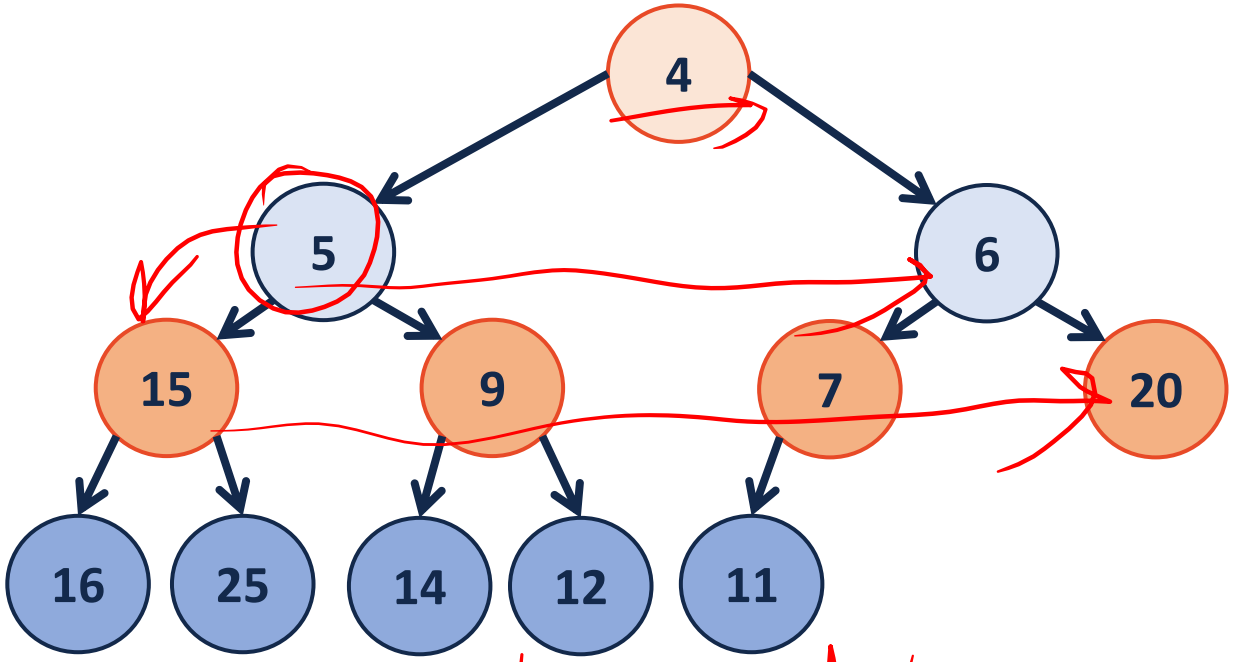
(min)Heap

Level order Trav

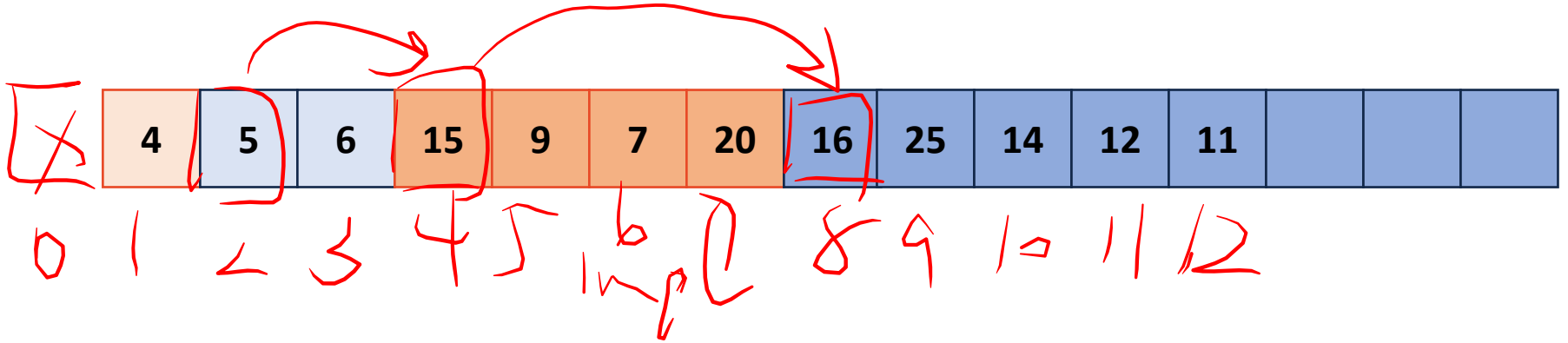
* In a heap impl [0] is not used

Left child = 2 * index

Right child = 2 * index + 1



visual model



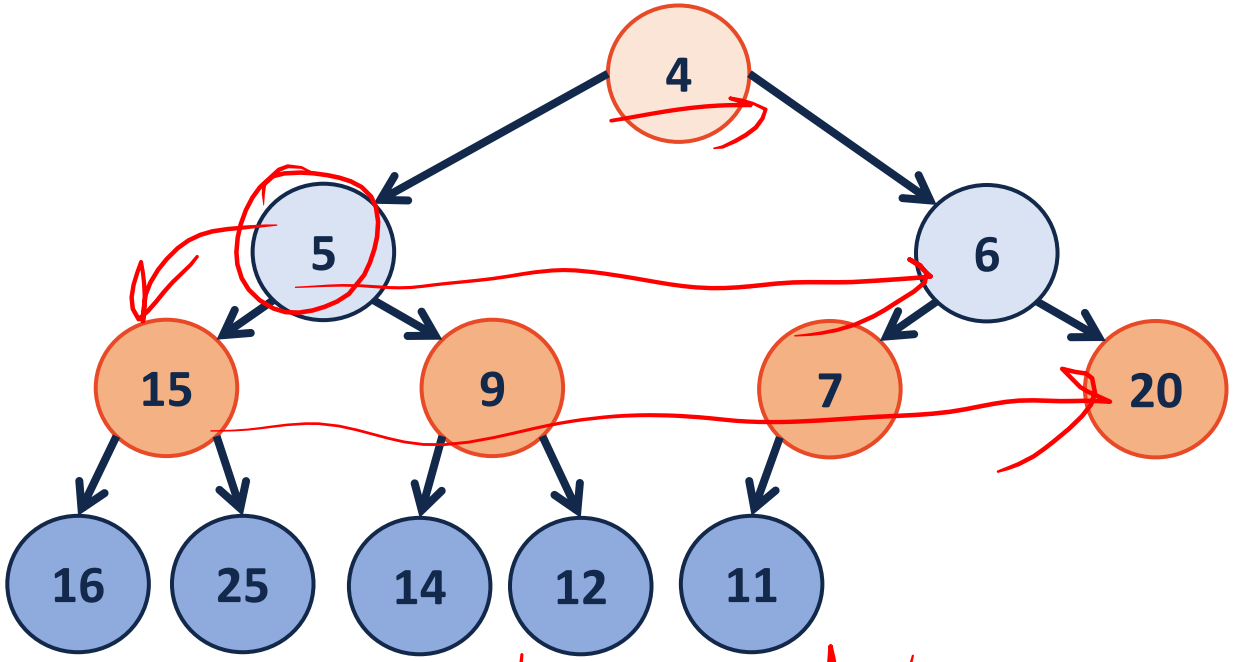
(min)Heap

Level order Trav

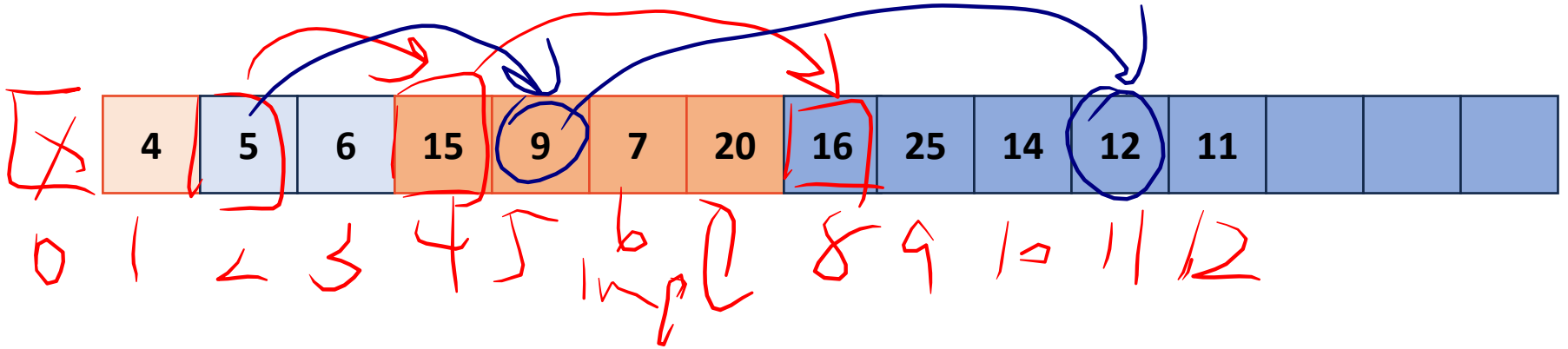
* In a heap impl [0] is not used

Left child = 2 * index

Right child = 2 * index + 1



visual model



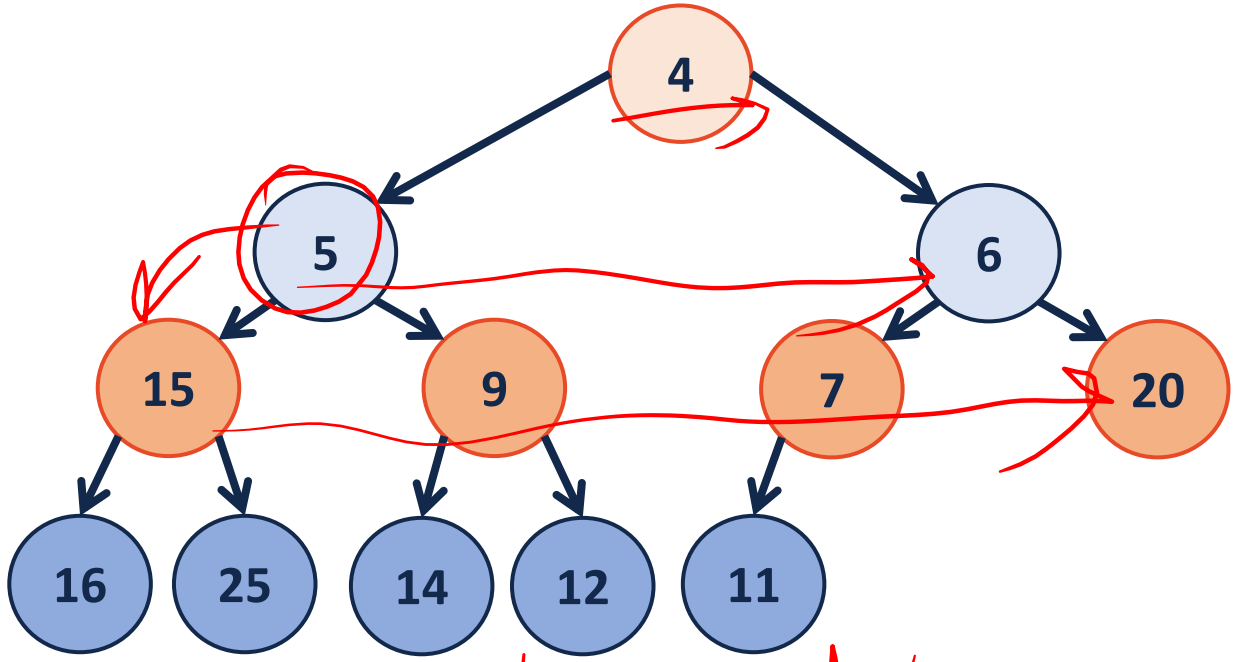
(min)Heap

Level order Trav

* In a heap impl [0] is not used

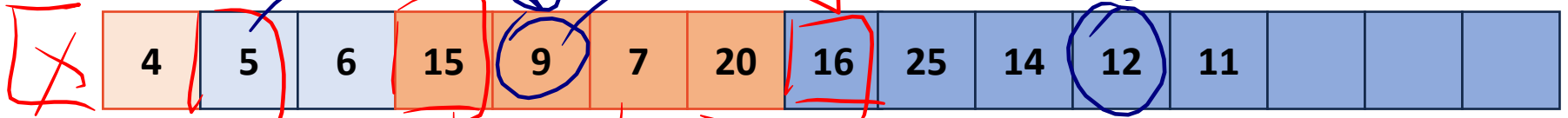
Left child = 2 * index

Right child = 2 * index + 1



Visual model

Parent = ⌊index/2⌋



0 1 2 3 4 5 6 7 8 9 10 11 12

impl

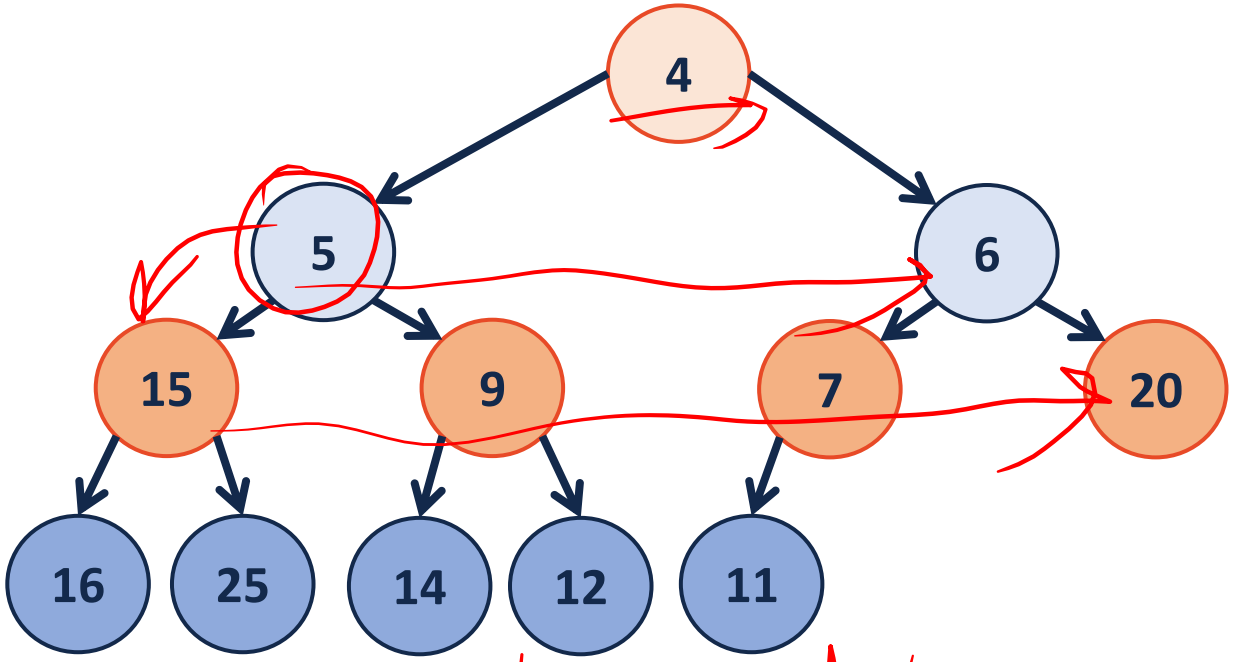
(min)Heap

Level order Trav

* In a heap impl [0] is not used

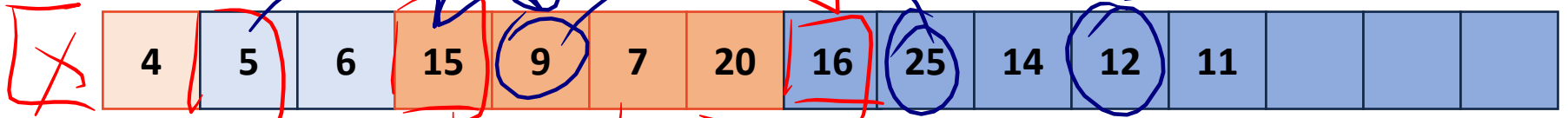
Left child = 2 * index

Right child = 2 * index + 1



Visual model

Parent = ⌊index/2⌋



0 1 2 3 4 5 6 7 8 9 10 11 12